00 · How to read this

01 · Standards

Dropdown sources

02 · Development plan

Phase 0 · Foundations

Phase 1 · Catalogue

Phase 2 · Report

Phase 3 · History

Phase 4 · Schedules & dashboard

Phase 5 · Build your own

Risks

03 · The screens

The four tabs

TAB 1 · CATALOGUE

1·1 Standard

1·2 Custom

OPENED FROM A CARD

2·1 Report · view

grouped · drilled · filters · refused

2·2 Report · edit

2·3 Schedule

2·4 Save as

TAB 2 · DASHBOARD

3·1 Dashboard

TAB 3 · SCHEDULED

4·1 Scheduled

4·2 Schedule opened

TAB 4 · HISTORY

5·1 History

5·2 Run opened

ELSEWHERE

E·1 Shell dashboard

E·2 Reports settings

REMOVED

– Account ledger

– By-premises

– Customer statement

THE REPORTS

45 The catalogue, drawn

×4 Statement & Ledger

BUILD BRIEF · SMARTBILLHUB · REPORTS

Reports, as it will be built

Thirteen screens across four tabs, six delivery phases, and a reporting framework in which **datasets are code and definitions are data** — so 46 reports are 46 rows, not 46 screens, and a utility can write the forty-seventh itself.

4	2	13	21	10	2	1	3	45	51	9
TABS	SUB-TABS	SCREENS	DRAWINGS	NEW	UPDATE	KEEP	REMOVED	CARDS	REPORT DRAWINGS	DATASETS

6	3
PHASES	DIAGRAMS

> **The read.** Branch `claude/gallant-goodall-22zcj5` at `c9aab1800`, which is also the tip of the repository's default branch. Version **2.340.0**. The running product was booted — PostgreSQL, Django, the seeded `brazosbend` tenant, the built SPA — and every existing screen was walked in Chromium at 1440.
>
> **The drawings are pictures of a light product.** This document follows your theme; the drawings do not. Every colour inside a drawing frame is pinned to its shipped value, read from `styles/tokens.css` and verified in the browser: accent `#ec3013`, ground `#f3f2f2`, surface `#eae9e9`, ink `#201e1d`, Archivo at 800, a 4/8/12/16/20/24/32 spacing scale, and **every corner square** — all three radius tokens are `0px` on purpose.

Three corrections to this document's own earlier draft, in the place they were wrong.

An earlier version of this page counted **84 reports** and **41 capabilities**. Recounted from its own tables: **95 requirement rows** and **42 capabilities**. Both numbers were quoted in three further places and all three were stale. Caught by re-deriving every number in prose rather than re-reading it — which is why that check is now in the publish list at the foot of §01.

An earlier draft also drew the download control as a labelled `↓ Download CSV` button. **The shipped component renders an icon, 36×36, with no label** — `DownloadButton.tsx:49-51`, `variant="secondary"`, `h-9 w-9 px-0`. The drawing showed the rule instead of the code, which launders a violation into something that looks shipped. Every drawing below is taken from the component.

And the third is the one that would have caused real damage. I read A1's screen inventory, which says the dashboard offers **four** period windows **defaulting to the last completed cycle**, cited `BR-REP-035` and `CR-015`, found the running product offering three and defaulting to *Last 30 days*, and wrote it up as a confirmed ruling the code had missed — as the **first task in the plan**. **It is the other way round.** `CR-029` D8① withdrew that window on 23 August; the PO's words were *"dont build it"* (`OQ-268`). The code has been right ever since and **A1 is a month stale**. Building my first reading would have re-added a feature the PO declined, on a real citation, past every check. It is now **P0.15**, and it amends the document. ⚠ **The lesson is the repo's own standing warning arriving in practice: where A1 contradicts a later ruling, A1 is stale — file it, do not follow it.**

What the PO changed after reading the first version of this document, and it is most of the shape.

Tabs. The Reports screen had no tab strip at all and this document never said what it would have. It is now **four, flat** — Catalogue · Dashboard · Scheduled · History. **Activity is deleted**, the word and the address; nesting Scheduled and History inside it bought nothing and Finance and Payments are both flat at six.

Custom reports. They had no home. Origin was going to be a *filter* on the catalogue, and a filter dropdown hides that you can author a report at all. It is a

04 · Journeys

05 · Cross-module

06 · Register

07 · The decisions

Dimensions & the cube

Scale

Security

Integration & BI

Decision register

08 · How it is built

The stack

Two read paths

Copy-on-write

Custom reports. They had no home. Origin was going to be a *filter* on the catalogue, and a filter dropdown hides that you can author a report at all. It is a **sub-tab** — **Standard · Custom** — so an empty sub-tab reading **0** is how a clerk learns the feature exists.

The dashboard. Ruled verbatim: *"no i dont want to update existing dashboard ... you can create a tab - dont replace existing one"*. The shell dashboard is **fenced off and untouched by every task in this plan**; Reports gets its own, as Tab 2, answering a different question for a different person at a different time of the month.

The finance statement. Wanted by account (regular and master), by customer, by premises and by unit. **Three of those four already exist as three separate bespoke pages** and the fourth was never built. They collapse into **two definitions with a Subject parameter**, drawn four times each.

The builder. *"user cannot add columns"* — right, and two thirds of it was mine to fix. Columns can now be **reordered and renamed**, and an unticked column is an empty box rather than greyed out. What stays true is that you cannot invent a column the dataset does not produce, and the panel now says how many it has.

00 How to read this

Every statement in this document is one of six things, and carries a chip saying which. A build lane reads this page *instead of* a conversation, so nothing here is left to tone of voice.

MARKER	MEANS	WHAT A LANE DOES WITH IT
TASK	Committed work, carrying a plan id `Pn.m`	**Build it.** It is in the plan and in the manifest
TODAY	What is true in the code now, with a file and line	**Do not re-derive it.** Open the file if you doubt it
GAP	A defect or a missing piece	**Information only**, unless it also carries a TASK
RULED	A Product Owner decision, with its date	**Binding. Never re-ask.**
OPEN	Nobody has answered, with the question's id	**Never resolve it by choosing a value.** Stub with the id
SOURCE	Where a control's values come from	See the register in §01. **No drawing here authorises a literal array**

A description commits nobody.

A GAP says something is missing. It is not an undertaking to build it. **Only a TASK is a commitment**, and every one of them appears in the development plan in §02 with an id, a reference and a checkable outcome. If a thing is not in a task row, nobody has undertaken to build it.

A route existing is not a door.

Almost every test in this repository asks whether a screen renders at its own address — the one place a door is not needed. Nothing asks whether a user standing somewhere else can get to it. Every drawing below carries a **Doors** row naming the screens that link to it, verified in the source, or the word **NOTHING** said outright. **No plausible door is ever supplied.** The current product has three screens that fail this, and all three are named in §04.

The manifest

A machine-readable index of every task, screen, dropdown source, ruling and open question sits

in a `<script type="application/json" id="blueprint-manifest">` block at the top of this file. **It is generated from the assembled HTML, never written by hand** — a hand-kept index is a second source of truth and it drifts. Generating it from the markup caught two real drifts in this rewrite that reading did not: a register row pointing at a screen id that no longer existed, and a task referenced in prose that had no row in any phase table.

If a task has no id or a screen has no class, it is absent from the manifest and its absence is visible. That is the point.

01 Standards

A violation is only a violation against a stated authority. Three rule sets apply, in this order of precedence, and every drawing below is judged against them.

AUTHORITY	WHAT IT IS	HOW IT IS USED HERE
B14 `04-docs/02-decisions/14-ui-conventions.md`	The binding UI rules, numbered 1–21 and 31–43. Its own status table is the count	**The authority.** Every drawing's **Rules** row names what it obeys and, with ✗, what it breaks
r10 `05-prototype/r10-2026-08-16/UI-RULES.md`	Twenty rules in five groups. B14's *source*, not its authority	The table-toolbar vocabulary, rules 12–17
The kit the repo's own components	`DataTable` (905 lines) · `WizardFrame` (42 consumers) · `DownloadButton` · `FormModal` · `KPIStrip` · `StatusPill` · `PageHeader`	**Draw the component, never your own version of it.** Where a drawing shows a table, it is `DataTable`

The two mistakes that cost the previous draft most.

1 · Draw from the code, never from the rule. The earlier draft drew a labelled download button because the rule says a control should say what it does. `DownloadButton.tsx:49-51` renders a 36×36 secondary icon with no label. An as-built drawing that shows the rule instead of the code **launders a violation into something that looks shipped**. Where the code breaks a rule below, the drawing shows the code and the breach is filed in the **Rules** row with a ✗.

2 · If the product has a component, draw that component. The earlier draft hand-drew a toolbar with filter chips and a `+ Filter ▾` control. `DataTable.tsx:324-328` records that those chip rows were **deliberately removed** after a PO review, and that row one — search, the export slot and `▦ Columns` — *"is the whole toolbar"*. A screen wanting another dimension declares it in `TableConfig.filters` and passes `filterBarSlot`; it does not grow a row. Every table below is drawn as that component renders.

The builder is a panel, not a wizard.

RULED

Put to the Product Owner on **2026-09-26** and answered **"panel"**. This breaks a 42-file convention and was raised as such. `WizardFrame` gives a numbered rail, a forward fence and **a single transactional commit at the end** — the right shape for a linear task. A report builder is **iterative**: change a column, see the rows change, change it back. A forward fence fights that, and commit-at-the-end means you cannot see your report until you have finished describing it. **So 2·2 is an edit panel beside the live table — the preview *is* the viewer.**

Everything else multi-step keeps the shared frame. The full list is in §02 under Phase 4.

How every drawing is made

- **Desktop only. One 1440px window, scaled as a unit, never reflowed.** `scale = container.clientWidth / 1440` , capped at 1, floored at 0.42; height reserved as `offsetHeight × scale` ; measured from `documentElement.clientWidth` , **never** `100vw` , which includes the scrollbar.
- **One kit for all fourteen.** Rail, topbar, page head, tab strip, toolbar, table, pager, KPI band, field, two-column grid, modal, panel, edit panel, consequence box — declared once, shared by every drawing, so no two screens have a different geometry for the same thing.
- **Colours are the shipped values, measured.** Not `var(--…)` resolved by your theme — literals, so a picture of a light product stays a picture of a light product in a dark client.
- **A screen appears once.** There is no as-built drawing and a separate remedy drawing. Each screen is drawn **at its end state**, with every difference from today marked on the drawing and listed under it as NEW or CHANGED.

The dropdown source register

One row per value-bearing control in the whole document. **NO DRAWING IN THIS DOCUMENT AUTHORISES A LITERAL ARRAY IN CODE.** Where a control has no source, that is the finding — not the design.

#	CONTROL	SCREEN	CLASS	SOURCE, AND WHAT A LANE MUST NOT DO
1	**Report group** — the six	1·1, 2·2, E·2	NOT SERVED	**The literal the framework exists to remove.** Today three Python constants (`catalog.py:83-85`) plus a **closed TypeScript union** (`reportsEndpoints.ts:42`). **P0.2** creates `report_categories` ; nothing seeds a fourth constant
2	**Sub-group** — the nineteen	1·1, 2·2, E·2	NOT SERVED	Same table, parented. **P0.2**
3	Service filter	1·1, 2·1, 2·2	SERVED	`tenancy.selectors.enabled_service_codes` . **The tenant's own enabled set, always.** The prototype's hardcoded Electric/Water/Sewer trio appears nowhere and must not return
4	Origin — built-in / mine / shared	1·1	SIGNED	Three values, closed. `BR-REP-019` RULED — creator-private by default with an explicit share toggle. A tenant cannot add a fourth
5	Bill type	2·1	SERVED	`billing.reporting.BILL_TYPES` , read live. **8 options today** — measured in the browser
6	Payment purpose	2·1	SERVED	`payments.models.PAYMENT_PURPOSES` . ⚠ reached today by a cross-module import at `queries.py:668` rather than through a seam — **P0.5**
7	Consumer class	2·1	SERVED	**The server already reads this** (`queries.py:255`) and **no control is drawn for it**. A served parameter with no control. **P2.6**
8	Rows per page	2·1,	SIGNED	50 / 100 / 200 / 500 / 1000 — B14 rule 16 names the set. `DataTable` owns it; do not mint a sixth

		4·1, 5·1		
9	Column chooser	2·1	DERIVED	The definition's own column list. No list to hardcode
10	Dataset picker	2·2	SERVED	The **9** registered datasets, from the metadata endpoint. **P0.4**
11	Column / filter / group-by / sort pickers	2·2	DERIVED	The chosen dataset's declared columns and filters, with their types and units. **A dataset that declares nothing offers nothing** — that is the contract, not a bug
12	Visibility — private / shared	2·2, 2·4	SIGNED	`BR-REP-019` RULED
13	Schedule cadence	2·3, 4·2	SIGNED	Daily · weekly · monthly · **on cycle close**. The fourth is the one a utility actually wants and the one a generic cron picker loses
14	**Export format**	2·1, 2·3	SIGNED	⚠ **A signed enum with a deliberate hole. CSV only.** PDF is absent because **no renderer library is declared anywhere in the product** (`exports.py:51-57`). Adding PDF to this list puts a control on screen that produces nothing. OPEN
15	**Schedule recipients**	2·3, 4·2	SERVED	⚠ **The second trap.** The tenant's own users — **internal staff only in v1**. `BR-REP-020` RULED: external recipients arrive at W6 with an explicit confirm step, because *external send = data leaves the tenant*. **A free-text email field here breaks a signed ruling**
16	Job status	4·1, 5·1	SIGNED	`core.jobs.STATUSES` . The `jobs` table already owns it
17	Report picker	4·1, 5·1	SERVED	`report_definitions` , scoped to this tenant
18	Minimum role	E·2	SERVED	The tenant's own roles, from the server's permission grid. **Never a client-side role-name check** — `BR-SHL-025`

The literal that matters most is not in that table — it is the catalogue itself.

TODAY

`apps/reports/migrations/` holds only `__init__.py` . The 23 reports the product ships are a **frozen Python tuple of literals** at `catalog.py:133-410` , identical for every tenant, and the server sends **no title at all** — every report's name lives in **eleven locale JSON files**. Adding one report today means hand-editing about **17 files** across six maps in two languages. **Until a report's name comes out of a row, no utility can ever author one**, because a utility cannot edit our locale catalogues. That is what **P0.2** and **P0.3** exist to end.

Before this page is published

- No horizontal page scroll at 1920 / 1440 / 1024 / 768 / 500 / 400, in **both** themes. No JS errors.
- Every nav link resolves to an id that exists; every screen in the register has a nav entry.
- **Regenerate the manifest from the assembled file** and make the counts reconcile.
- **Grep every number stated in prose and re-derive it.** Two were stale in the last draft.
- Screenshot the drawings that changed and look at them.

02 Development plan

Six increments, each shippable on its own. **No dates** — the order is set by what depends on what, and by putting the cheapest value first. Every task carries a reference to open and an outcome that can be checked rather than asserted. A ⚠ marks a decision or a hazard to settle before code is written.

Every multi-step task in this module, and whether it uses the shared frame — with one correction.

The brief asks for this list before any redraw. Here it is, and **row two is a correction of my own earlier answer.**

TASK	STEPS	SHARED WIZARDFRAME ?
Build a report (2·2)	4–5	**No — an edit panel.** RULED 2026-09-26. Breaks a 42-file convention, raised as such. A forward fence fights an iterative task
Create a schedule (2·3)	**1**	**No — and my earlier answer said it should use the frame.** That was wrong. You schedule *from* a report you are already looking at, so the report and its parameters are already chosen: what is left is cadence, format and recipients. **Three fields, one form.** A three-step rail over three fields is ceremony
Export → download	3 machine hops	No, correctly. A button plus the one shared job poller
Migration assurance (R41)	2	No. Two report definitions, not a flow

So no flow in Reports uses the shared wizard frame — the module has no multi-step user task at all. That is worth knowing before someone reaches for one.

Phase 0 · Foundations

SHIPS	Three migrations, the dataset registry, the runner, the routes, and the 46 definitions seeded from today's literals
WHAT A CLERK CAN NEWLY DO	**Nothing.** Said out loud: this is the only phase with no user value, which is why it is kept as short as it can be. Everything after it is user-visible
TASK MIX	3 DB · 4 Backend · 1 Front end · 0 Settings · 2 Test · 0 E2E · 2 Other-module
DONE WHEN	`GET /reports` returns the same 46 cards it returns today, **read from a table instead of a Python tuple**, and the six hand-typed maps are deleted

ID	KIND	WHAT	WHAT IT INVOLVES	REFERENCE	OUTCOME
P0.1	DB	`report_definitions`	`tenant_id` **nullable** (null = platform), key, name, `name_i18n` jsonb, dataset, columns[], filters, group_by, sort, `service_codes[]`, visibility, owner_user_id, version. RLS FORCE. Index on (tenant_id, dataset). ⚠ **Make** `visibility` **a CHECK constraint (two signed**	`apps/reports/migrations/` — today only `__init__.py`	A platform row and a tenant row coexist, and a tenant session sees

			values) and categories a TABLE. A CHECK makes a third value a migration — which is right for a signed enum and wrong for a tenant-extensible list		its own plus the platform's and nothing else
P0.2	DB	`report_categories`	`tenant_id` nullable, code, name, parent_code, sort_order. RLS FORCE. Seed the **6 groups and 19 sub-groups**	`catalog.py:83-85` — the three constants this replaces	A tenant inserts a category called **Board pack** and it appears in 1·1 without a release
P0.3	DB	`report_tenant_settings`	tenant_id, definition_id, enabled, pinned, display_order, name_override. RLS FORCE. Unique (tenant_id, definition_id)	—	A water-only tenant disables the gas reports and 1·1 stops showing them
P0.4	Backend	The dataset registry	**9 datasets**, each declaring its columns with type, unit and money-basis, plus its accepted filters. A metadata endpoint that serves the declaration	§05 · the 9 datasets	`GET /reports/datasets` lists 9, and every column carries a type and, where it is a quantity, a unit
P0.5	Backend	The runner	resolve definition → bind params → execute dataset → shape → hand to the renderer or the export writer. **One path.** Today the viewer and the export call the same query function separately, and the export calls it twice	`exports.py:94 · services.py:6202`	A report rendered on screen and the same report exported produce byte-identical rows
P0.6	Backend	Seed the 46	A data migration writing the 46 definitions, each mapped to its group, sub-group, dataset and parameters	§06 register	46 platform rows exist; the catalogue count reads **46**, not 23
P0.7	Backend	Delete the six maps	`CATALOG · VIEWERS · VIEWER_SEGMENTS · MONEY_COLUMNS · NUMERIC_COLUMNS · REPORT_FILTERS` . ⚠ `catalog.VIEWER_KEYS` is already dead — **no reader anywhere** — and its comment claims a guarantee nothing enforces	`catalog.py:412-425`	Adding a report touches **one row**. A grep for all six names returns nothing

P0.8	Front end	Routes	`/reports` · `/reports/custom` · `/reports/dashboard` · `/reports/scheduled` · `/reports/history` · `/reports/{key}` · `/reports/{key}?edit=1`. ⚠ **All four reserved segments must rank above** `{key}` or the catch-all swallows them and a user asking for History gets a report named “history”	`App.tsx:1530-1535` — the same ordering already solved once, with a comment saying why	All seven resolve; each reserved segment opens its own tab, not a 422
P0.9	Settings	**None — and that is a finding**	Phase 0 adds no Settings row at all. Stated rather than omitted, because a phase with no Settings task usually means somebody forgot to look. The Reports settings section lands in **P5.4**, and the full Settings picture is in §05	§05	Nothing in Settings changes, deliberately
P0.10	Test	RLS probes	⚠ **Postgres only.** The unit suite hardcodes in-memory SQLite and ignores `DATABASE_URL`, and SQLite has no row-level security — a probe there proves nothing and passes	`config/settings/test.py`	Tenant A cannot read tenant B's definitions, categories or overlay rows, **proved on PostgreSQL**
P0.11	Test	**Four** reserved segments rank above `{key}` — `custom`, `dashboard`, `scheduled`, `history`. ⚠ One was enough in the first draft; the tab strip made it four, and **each one is a word a utility could name a report**	Assert each of `/reports/custom`, `/reports/dashboard`, `/reports/scheduled` and `/reports/history` resolves to its own tab and **not** to the report viewer	P0.8	Fails by name if someone adds a route above it
P0.12	E2E	**No new journey**	Honestly: nothing a user does changes in Phase 0, so no journey file is added. The existing `US-REP-report-centre.spec.ts` must still pass unchanged	`07-frontend/e2e/`	The existing spec passes against the table-backed catalogue with no edit
P0.13	Other · payments	A seam for payment purposes	`queries.py:668` reaches into `apps.payments.models` directly. It reads a constant and no ORM, but it crosses the boundary outside the seam file	`report_seams.py`	No import of another module's `models` anywhere in `apps/reports/`
P0.14	Other · ledger	A seam for the two ledger reports	`api.py:95` imports `apps.ledger.reporting` directly, so a missing owner is an `ImportError` at module load rather than a named gap. **The module's two most important reports bypass its own refusal machinery**	`report_seams.py:96-111`	Both resolve by name at call time and degrade to a named gap
P0.	Docs	Amend A1's dashboard	A1's screen inventory describes **four** period windows **defaulting**	`04-docs/01-specs/01-`	A1 matches

P0.15	Docs	Amend A1's dashboard line	A1's screen inventory describes **four** period windows **defaulting to the last completed cycle**, citing `BR-REP-035` and `CR-015`. `CR-029` D8① withdrew that window on 23 August — PO verbatim *"dont build it"*, `OQ-268` — and the code has been right ever since. ⚠ **This document quoted the stale line and proposed re-adding the feature the PO declined.** Amend A1, and add the one assertion that stops the drift running the other way: **the control is drawn from `period_filter.options`, never from a front-end constant**	`04-docs/01-spec/01-screen-inventory.md:15` · `dashboard_period.py:95-108`	A1 matches the ruling · a test fails if the front end hardcodes the vocabulary
P0.16	Backend	**Declare the dimension model**	Every dataset states its **grain**, its **measures** with aggregation *and additivity*, and its **dimensions** with attributes and hierarchies. ⚠ **Metadata, not a feature** — a data structure beside each of the nine queries. It is what turns *"do not sum a balance across time"* from a comment into a refusal, and it is what lets a report inherit thirteen slicers instead of two being hand-picked	7.2 · `queries.py`	Summing a semi-additive measure across time is refused at build time, naming the measure
P0.17	Test	**Conformed dimensions**	*Service* must mean the same thing, carry the same label and offer the same values in **every** dataset that has it. ⚠ **This is how two reports come to disagree** — one calls it *Service* with eleven values, another calls it *Utility* with three, and a clerk comparing them is comparing nothing. A test, not care	P0.16	A dimension declared twice with different labels or value sets fails the suite by name
P0.18	Other · customers	**Effective-dated class, type and status**	**PO ruling, option (b).** One effective-dated table on customers for the three attributes revenue reporting slices by. ⚠ **It is in Phase 0 because every month it waits is a month of history nobody can ever produce** — the column overwrites today and the audit log that holds the old value is pruned. **It recovers nothing; it only starts recording.** Measured: `customers/models.py` has no effective dating at all, while billing, metering, rating and work all do	`06-backend/apps/customers/` · 7.2.4	Moving account 100244 to Small Commercial leaves July readable as Residential

Phase 1 · Catalogue

SHIPS	Screen 1·1 reading the table — 6 groups, 19 sub-groups, search, filters, the tenant overlay
WHAT A CLERK CAN NEWLY DO	**Find a report by name or group** — and **a water utility stops being shown gas reports**
TASK MIX	0 DB · 2 Backend · 2 Front end · 0 Settings · 2 Test · 1 E2E · 1 Other-module
DONE WHEN	1·1 shows 46 cards in 6 groups, a search finds one by name, and the broken card door is gone

ID	KIND	WHAT	WHAT IT INVOLVES	REFERENCE	OUTCOME
P1.1	Backend	Catalogue from the table	`GET /reports` joins definitions to categories and the tenant overlay. ⚠ Today it reads **no query parameter at all** and returns all 23 on every call	`api.py:288`	The response carries 46 cards, each with a group and sub-group code

P1.2	Backend	Catalogue filters	`q · group · service`, read server-side. ⚠ `origin` **is NOT a filter** — PO ruling: it is the **Standard · Custom** sub-tab, because a filter dropdown hides that you can author a report at all	B14 rule 12	A search for *ageing* returns one card; the sub-tab decides origin
P1.3	Front end	Screen 1·1	Group headings, sub-group rails, cards, search, origin and service filters. **An unavailable card stays a non-link with its reason on its face** — that decision is kept	1·1	Matches the drawing
P1.4	Front end	**Fix the broken card**	`ROUTE_TARGETS.S61` points at `/settings/audit-log`, which matches **no Settings tab**; `SettingsPage.tsx:475` falls back to `tabs[0]`, so the card silently opens **Settings → Company**. Walked and confirmed	`ReportLibraryPage.tsx:76`	The card opens a page titled **Audit log** at `/{tenant}/audit-log`
P1.5	Test	No literals	The catalogue renders from fixture *rows*; deleting the seed empties the screen rather than falling back to a tuple	P0.6	A catalogue with zero rows renders the empty state, not 23 cards
P1.6	Test	Every destination resolves	Assert each route target against the router's own table, **failing by name**: *"S61 → /settings/audit-log matches no route"*	P1.4	The next wrong address is caught at merge, not by a clerk
P1.7	E2E	Extend the report-centre journey	Add: filter by group, search, and click the audit card to its real destination	`e2e/US-REP-report-centre.spec.ts`	Green against the seeded tenant
P1.8	Other · tenancy	Enabled services on the response	Already served on the same call the catalogue makes — wire it to the service filter rather than adding a request	`queries.py:620-663`	Opening a report from the catalogue costs no extra request

Phase 2 · Report

SHIPS	Screen 2·1 on the shared `DataTable` — server paging, sort, search, column choice — and the three bespoke report pages retire into definitions
WHAT A CLERK CAN NEWLY DO	**Open a 38,000-row billing register and find one bill.** And the account ledger, by-premises summary and customer statement gain paging, sort, column choice and export for the first time
TASK MIX	0 DB · 4 Backend · 4 Front end · 0 Settings · 2 Test · 1 E2E · 1 Other-module
DONE WHEN	A report of 38,412 rows opens in under two seconds showing 50, and no export is refused for being large

ID	KIND	WHAT	WHAT IT INVOLVES	REFERENCE	OUTCOME
P2.1	Backend	Paging, in SQL	`page` / `per` pushed into each dataset as `LIMIT/OFFSET`. ⚠ **Not a Python slice** — today the owning read models materialise the whole book first	`payments/reporting.py:235`	The query plan shows a LIMIT; memory does not scale with the tenant
P2.	Backend	Sort, in	`sort` / `dir`, with a stable secondary sort by id. ⚠ **The account ledger**	`reportsShared.tsx` —	Sortable where

2		SQL	**declines sort and is right to** — sorting a running balance by amount destroys the column. Sort is per-dataset, not universal	the argument, which stands	declared; absent where the dataset says so
P2.3	Backend	Search	`q` against the dataset's declared searchable columns	`DataTable.tsx:573`	A search reaches the server; it does not filter the loaded page
P2.4	Backend	**Remove the row cap**	`EXPORT_ROW_CAP = 100_000` is a hard cap and `NFR-5` forbids hard caps. Killeen at 57,066 accounts × 14 cycles is ~685,000 rows a year. ⚠ **Also delete the pre-count**: `selection_count` falls through to `len()` on a Python list, so the whole report is built twice. **The writer already streams row-by-row from a generator** — this is a deletion, not a rewrite	`exports.py:120` · `:546` · `services.py:6202`	A 685,000-row export completes and is never refused for size
P2.5	Front end	2·1 on `DataTable`	Toolbar, pager, column chooser, empty state — **the component, not a second implementation**. Today the module has four different table implementations	`DataTable.tsx`	One table component renders all 46
P2.6	Front end	The class control	The server has read `class` since day one and **no control was ever drawn for it**	`queries.py:255`	The ageing filters by consumer class from the screen
P2.7	Front end	Real date fields	Five bare `<input type="date">` become `DatePickerField` with `min` / `max`, a Today shortcut and the zone named. ⚠ **An inverted range is refused at the control.** Today it costs two 422s — the first fired by a From with no To yet	B14 rule 7 · walked 2026-09-23	Typing 09/30 then 09/01 sends **zero** requests
P2.8	Front end	**Retire three screens**	Account ledger, by-premises and customer statement become definitions rendered by 2·1. ⚠ **Keep the addresses** — they are bookmarked and linked from Finance. Redirect to `/reports/{key}`	§06 · the three Removed rows	Three page components deleted; three definitions added; the old addresses still open
P2.9	Test	Paging on real data	⚠ **Postgres only.** Seed 40,000 rows and assert the page-two query reads 50, not 40,000	P0.10's settings note	Proved on PostgreSQL; the SQLite suite is not evidence
P2.10	Test	The control refuses	Assert zero network calls on an inverted range	P2.7	A network spy sees nothing
P2.11	E2E	New journey — `J-REP-01`	Open the register, page to 3, sort, search one bill, export it	`08-qa/01-e2e/journeys/`	The journey file exists and runs
P2.12	Other · payments, billing, ledger	**The gating dependency**	All three read models `list()` the whole book before Reports sees it. **Reports cannot page what it is handed whole.** Each must accept limit, offset and sort	`payments/reporting.py:235,291` · `billing/reporting.py:199` · `ledger/reporting.py:`	Three read models accept paging; Reports stops sorting in Python

				`ledger/reporting.py:2003`	
P2.13	Front end	A tile with no number says so	The AR-total tile renders **empty** when the receivable holds more than one currency, while the refusal and its sentence already exist at `queries.py:320`. ⚠ **No design work is needed** — `BR-REP-033/024` settled this pattern and it ships on **E·1**: the tile reads **Not counted yet** with the reason in its support text	`queries.py:320-322` · `regions.py`	Every KPI tile in the module renders its own refusal rather than nothing
P2.14	Backend	**The Subject parameter**	One parameter on two definitions — Account · Customer · Premises · Unit. Selects on `accounts.id`, `customer_id`, `premises_id`, `unit_id`. ⚠ **Master is not a fifth value**: *"a master account is an ordinary row"*, `kind='master'`, same lookup	`customers/models.py:472,495,533`	Statement and Ledger each answer all four; the premises subject marks the master row and never merges it
P2.15	Front end	**The unit grain**	The one subject nobody built. The head reads **premises address + unit label** and the unit declares no address (hard rule 8); an **Occupant** column appears on this subject only, because a unit outlives its tenants	R24d · R25d	A three-month unit statement spanning a move-out names both occupants
P2.16	Front end	**Retire the third page**	`CustomerStatementPage.tsx` joins the account ledger and by-premises in becoming a definition. ⚠ **Its careful parts are kept** — the `held_with` guard that stops a stranger's arrears appearing on this person's statement, and the server-reasoned `AbsentFigure`	§06, the three Removed rows	Three page components deleted, two definitions gained, the old addresses still open
P2.17	Backend · front end	**Slicers from the model**	⚠ **The fix the PO asked for.** Filters stop being hand-written per report and are **derived from the dataset's declared dimensions**. AR Ageing goes from 2 slicers to the 13 its fact actually has, and so does everything else on that dataset, for free. Also: **group by up to three levels** with subtotals, and drill down one level	P0.16 · 2·1	Adding a dimension to a dataset makes it available on every report built on it, with no report edited
P2.18	Front end	Drill through to the facts	An aggregate row opens the rows behind it, with the group's values pre-applied as filters. ⚠ **Without this a subtotal is a number you cannot check**, and a number a clerk cannot check is one they will not defend to a customer	2·1	A $48,221.14 subtotal opens the 412 rows that make it
P2.19	Front end	**State the basis on screen**	A report sliced by a customer attribute says it is **as-is** — today's class, not July's. ⚠ **Ships whichever way the as-was ruling goes**, because a report that silently answers a different question from the one asked is worse than one that refuses. It is a FACT, six words, not rationale — rule 33	7.2.4	Revenue by class names its basis; nobody mistakes it for as-was
P2.20	Backend	**BI tier 1 — service accounts**	A machine principal: named, scoped to a tenant and **a named list of reports — ruled**. Granted *Revenue Analysis* and *Trial Balance*, it can read those two and nothing else. **Administrators create them — ruled**; any staff member may hold one. Revocable without touching a person, rate-limited, and it appears in *Started by* like anyone else. ⚠ **Not a person's password in a BI config file** — that is how a clerk who leaves takes the reporting with them	7.8	Power BI reads a report over the same API a screen uses; RLS applies unchanged

		fits	— that is how a clerk who leaves takes the reporting with them		
P2.21	Front end	**Attributes and measures, split**	The edit panel's one flat column list becomes two sections, and **each measure carries its aggregation** — Sum · Count · Average. ⚠ Today a column is a column and the total row guesses	2·2 · P0.16	A measure's total is what its aggregation says, not what the renderer assumed
P2.22	Backend · front end	**Refuse an unlawful total**	⚠ **The money task.** A footer that cannot lawfully sum shows **an em-dash and the reason**, on screen *and in the CSV* — never a number. Three cases are wrong today and nothing catches them: a balance summed across time, a DSO averaged across routes, and two units added together	P0.16 · 2·1	AR Ageing grouped by month refuses its balance total and says why
P2.23	Front end	The grain line	Every report states what **one row** is, under its title — *"One row per account"*. A FACT in four words, not rationale	2·1 · P0.16	All 45 state their grain; none states it twice
P2.24	Front end	Drill down in place	A group row expands to the next level **without leaving the report** — Area opens its routes, a route its premises. Collapse restores it	2·1	Three levels open and close without a page load
P2.25	Backend · front end	**One calendar per report**	⚠ A billing cycle **straddles two months**. A report declares **Calendar** or **Billing**, every time control on it uses that one, and **mixing is refused**. The head says which. Same fault line as the dashboard period ruling	2·1 · 2·2	Filtering by accounting period and grouping by cycle is refused at save, naming both
P2.26	Front end	**The slicer bar**	⚠ **Thirteen chips on screen is a worse screen, not a better one.** Four visible slicers, named by the definition, and `+ Filter` opens the rest. Applied filters appear as chips. This is the control that answers the finding	2·1 · P0.16	AR Ageing reaches all 13 of its dimensions; four are one click, the rest are two
P2.27	Backend · front end	**As-was where history exists**	A report slicing by class, type or status answers **as-was** — what it was then — because that is what a regulator asks. ⚠ **Before** `P0.18` **'s table starts, it falls back to as-is and says so.** One report will state **both bases** for the first year, and that is correct rather than untidy: the alternative is a single confident line that is true of half the rows	P0.18 · P2.19	Revenue by class for July 2026 puts $184.52 under Residential, and says which rows it could not date

Phase 3 · Activity

SHIPS	Screen 5·1 History and 5·2 Run opened, over the `jobs` table that already exists
WHAT A CLERK CAN NEWLY DO	**Find the CSV they exported yesterday** — and the live toast that says *"It appears in Jobs when it is ready"* stops pointing at a screen that does not exist
TASK MIX	0 DB · 1 Backend · 3 Front end · 0 Settings · 1 Test · 1 E2E · 0 Other-module
DONE WHEN	An export made on 2·1 is findable on 5·1 within a second, and downloadable for three days

ID	KIND	WHAT	WHAT IT INVOLVES	REFERENCE	OUTCOME
P3.1	Backend	Runs endpoint	Paged read over `jobs` filtered to report kinds, with the report name joined in. **No new table** — the job record is the run record, and the module already says so	`exports.py:60` · `tenancy/models.py:1551`	5·1 lists runs with status, who, when and the file
P3.2	Front end	Screens 5·1, 5·2	History tab on `DataTable`; a run opens to its parameters, progress, error and download	5·1, 5·2	Matches the drawings
P3.3	Front end	**Point the toast at History**	The toast says a file *"appears in Jobs"* and there is no Jobs screen anywhere a clerk can reach. ⚠ **The destination is the History tab** — and the string changes with it, because *Jobs* is our word and not theirs	`en.json` — `reports.export.queued`	The toast links to History and the link resolves
P3.4	Front end	Re-run	A finished run re-runs with its original parameters, unchanged	5·2	Last quarter's report reproduces exactly
P3.5	Test	Retention	A file older than three days is swept and the row says **expired** rather than offering a dead link	`exports.py:522`	No download button over a purged file
P3.6	E2E	`J-REP-02`	Export → poll → find it on Activity → download	`08-qa/01-e2e/journeys/`	Green end to end

Phase 4 · Schedules

SHIPS	`report_schedules`, the scheduler, screen 2·3 and the Scheduled tab
WHAT A CLERK CAN NEWLY DO	**The month-end pack arrives without anyone remembering to run it**
TASK MIX	1 DB · 2 Backend · 2 Front end · 1 Settings · 2 Test · 1 E2E · 1 Other-module
DONE WHEN	A schedule set on Friday produces a file on Monday, delivered to named internal staff

ID	KIND	WHAT	WHAT IT INVOLVES	REFERENCE	OUTCOME
P4.1	DB	`report_schedules`	tenant_id, definition_id, cadence, params, format, recipients[], next_run_at, enabled, last_job_id. RLS FORCE. Index on `next_run_at` where enabled. ⚠ **Cadence as a CHECK constraint** — four signed values, so a fifth is a migration, which is correct here	Source register row 13	A schedule row survives a restart and fires once
P4.2	Backend	The scheduler	celery beat picks up due schedules and enqueues a job. ⚠ **beat is not deployed today** — celery itself is, with `JOBS_EAGER=false` in production. Reuse the idempotency the jobs table already has: one active job per tenant + kind + scope	`config/celery.py` · `jobs` unique constraint	A schedule that fires twice produces one file
P4.3	Backend	Recipients	⚠ **Internal staff only.** `BR-REP-020` `RULED`: external recipients arrive at W6 with an explicit confirm step, because external send means data leaves the tenant. **A free-text email field breaks the ruling**	Source register row 15	The picker offers this tenant's users and refuses anything else

P4.4	Front end	Screen 2·3	**One** `FormModal` **, three fields** — not a wizard. See the correction at the head of this section	`FormModal.tsx`	Scheduling takes one dialog from the report you are looking at
P4.5	Front end	Screens 4·1, 4·2	Scheduled tab and a schedule opened — pause, edit, run now, delete	4·1, 4·2	Matches the drawings
P4.6	Settings	Retention and default format	⚠ `BR-REP-018` makes the 3-day retention **tenant-configurable** and **that half has no storage**. A tenant row, or the constant stays and the rule stays half-kept	`exports.py:123`	A tenant sets 7 days and files live 7 days
P4.7	Test	Cadence arithmetic	Each of the four cadences computes the right `next_run_at` , including **on cycle close**, which is not a cron expression	P4.1	Four cases, four correct dates
P4.8	Test	Failure is visible	A schedule whose report now errors marks the run failed and **keeps the schedule**, rather than silently stopping	—	4·1 shows a failing schedule as failing
P4.9	E2E	`J-REP-03`	Schedule, force the clock, assert the file and the notification	`08-qa/01-e2e/journeys/`	Green
P4.10	Other · notifications	Deliver the file	The outbound queue carries the link, not the file	`BR-NOT-005`	Recipients get a link that respects their permissions
P4.11	Front end	**Screen 4·1 — the Dashboard tab**	Four KPIs, five panels, every panel a definition rendered small. ⚠ **Ruled as a NEW tab, not a replacement** — PO: *“dont replace existing one”*. Nothing in `apps/reports/dashboard.py` is touched	4·1	The shell dashboard is byte-identical after this task
P4.12	Backend	Period and comparison	A real accounting period, and a second one to compare against. ⚠ **Not a rolling window** — thirty days straddles two cycles and answers nobody's question	`dashboard_period.py` — read, **not edited**	September against September last year, both labelled in dates
P4.13	Front end	Panel → report carries the period	Opening a panel opens its report **already on this period**. ⚠ A dashboard that drops you into a report on a different window is worse than one with no links	4·1 → 2·1	Every panel link round-trips the period in the URL
P4.14	Backend	**BI tier 2 — scheduled delivery**	The scheduler already writes a file; send it to **SFTP or a signed URL** on a timer. **Ruled, with three conditions:** destinations are set up **by an administrator** and never by whoever creates the schedule; **encrypted in transit**; **every delivery logged**. ⚠ Without the first, a clerk schedules the debtor list — 412 names, addresses and balances — to a personal address and nothing records it	7.8 · P4.2	A monthly close pack lands in the utility's SFTP drop without anyone opening

					the product
P4.15	Other · portal	**A landlord sees their own account only**	**PO ruling, option (a).** In the customer portal a premises statement is scoped to the accounts the signed-in person holds. At **1408 Elm Street** the owner sees the **$69.60** of common-area charges and **not** Unit 3's $100.00. ⚠ **Not an exception:** where the building bills the owner rather than the tenants, the owner *is* the account holder and sees it as their own. **Staff are unaffected** — they see the whole premises, as they already do	R24c · 7.7	A portal premises statement never shows a balance belonging to somebody else

Phase 5 · Build your own

SHIPS	Screen 2·2 the edit panel, 2·4 Save as, and E·2 the Reports settings section
WHAT A CLERK CAN NEWLY DO	**Answer their own question without calling us** — pick a dataset, choose columns, filter, save it, share it
TASK MIX	0 DB · 2 Backend · 3 Front end · 1 Settings · 2 Test · 1 E2E · 1 Other-module
DONE WHEN	A utility saves a report we never wrote, shares it, and it appears in the catalogue for their colleagues and nobody else's

ID	KIND	WHAT	WHAT IT INVOLVES	REFERENCE	OUTCOME
P5.1	Backend	Definition CRUD	Create, update, delete a tenant definition. ⚠ **A tenant may never edit a platform row** — copy on write	P0.1	Editing a built-in creates a tenant copy; the platform row is untouched
P5.2	Backend	**Four reserved names** — `custom` , `dashboard` , `scheduled` , `history` — refused BY NAME when a definition is saved. ⚠ **A report key sits in the same URL position as a tab**, so a utility naming one *History* shadows the tab and the tab stops opening. Invisible until a real customer does it, and then the product looks broken. Validate a definition	Columns and filters must exist on the chosen dataset. ⚠ **A definition is user input** — it names columns, never SQL	P0.4	A definition naming an unknown column is refused with the column named
P5.3	Front end	Screen 2·2	The edit panel beside the live table. **Every change re-runs the preview**	2·2 · the RULED box in §01	Adding a column shows it in the rows below without a save
P5.4	Front end	Screen 2·4	Save as — name, category, visibility	2·4	The saved report appears under *Mine* in 1·1
P5.5	Settings	Screen E·2	Categories, the tenant overlay and role gating, as a section inside **Settings** → **Configure** → **Reference data**. ⚠ **No**	`/{tenant}/settings/reference-data/reference-data`	A tenant adds a category and hides a report

			new route — it is a section of the existing sub-tab, so no four-segment route and no ranking hazard		without a release
P5.6	Test	Isolation of authored rows	⚠ **Postgres only.** A tenant-authored definition cannot read another tenant's rows however it is filtered	P0.10	Proved on PostgreSQL
P5.7	Test	Private by default	A new definition is invisible to a colleague until shared	`BR-REP-019`	Two sessions, one row, one sees it
P5.8	E2E	`J-REP-04`	Build, preview, save, share, a colleague opens it	`08-qa/01-e2e/journeys/`	Green
P5.9	Other · identity	Role gating on a definition	`min_role` resolved server-side. ⚠ `BR-REP-023` restricts the access-review report to System Admin and Executive and **nothing enforces it** — every card is served to any tenant role today	`catalog.py:9-10` · `api.py:287`	A Billing Clerk cannot run the access-review report
P5.10	Front end	**The Custom sub-tab**	Standard · Custom under Catalogue, with **Owner**, **Based on** and **Private**. ⚠ **Origin was a filter in the first draft** — PO ruled it a sub-tab, because a filter hides that you can author reports at all	1·2	Custom reads 0 on day one and says what to do about it
P5.11	Front end	Columns: reorder and rename	⚠ **The PO looked at the panel and said the user cannot add columns.** Two thirds of that is fixable here and is definition data: **drag to reorder** and **rename a heading**. An unticked column is drawn as an empty box and **never greyed** — grey means *you cannot*	2·2	*Account* can become *Account No* and move left, without touching code
P5.12	Front end	A door to the retention setting	The retention line on **5·1 History** — *"Files are kept for 3 days"* — links to Settings → Configure → Reference data → Reports, **for an admin only**. ⚠ **Found by the door audit**: a setting reachable only from a grid of forty cards is a setting nobody changes. The sentence stays six words; a link is not rationale	5·1 → E·2	An admin reaches the retention control in one click from the screen it governs; a clerk sees the fact and no link

What will bite, and where

RISK	WHY IT BITES HERE	WHAT TO DO ABOUT IT
A green suite proves nothing	No gate in this repository reads rendered output. The Reports module's **7 suites and 83 cases all pass** — over a card that opens the wrong screen, three false sentences and a toast pointing at a screen that does not exist	Every phase carries a test that would have failed on the defect it is written for, and P1.6 asserts destinations rather than rendering

The test database hides a whole class of failure	`config/settings/test.py` hardcodes in-memory SQLite and ignores `DATABASE_URL`. SQLite has **no row-level security** and **no aborted-transaction state** — so RLS probes and error-recovery code pass there and fail in production	P0.10, P2.9 and P5.6 are marked **Postgres only** and must be run that way or they are decoration
A hold outlives its reason	Three live strings describe a product that no longer exists: the download note says nothing writes a file (four reports export today), the premises note describes a state of the books that ended 2026-08-16, and a dead gap sentence survives in all eleven locales	Every phase that lands a capability **deletes the sentence that said it was missing**, in the same commit — and P2.8 and P3.3 name theirs
Two answers to one question	Money is formatted **server-side** on the dashboard and **client-side** everywhere else. They agree today because the server formatter was built to reproduce `MoneyCell` byte for byte. **Nothing checks it**	A test that renders the same figure both ways and compares. Until then it is a convention, and a convention does not stop a caller
Immutability is not a convention	Reports writes nothing and never should. There are **zero** write calls in `apps/reports/` today — **because nobody has added one**, not because anything refuses	A gate that fails on `.create(`, `.update(`, `.save(` or `.delete(` anywhere in the module. Cheap, and it makes the boundary real
A migration with no way back	P0.6 seeds 46 rows and P0.7 deletes the literals they replace. **After P0.7 there is nothing to fall back to**	Land P0.6 and P0.7 in **separate releases**, with the literals dead but present for one version. Every migration in this plan gets a tested reverse

03 The screens

Four tabs, two sub-tabs, thirteen screens, seventeen drawings — and then, at the end, **two screens this module serves that are not reached through Reports at all**. Every flow is drawn in one order and only this order: ① **the list** · ② **one row opened** · ③ **what you can do to it** · ④ **what each of those changes, and where.** Where a step is empty for a tab it says so rather than being skipped, because an empty step is a decision. **A screen appears once**, at its end state, with every difference from today marked on the drawing.

Step ④ is nearly empty for this whole module — and that is the finding, not an omission.

A report writes **no business data**. There are zero write calls in `apps/reports/` today. What the acts below do write is narrow and worth knowing exactly: **a job row** (an export), **a definition row** (a saved report), **a schedule row**, and **an `audit_log` row** for each. **Nothing touches the ledger. Ever.** Where an act below says it writes nothing, that is checked, not assumed.

The Reports screen — four tabs

Three tabs, flat, and one of them has two sub-tabs. Ruled.

An earlier draft of this document put **Scheduled** and **History** inside one tab called **Activity**, and put the utility's own reports behind a *filter*. Both were wrong and both were corrected by the PO. **Activity is deleted — the word and the address.** `/reports/activity` never ships and nothing in the product is ever called that. And a filter hides the fact that you can author a report at all; **a sub-tab advertises it**.

⚠ **The fourth tab was added by PO ruling after the first three were drawn, and it comes with an explicit fence:** *"don't replace existing one"*

⚠ **The fourth tab was added by PO ruling after the first three were drawn, and it comes with an explicit fence: *"don't replace existing one"*.** The shell's dashboard at `/{tenant}/dashboard` — rail item 1, the screen after every sign-in — **is not touched by any task in this plan**. It answers *what needs me today*. This one answers *how are we doing*, and they are different questions for different people at different times of the month.

TAB	ADDRESS	SUB-TABS	HOLDS	PHASE
Catalogue	`/reports`	**Standard** · **Custom**	Every report you can run. Standard is the 46 that ship; Custom is what this utility wrote	1 · 5
Dashboard	`/reports/dashboard`	none	**The picture** — charts, trends, a period against another period. ⚠ **Every panel is one of the catalogue's own reports**, so the dashboard and the reports can never disagree	4
Scheduled	`/reports/scheduled`	none	What goes out on its own, and to whom. **One list** — a schedule of a standard report and of a custom one are the same object	4
History	`/reports/history`	none	What has been run, by whom, and the file it left. **One list** — the *Started by* column says whose	3

Four segments are now reserved, and a utility can name a report.

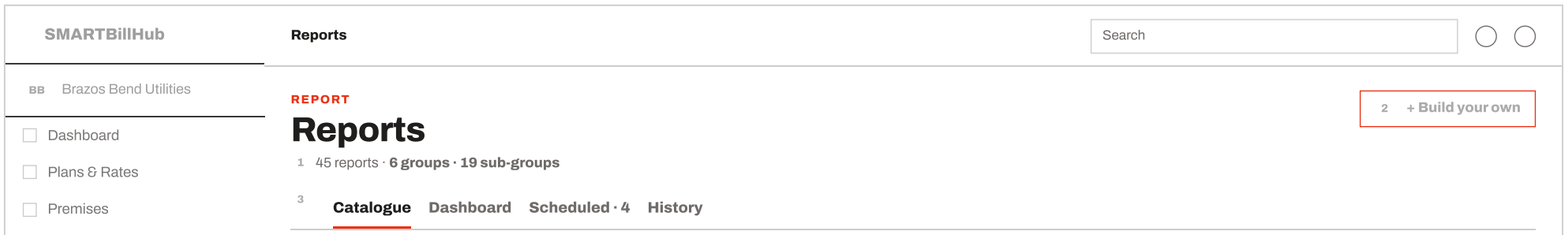
`custom` , `dashboard` , `scheduled` and `history` sit in the same position as a report key — `/reports/ar-ageing` . A utility that names a custom report *History* would shadow the tab and the tab would stop opening. **Saving a definition refuses those four words by name** (**P5.2**), and the router ranks all four above `{key}` (**P0.11**). ⚠ Invisible until a real customer does it, and then it looks like the product is broken.

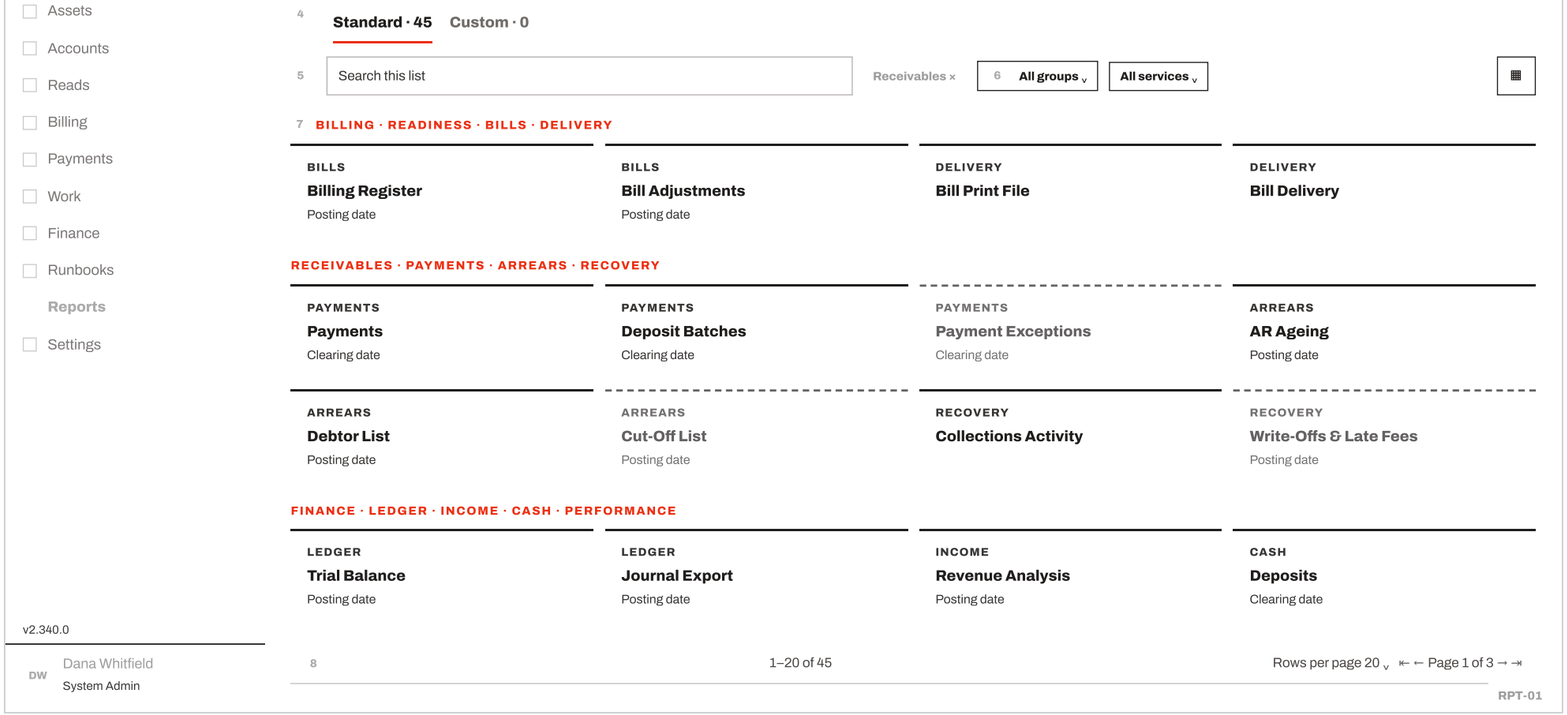
Tab 1 · Catalogue

① THE LIST	**1·1 Catalogue** — 46 definitions in 6 groups and 19 sub-groups
② ONE ROW OPENED	A card opens **2·1**, in Tab 2. There is no detail screen for a card itself — the card *is* the pointer
③ WHAT YOU CAN DO	Search, filter, switch sub-tab, and open. **One page action —** `+ Build your own` , and it is **not drawn until Phase 5**, because a button that points at nothing for four phases is the defect that left five portal cards dead. The one act that looks like an exception — an unavailable card — is **not a control**: it is a card that is not a link, with its reason on its face
④ WHAT IT CHANGES	**Nothing. No write of any kind.**

1·1 · CATALOGUE TASK P1.3

END STATE · DESKTOP 1440 · TAB 1, STANDARD SUB-TAB — WHERE THE RAIL LANDS YOU





1 **Page head** · B14 rule 40, 32px · **UPDATE** · ⚠ **45, not 46.** The forty-sixth answer is the **Dashboard tab**, which is in the strip rather than in the grid — a card pointing at a sibling tab is noise. The sub-line states the shape of the catalogue, never an availability count, because after Phase 1 **every card here runs**

2 **+ Build your own** · the one page action, filled accent · **NEW, HELD** · ⚠ **not drawn until Phase 5**, when the builder lands. Until then the screen says the thing is not built rather than showing a button that does nothing — and **the test that fires when the hold expires ships in the same commit** as the hold. It sits **above** the sub-tab strip, not inside Custom, because the usual way a custom report gets made is opening a standard one and changing it

3 **The four tabs** · Catalogue · Dashboard · Scheduled · History · **NEW** · flat, ruled. The tab rides in the URL, so a link to History is a link to History. ⚠ **The strip is not drawn until the second tab exists** — a one-tab strip is furniture

4 **Standard · Custom** · the sub-tab strip · **NEW** · second-level strips are the house pattern — Settings → Configure has five, Users & roles two, Company two. **Custom reads 0 on day one** and that is the point: an empty sub-tab is how a clerk learns they can author a report

5 **Search** · `DataTable` 's own, server-side · **NEW** · **SOURCE: SERVED**

6 **Two filters, not three** · group and service · **NEW** · ⚠ **origin was a third filter in an earlier draft and is now the sub-tab.** A filter dropdown hides that you can author reports at all · **SOURCE: SERVED** for both

7 **Group heading with its sub-groups** · 6 groups, 19 sub-groups · **NEW** · **SOURCE: SERVED**, `report_categories` — ⚠ never a literal in the front end, which is what **P1.5** asserts

8 **Pager** · **NEW** · 46 cards do not need paging at T7 and `NFR-3` admits no exceptions, so it pages anyway — the rule exists because four endpoints claimed an exception and are 13× short at T5

FOR	Any staff member with the reports permission, choosing what to run
DOORS	**Four, all read from the source.** The sidebar's *Reports* item (`AppShell.tsx:305` , item 12 of 13) · the dashboard's Service-mix panel, *See revenue by service* (`DashboardPage.tsx:1284`) · the viewer's ← *Reports* back link (`ReportViewerPage.tsx:413`) · the breadcrumb ancestor, **but only from the two ledger reports**, because only those two are declared as nav children (`AppShell.tsx:316-317`)
PRIMARY ACTION	**+ Build your own**, filled accent, from **Phase 5**. ⚠ **Not before.** r10 draws two header buttons here — *+ Build your own* and *Scheduled & history*; the second is now a tab, and the first waits for the builder. Today the screen states the absence instead, which is right and stays right until the thing exists

FIELDS	Per card, in JSX order: badge · name · basis label · reason sentence (unavailable only). **The server sends no title** — the name is an i18n key today, and becomes `report_definitions.name` at **P0.1**
TABLE CONTROLS	**Search** NEW, server-side — `?q=` , P1.2 · **Filter** NEW, **two** chips — group and service; **origin moved to the sub-tab**, P1.2 · **Columns** n/a, a card grid · **Sort / pagination** NEW — today `GET /reports` reads **no query parameter at all** (`api.py:288`) and returns all 23 every call
WRITE SURFACES	**Detail** — a card opens its owning screen. **Add** — `+ Build your own` , from Phase 5; it writes nothing until you save, which is 2·4. **Edit** — **NOTHING**. **Update** — **NOTHING**
TODAY TODAY	Renders 23 cards from a frozen Python tuple (`catalog.py:133-410`), **10 as links and 13 as non-links**, identical for every tenant. No search, no filter, no pager. Walked 2026-09-23
TASK TASK	**P1.1** table-backed and paged · **P1.2** search and filters · **P1.3** the screen · **P1.4** the broken card door
WALKED	**YES** — 2026-09-23, Chromium at 1440, signed in as the seeded owner. 23 cards, 10 links, 13 non-links, matching the server's own `available: 10, total: 23`
RULES	Obeys B14 2 (one page header) · 32 (one sentence per section) · `BR-SHL-025` (a control it cannot honour is stated, not hidden). ✗ **Breaks** B14 12 (no toolbar) · 16 (no pager) — both closed by P1.2 and P1.1

The finding that matters most on this screen, and it was found by clicking.

GAP TASK P1.4

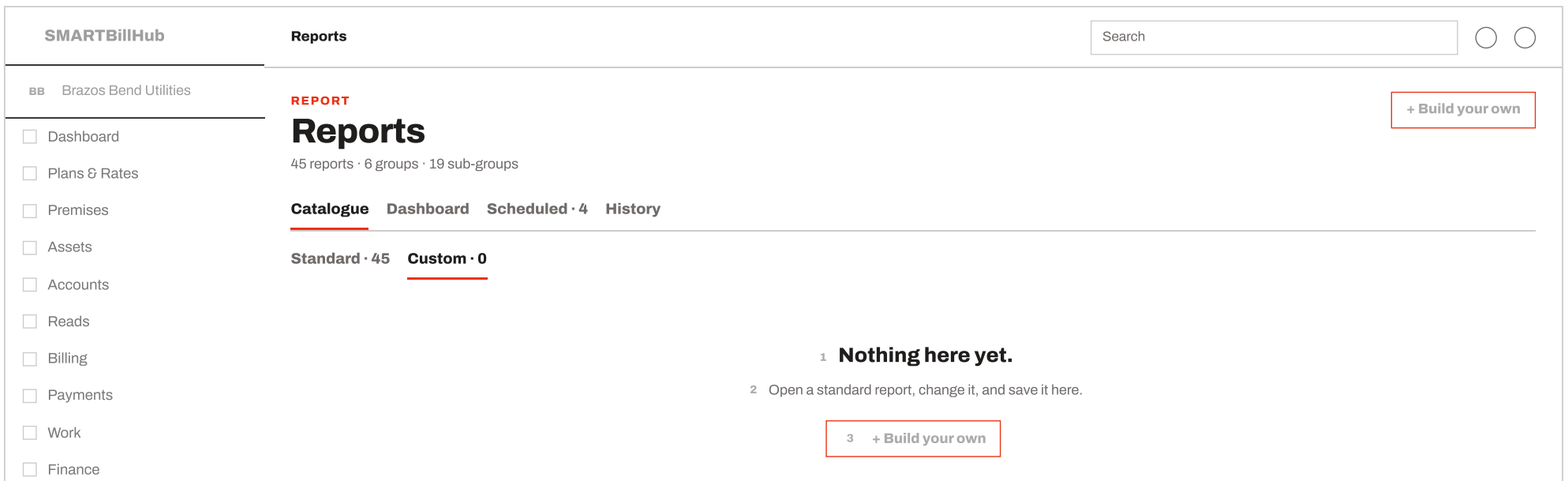
The **User activity audit** card is `available: true` and renders as a live link. `ReportLibraryPage.tsx:76` maps it to `/settings/audit-log` . **There is no `audit-log` tab in Settings** — the ids are company, users, services, reference-data, accounting, notifications, integrations, security, billing-plan — and `SettingsPage.tsx:475` resolves an unknown tab as `tabs.find(…) ?? tabs[0]` .

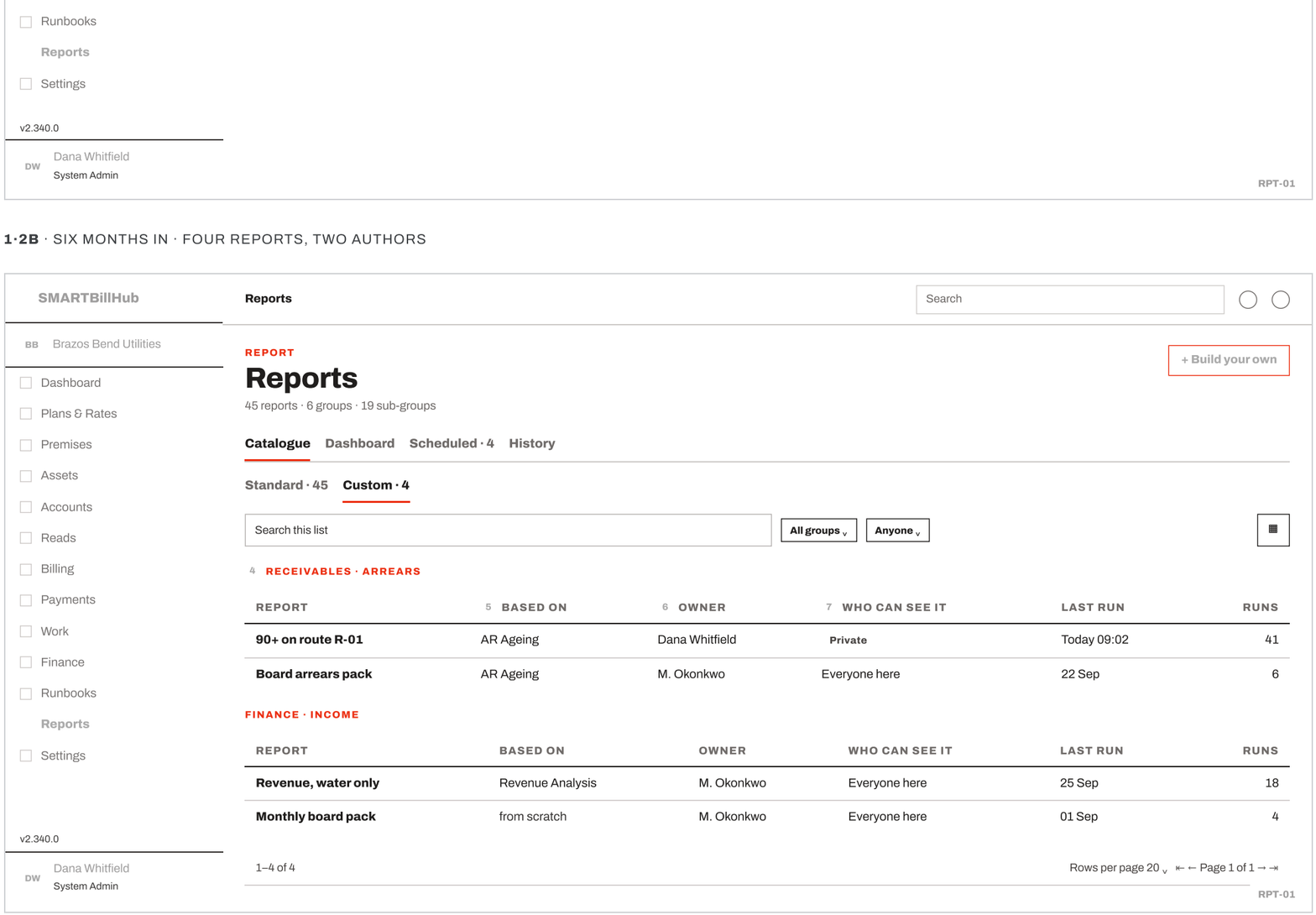
Walked: clicking it lands on `/brazosbend/settings/audit-log` , page title **Settings**, **Company** tab selected. No error, no 404, no hint. The audit log is at `/{tenant}/audit-log` . The other four route cards were clicked and all four land correctly.

1·2 · CATALOGUE → CUSTOM TASK P5.10

DRAWN TWICE · DESKTOP 1440 · **DAY ONE**, THEN **SIX MONTHS IN** — BECAUSE THE EMPTY STATE IS THE SCREEN MOST UTILITIES WILL SEE FOR LONGEST

1·2A · DAY ONE · NOBODY HAS WRITTEN ANYTHING





1·2B · SIX MONTHS IN · FOUR REPORTS, TWO AUTHORS

1 **The empty sentence** · three words · **NEW** · B14 rule 33 caps a heading at three and forbids a sentence; this is the heading

2 **What to do about it** · **NEW** · an INSTRUCTION, so rule 33 allows six words per sentence — this is nine and **is over**. It reads *"Open a standard report. Change it. Save it here."* at build time: three sentences of three beat one of nine, which is the rule's own preference stated in the rule

3 **The action, twice** · **NEW** · in the head and in the empty state. ⚠ **Not a duplicate control — the empty state is the only place a first-time clerk looks**, and a page action in the top right is furniture they have learned to ignore. It disappears from the empty state as soon as there is one row

4 **Grouped like Standard** · the same 6 groups and 19 sub-groups · **NEW** · ⚠ only groups **that have something in them** are drawn, so this list is two headings and not six. A custom report belongs to a group like any other

5 **Based on** · **NEW** · which standard report this was copied from, or *from scratch*. ⚠ **This column is the difference between a catalogue and a junk drawer.** Six months in, four reports have similar names and nobody remembers which came from where

6 **Owner** · **NEW** · why the sub-tab is called **Custom** and not *My reports*: two of these four are somebody else's. **SOURCE: SERVED**, `report_definitions.created_by`

7 **Who can see it** · **NEW** · **Private** or **Everyone here**. ⚠ **Private is the default** (**P5.7**) — a half-finished report that appears on a colleague's screen the moment it is saved is how people stop saving

ADDRESS	`/{tenant}/reports/custom` — ⚠ `custom` is one of the four **reserved** segments a definition may not be named

CLASS	`NEW` · the only screen this feedback round adds
DOORS	**Two, and the second one matters more than it looks.** The **Standard · Custom** sub-tab strip on 1·1 — which is the whole argument for a sub-tab over a filter. And the **confirmation after Save as** (2·4): saving a report drops you here with it in view, because a save that gives no sign of where the thing went is how a clerk saves the same report three times. ⚠ **No door from the left rail.** The rail lands on Catalogue → Standard, always; Custom is one click in, deliberately, because on day one it is empty at every utility
REPO	Nothing. Neither the sub-tab nor the table this draws from exists
DATASETS	None. It lists **definitions**, which are rows about reports rather than reports
TABLE CONTROLS	**Search** · **Filter** group and **owner** — ⚠ **owner, not origin**: origin is what the sub-tab already decided · **Columns** the picker · **Pager** yes, `NFR-3`
WRITE SURFACES	**Detail** a row opens 2·1 with that definition · **Add** `+ Build your own` · **Edit** the row's own report, through 2·2, **if it is yours** · **Update** rename, reshare, delete — all through 2·2 and 2·4, never inline here
TODAY `TODAY`	**Does not exist, and nor does the idea.** Every tenant sees the same 23 cards from a frozen Python tuple; there is no table a saved report could live in
TASK `TASK`	**P5.10** the sub-tab · **P5.1** the CRUD behind it · **P5.7** private by default
WALKED	**NO.** Nothing to walk
RULES	Obeys B14 5 · 12 · 16 · 31 (no database id on screen — a custom report is named, never keyed) · 32 · 33 · 40 · 43

④ What saving a custom report changes, and where

Writes:

one `report_definitions` row, `tenant_id` set · one `audit_log` row.

The ledger:

nothing.

No business data is written by anything on this tab.

Elsewhere:

the Custom count in the sub-tab strip goes up · the report becomes schedulable, so it can appear on **Scheduled** · running it writes a job, so it appears on **History** . ⚠ **The Standard sub-tab does not change.**

Editing *AR Ageing* and saving copies it; the standard row is untouched and every other utility's is untouched, because a platform row has `tenant_id` null and a tenant never writes to it. That is copy-on-write, and it is the reason a utility can change anything here without us shipping them a fork.

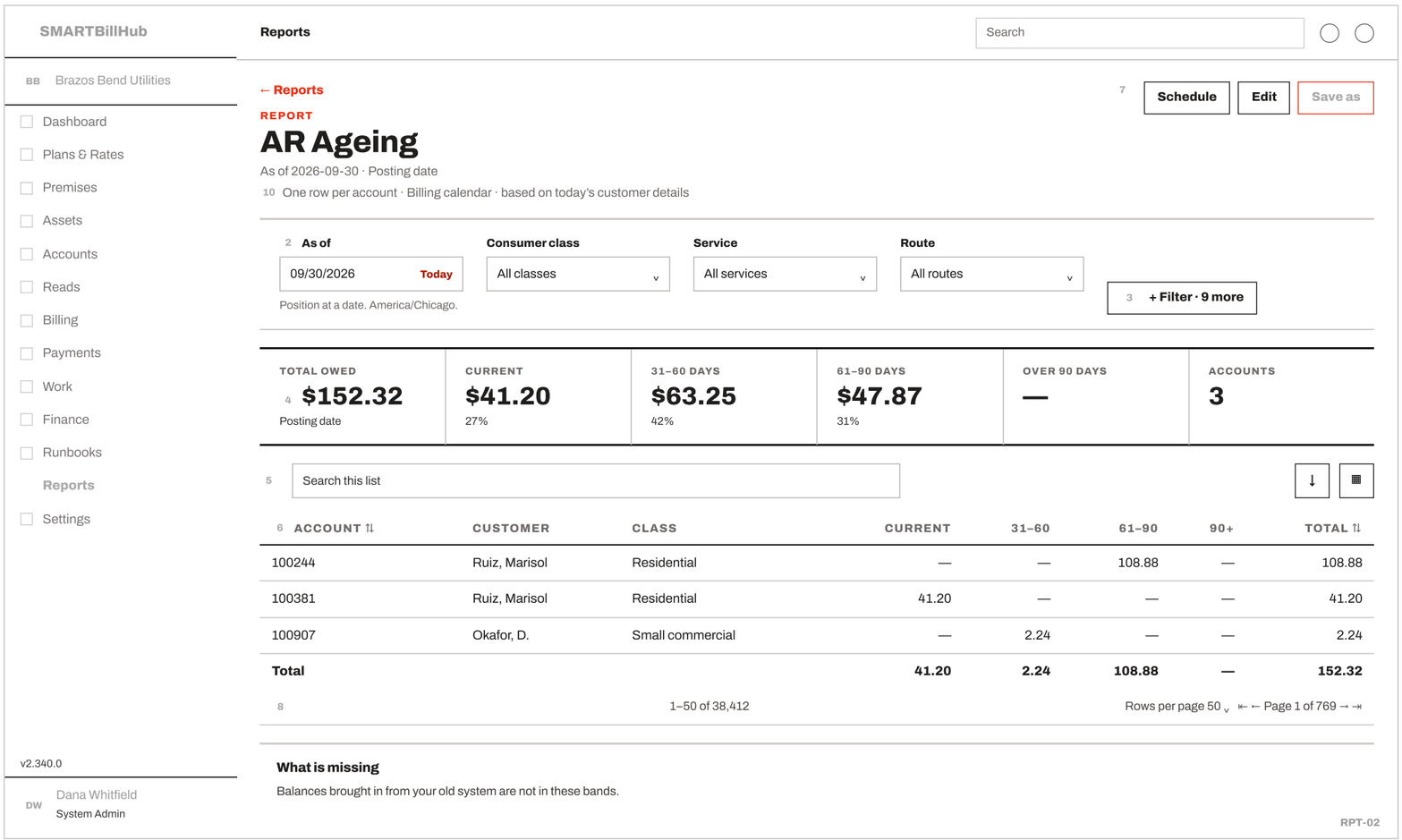
Tab 2 · Report

① THE LIST	**2·1 Report** — one screen rendering any of the 46. Four shapes: table only · KPI band + table · KPI + table + detail · a measured refusal
② ONE ROW OPENED	A row opens **its record, in another module** — a bill, an account, a payment. **There is no detail screen inside Reports**, and that is right: a report is a way in, not a place to live
③ WHAT YOU CAN	**Export** · **Schedule** (2·3) · **Edit** (2·2) · **Save as** (2·4)

DO	
④ **WHAT EACH CHANGES**	See the consequence block under each act. In summary: export writes a **job row** and a file; schedule writes a **schedule row**; save writes a **definition row**. All four write an `audit_log` row. **None touches the ledger, and none changes a bill, a payment or an account**

2·1A · REPORT — KPI BAND AND TABLE TASK P2.5

END STATE · DESKTOP 1440 · THE SHAPE MOST OF THE 46 TAKE. FIGURES ARE THE FIXTURE'S, NOT SERVED



1 **Back link** · returns to the catalogue · r10 rule 1 · **CHANGED on three of the retired screens** — the account ledger, by-premises and customer statement all read ← **Finance** today, under a breadcrumb reading *Reports ›*. Walked and confirmed on all three

2 **Date field** · `DatePickerField` with min, max, a Today shortcut and **the zone named** · B14 rule 7 · **CHANGED** — five bare `<input type="date">` today, no bounds, no zone · URL `f.to` · **P2.7**

3 **The slicer bar** · **four visible,** `+ Filter` **for the other nine** · **NEW** · ⚠ **This is the control that answers the finding.** AR Ageing offered **two** slicers where its fact has **thirteen** — and the fix is not thirteen chips, which is a worse screen. The **dataset declares them once** and the definition names which four are visible; the rest are one click. Applied filters return as chips on the toolbar. **P2.17** · **P2.26** · **SOURCE: SERVED** from the dataset's declared dimensions

3b **Consumer class** · one of the four · **NEW on screen, no new read** — the server has read this since day one at `queries.py:255` and no control was ever drawn · register row 7 · **P2.6**

4 **Money KPI** · the owning module's own figure · `BR-REP-024` — never 0, never the string `null` · **CHANGED**: where it cannot be computed it prints **the reason that is actually true**, and **no internal code appears beside it**

5 **Toolbar** · `DataTable` 's own row — **search, spacer, export, ▦ Columns**, and nothing else · `DataTable.tsx:519-630` · **NEW** · ⚠ the chip rows were **deliberately removed** after a PO review (`:324-328`); a screen wanting another dimension declares it in `TableConfig.filters`

6 **Sortable header** · sends `sort` / `dir` · UI-RULES 13 · **NEW READ**, and it must be SQL — today the owning read model sorts in Python · **P2.2** · ⚠ **declared per dataset**: the account ledger declines sort, correctly

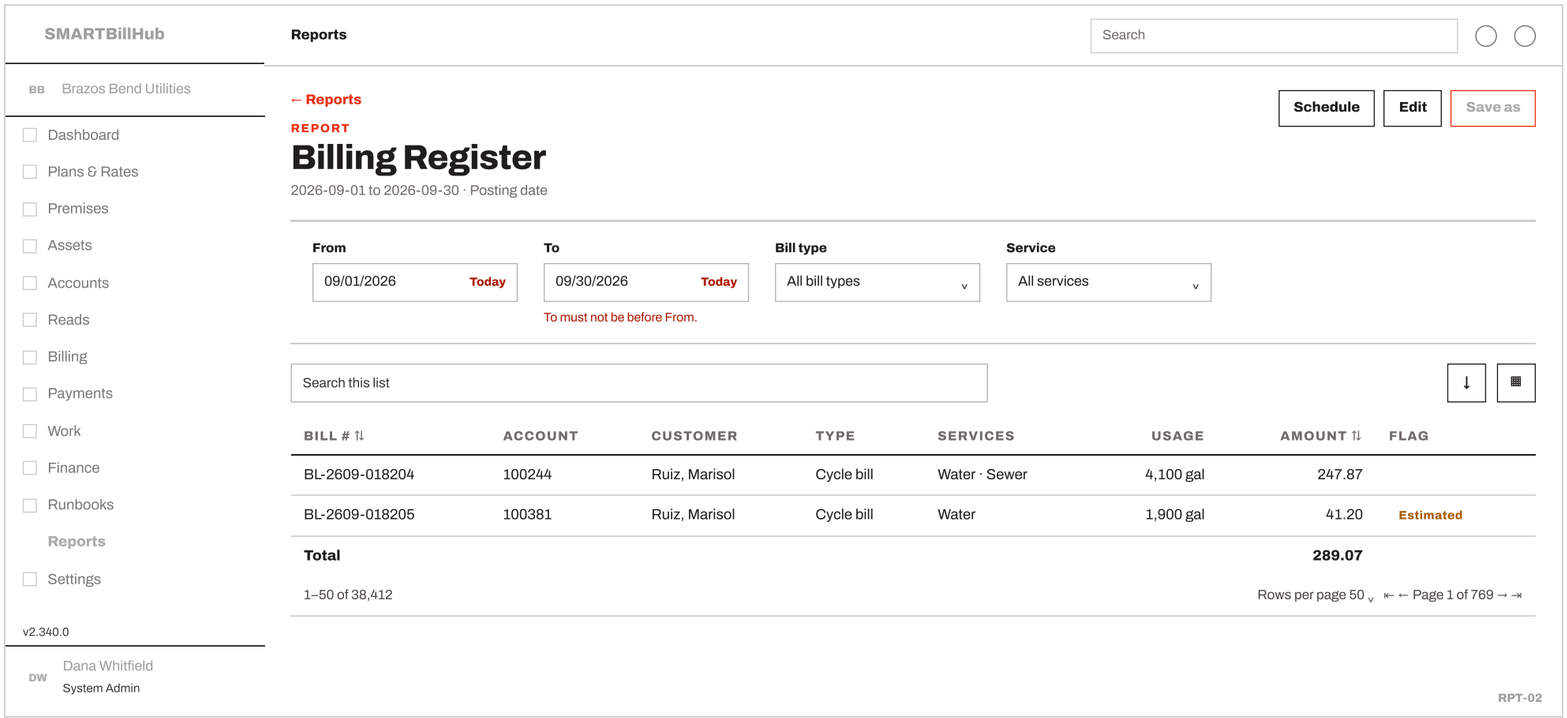
7 **The three acts** · Schedule · Edit · **Save as** (primary) · B14 rule 5's two tiers · **NEW** · ⚠ **Save as is the filled button because it is the only one that commits** — Schedule and Edit open a surface, they do not write

8 **Pager** · `page` / `per` · UI-RULES 16 · `NFR-3` · **NEW READ** · **P2.1** — the change that makes a 57,000-account utility possible

10 **Grain · calendar · basis**, in one line under the title · **NEW** · three facts a report cannot be trusted without, and none of them is rationale. **One row per account** — what a row *is* (**P2.23**). **Billing calendar** — which hierarchy it rolls up, because a cycle straddles two months and nothing else on screen would reveal a mix (**P2.25**). **Based on today's customer details** — as-is, not as-was, stated because customers keep no attribute history (**P2.19**). ⚠ **Eleven words in total**, and each is a FACT under rule 33, not an explanation

2·1B · TABLE ONLY

SAME SCREEN, A DEFINITION WITH NO KPI BAND — THE BILLING REGISTER



Two differences from 2·1a, both driven by the definition row and not by code: no KPI band, and four parameters instead of two. The **Bill #** cell is a link into billing — step ② — which it is not today. The inverted-range message under **To** is **refused at the control**: today those two keystrokes send two requests and collect two 422s, the first fired by a From with no To yet. Walked 2026-09-23.

2·1C · KPI, TABLE AND A SECOND TABLE

PAYMENTS BY METHOD — THE ONLY DEFINITION THAT RENDERS A DETAIL SET UNDER ITS SUMMARY

SMARTBillHub

Reports

Search

BB Brazos Bend Utilities

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

← Reports

REPORT

Payments by Method

2026-09-01 to 2026-09-30 · **Clearing date**

Schedule | Edit | Save as

From 09/01/2026 **To** 09/30/2026 **Purpose** All purposes

TOTAL RECEIVED	PAYMENTS	AGAINST BILLS
$14,208.55	**312**	**$12,940.10**
Clearing date		91%

METHOD	COUNT	AMOUNT	SHARE
Card	188	8,104.22	57%
Cheque	74	4,835.88	34%
Cash	50	1,268.45	9%
Total	**312**	**14,208.55**	

Not inside the collection figure

A deposit is shown as a deposit, on its own line, outside every collection total.

PURPOSE	COUNT	AMOUNT
Deposit	9	1,150.00
Prepayment	4	118.45

v2.340.0

DW Dana Whitfield
System Admin

RPT-02

This is the one screen where the basis rule visibly earns its keep. It is the only one of the four canned reports headed **Clearing date**; the other three read **Posting date**. That disagreement is the design — a July register and a July cash file are both correct and hold different money — and `BR-REP-001` makes the label mandatory in the page head, on every money tile and in the download filename. The purposes panel is `BR-REP-014` made visible: a deposit is never inside a collection figure.

2·1D · A MEASURED REFUSAL

BY-PREMISES — WHAT THE SCREEN SHOWS WHEN THE READ MODEL ANSWERS `AVAILABLE: FALSE`

SMARTBillHub

Reports

Search

BB Brazos Bend Utilities

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads

← Reports

REPORT

By-Premises Summary

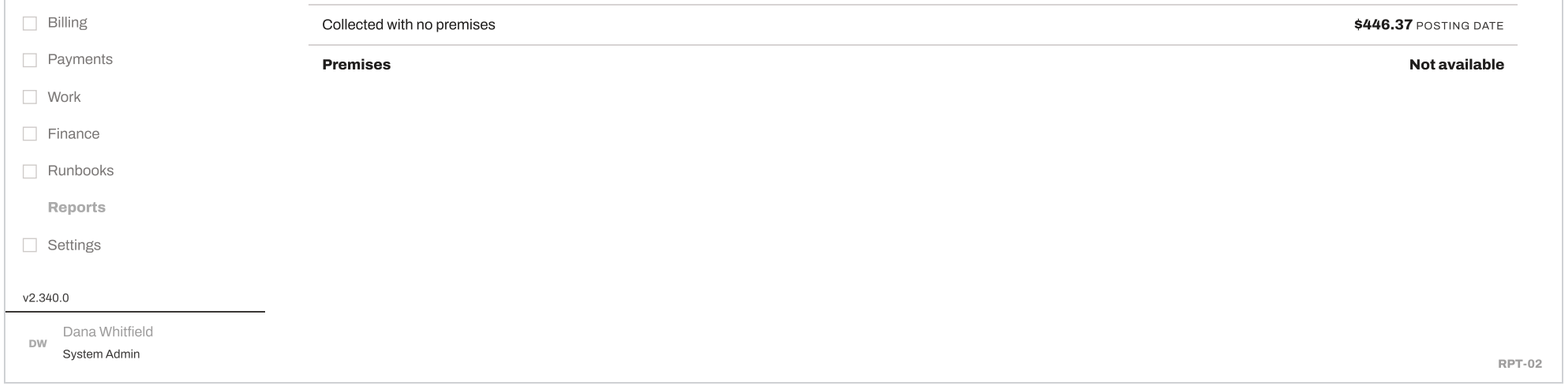
2026-09-01 to 2026-09-30 · Posting date

Schedule | Edit | Save as

Report not available

No line here carries a premises. Older entries never will.

Billed with no premises **$553.69** POSTING DATE



This refusal is the best-built thing in the module and it is kept exactly as it is.

The read model considered three answers and called two of them defects: an **empty table**, which says *"no activity this period"* to the one person who most needs to know otherwise; and **one row labelled Unassigned holding the whole book**, which sorts, exports and reads like a premises and is a finding disguised as data. It chose the third — **the money at stake, stated as a figure**, beside a sentence naming what is missing.

The KPI band and the table are built, wired, and never drawn here, because `kpis` reads 0/0.00 on an unavailable response and four zeroed tiles over a book holding $553.69 is the plausible-zero defect in its purest form. **Schedule is disabled** — you cannot schedule a report that cannot be produced.

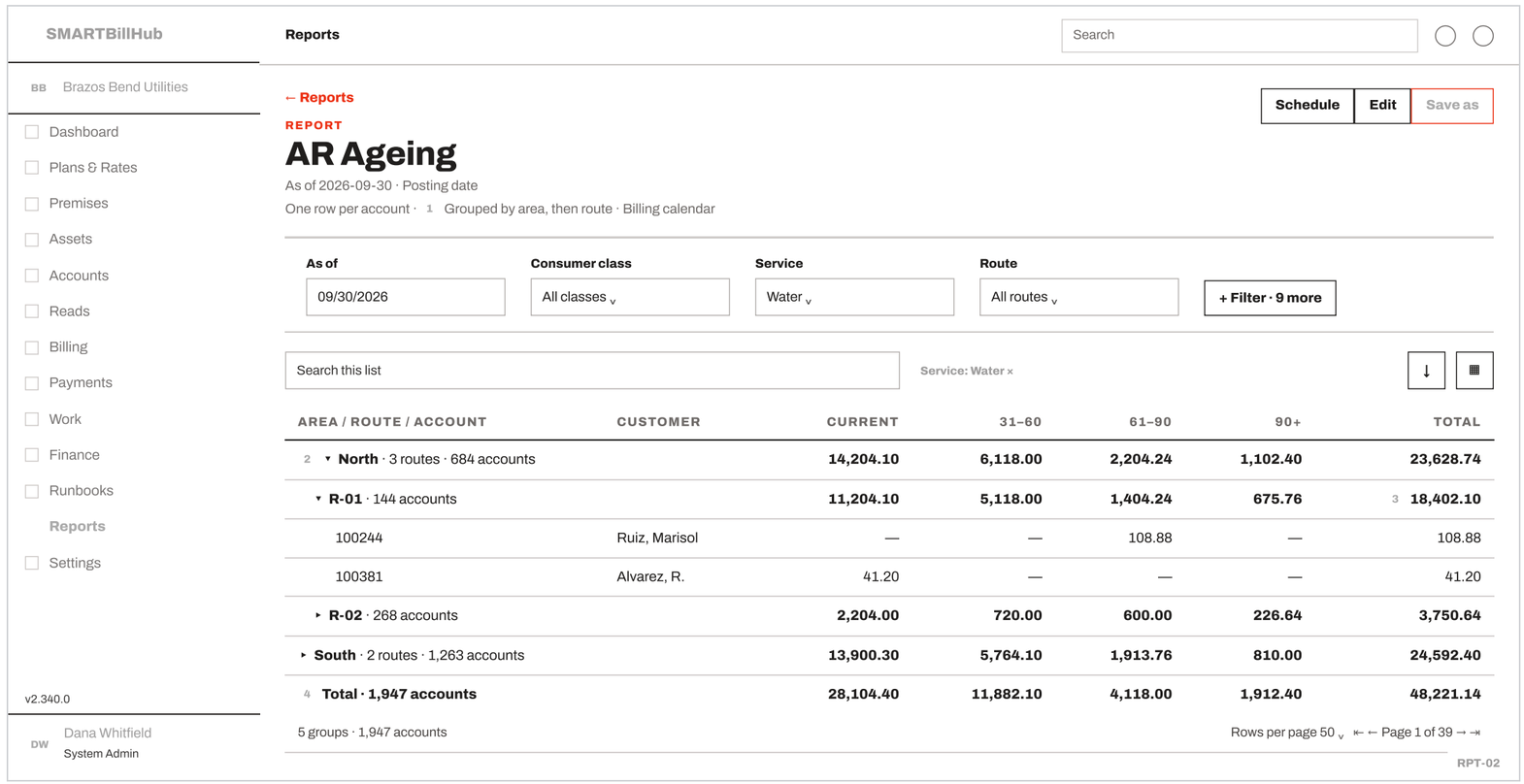
⚠ GAP **One sentence on this screen must go.** Today a note in the parameter bar says choices are withheld *"while the premises dimension is missing"* — a product-wide claim **superseded on 2026-08-16**, when posting began stamping the chain. The backend split its gap in two specifically to stop saying it, and warned why: it tells a clerk the books hold no premises while the table below shows them premises. **P2.8**

FOR	A finance officer or billing clerk running any of the 46. **This screen computes nothing** — no sum, no percentage, no rounding. Every figure is the owning module's
DOORS	**One, and only one: the catalogue card.** Nothing else in `07-frontend/src` navigates to a report key — they exist only as a segment↔key table at `reportsShared.tsx:79-84`. **The dashboard links to no individual report and neither does Finance.** After P2.8 the three retired screens add three more doors, from Finance → Revenue & AR
PRIMARY ACTION	**Save as.** It is the filled button because it is the only one of the three that commits a row — Schedule and Edit open a surface. Export is the toolbar's icon, not a page action, because it belongs to the table
FIELDS	In JSX order: back link · kicker · title · window/as-at · basis label · the definition's parameters · KPI tiles · toolbar · column headers · rows · totals row · pager · gap panel
TABLE CONTROLS	**Search** NEW, server `q` — P2.3 · **Filter** the parameter bar, declared per definition · **Columns** NEW, `DataTable`'s chooser, remembered per table · **Sort** NEW, SQL, declared per dataset — P2.2 · **Pagination** NEW, SQL — P2.1. **Today all five are absent on both sides**: `GET /reports/{key}` reads no `page`, `per`, `sort` or `dir` at any of six keys
WRITE SURFACES	**Detail** — a row opens its record in the owning module (NEW; today a row is inert text). **Add** — NOTHING, correctly. **Edit** — 2·2, and it edits *the report*, never the data. **Update** — NOTHING. ⚠ Issued invoices, payments and journal entries take no UPDATE and no DELETE, so an edit control over any of them would be a finding, not a feature
TODAY TODAY	Renders four canned reports with a parameter bar, KPI tiles and a plain table. No paging, sort, search or column choice. Walked 2026-09-23 on an empty book: **three of the four printed a false multi-currency sentence** and one printed the internal code `currency_not_derivable` beneath it
TASK TASK	**P2.1–P2.4** server side · **P2.5** the screen on `DataTable` · **P2.6** the class control · **P2.7** the date fields · **P2.8** the three retirements
WALKED	**YES** — all four canned reports and all three bespoke ones, 2026-09-23

RULES	Obeys B14 2 · 5 (two button tiers) · 31 after P2.8 · `BR-REP-001` (basis in three places) · `BR-REP-002` (the screen computes nothing) · `BR-REP-024` . ✗ **Breaks today** B14 7 (bare date inputs) · 8 (an 8-option select with no type-to-filter) · 12 · 14 · 16 · 17 (no empty state — an empty result draws a header and a zeroed Total row) · 31 (an internal code on screen) · 33 (33 words of rationale in a note). Every one is closed by a task in Phase 2

2·1E · REPORT — GROUPED, WITH SUBTOTALS TASK P2.17

NEW STATE · DESKTOP 1440 · AREA → ROUTE, SUBTOTAL AT EACH LEVEL



1 **Three facts in the head** · grain · **what it is grouped by** · which calendar · **NEW**

2 **A group row** · ▾ **open** · ▸ **closed**, expands in place, no page load · **NEW** · **P2.24** · it carries its own count so a subtotal is never a number without a denominator

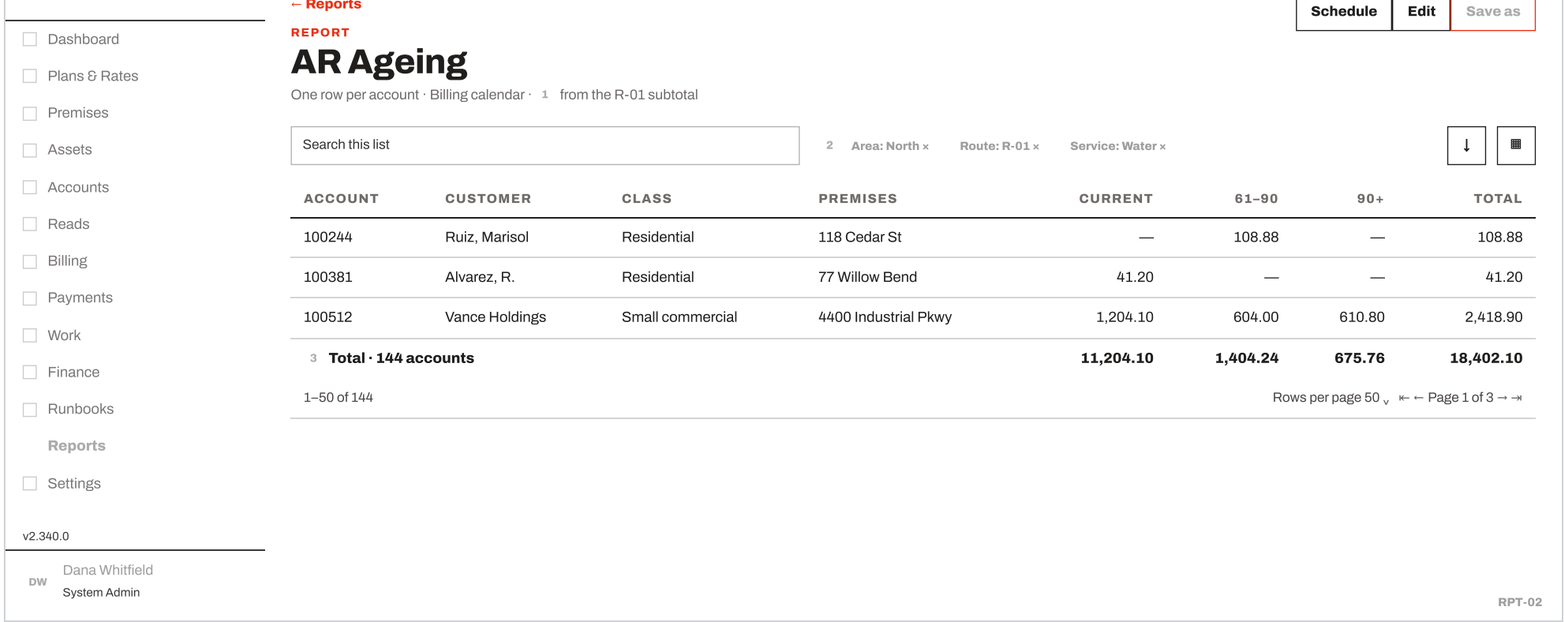
3 **A subtotal is a link** · **$18,402.10** opens the 144 accounts behind it, with area and route already applied as filters · **NEW** · **P2.18** — ⚠ **a subtotal you cannot check is a number a clerk will not defend to a customer**

4 **The grand total still foots** · 23,628.74 + 24,592.40 = 48,221.14 · ⚠ **lawful here because a balance sums across accounts.** Group this report by *month* instead and the same column refuses — see **2·1h**

2·1F · REPORT — DRILLED THROUGH TO THE ROWS TASK P2.18

NEW STATE · DESKTOP 1440 · WHAT $18,402.10 IS MADE OF





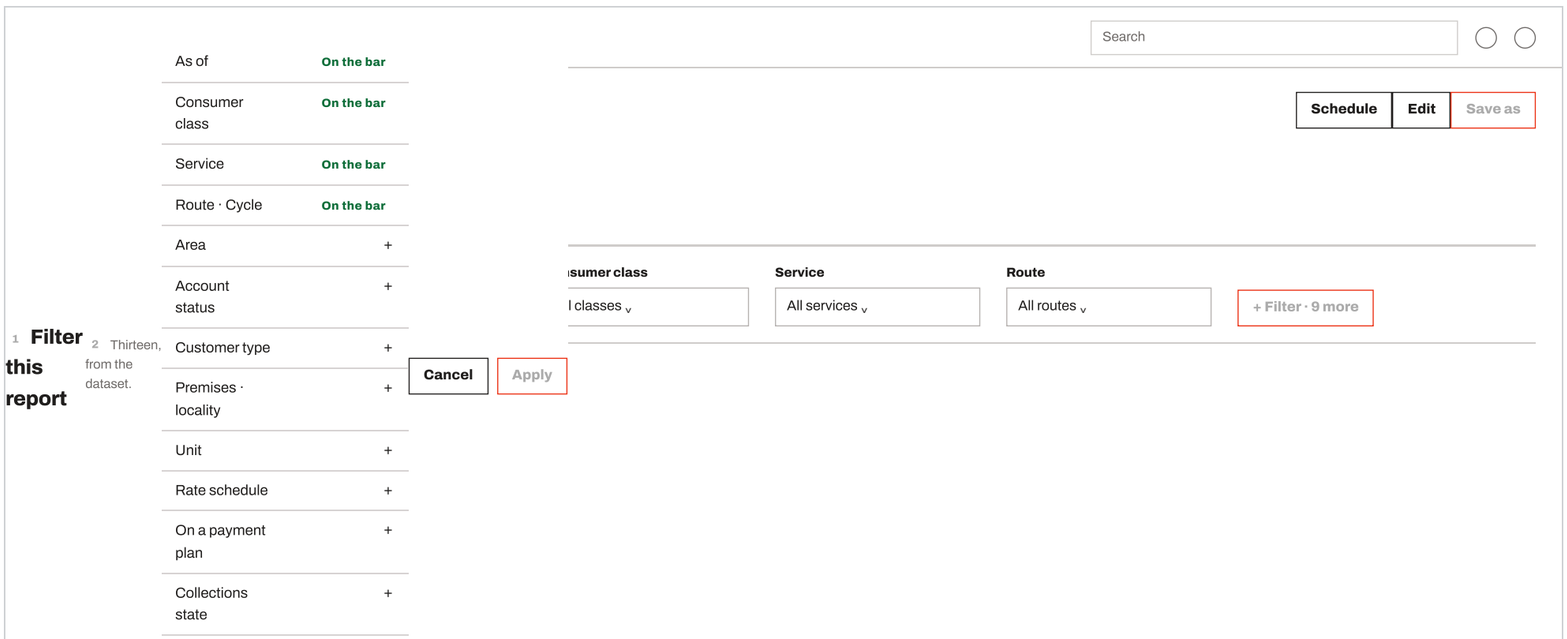
1 **Where you came from**, in the head · **NEW** · four words. Without it a clerk who followed a subtotal has no idea why the report is showing 144 rows instead of 1,947

2 **The group becomes chips** · area and route arrive **as ordinary, removable filters** · **NEW** · ⚠ **Not a special mode.** Take the chips off and you are back on the whole report — a drill-through you cannot escape is a trap

3 **It reconciles, exactly** · **18,402.10** here and **18,402.10** on the group row above · ⚠ **that is the whole point of the feature**: a subtotal a clerk can open is a subtotal they can defend

2·1G · REPORT — + FILTER OPEN TASK P2.26

NEW STATE · DESKTOP 1440 · **THE DIRECT ANSWER TO “TWO SLICERS WHERE THE FACT HAS THIRTEEN”**



Age bucket 3 in the columns

RPT-02

1 **Not a settings screen** · it applies to **this run**, not to the definition · **NEW** · ⚠ changing what appears *permanently* is the edit panel's job — two ways to change one thing is how they diverge

2 **Thirteen, and it says thirteen** · **NEW** · ⚠ **the count is the point.** A tick-list without one always raises *is this all of them?*, and here the honest answer is that the dataset declares exactly these and a report cannot reach past them

3 **Age bucket is already on screen**, as the columns · **NEW** · it is a real dimension and it is **not offered twice** — a list that repeats what the table already shows teaches a clerk to stop reading it

Why four on the bar and nine behind a button.

Thirteen slicers drawn at once is **a worse screen, not a better one** — it pushes the table below the fold and breaks B14 rule 32's description budget. The **definition** names which four a utility sees first, so AR Ageing opens on the four a Brazos Bend clerk actually uses and a different utility can pick differently. ⚠ **Everything remains reachable in one more click**, which is the difference between a considered default and a limit.

2·1H · REPORT — A TOTAL THE ENGINE REFUSES TASK P2.22

NEW STATE · DESKTOP 1440 · **THE SAME REPORT, GROUPED BY MONTH INSTEAD OF ROUTE — AND NOW THE COLUMN CANNOT BE SUMMED**

SMARTBillHub

Reports

Search

BB Brazos Bend Utilities

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

v2.340.0

DW Dana Whitfield
System Admin

← Reports

REPORT

AR Ageing

One row per account · 1 Grouped by month · Calendar

Schedule | Edit | Save as

Search this list

MONTH	ACCOUNTS	CURRENT	31–60	61–90	90+	TOTAL OWED
July 2026	1,921	26,880.10	10,204.00	3,918.20	1,704.00	42,706.30
August 2026	1,934	27,410.55	11,002.40	4,001.10	1,808.20	44,222.25
September 2026	1,947	28,104.40	11,882.10	4,118.00	1,912.40	48,221.14
2 **Total**	—	—	—	—	—	—

3 **Why there is no total**

A balance does not add across time. Each month is a position, not an amount earned.

3 months

Rows per page 50 ⌄ ⇤ ← Page 1 of 1 → ⇥

RPT-02

1 **Calendar, not Billing** · grouped by month, so the head says so · **NEW** · ⚠ the same report grouped by *cycle* is a different question with different rows, and a clerk reading a screenshot a month later needs to know which

2 **An em-dash in every money column** · **NEW** · **P2.22** · ⚠ **This is the money task.** Today this row shows **$135,149.69** — the three months added together — and that number is **meaningless**: it is the same debt counted three times. **Nothing in the product stops it, and nobody would notice**. Note the *Accounts* column refuses too: 1,921 + 1,934 + 1,947 is not 5,802 customers, it is roughly 1,950 counted three times

3 **The reason, in twelve words over two sentences** · **NEW** · rule 33 caps an explanation at six words per sentence and prefers two short ones to one long one. ⚠ **The reason is a FACT, not rationale** — it says what is true of the data, not why we built it this way, and that is the line rule 33 actually draws

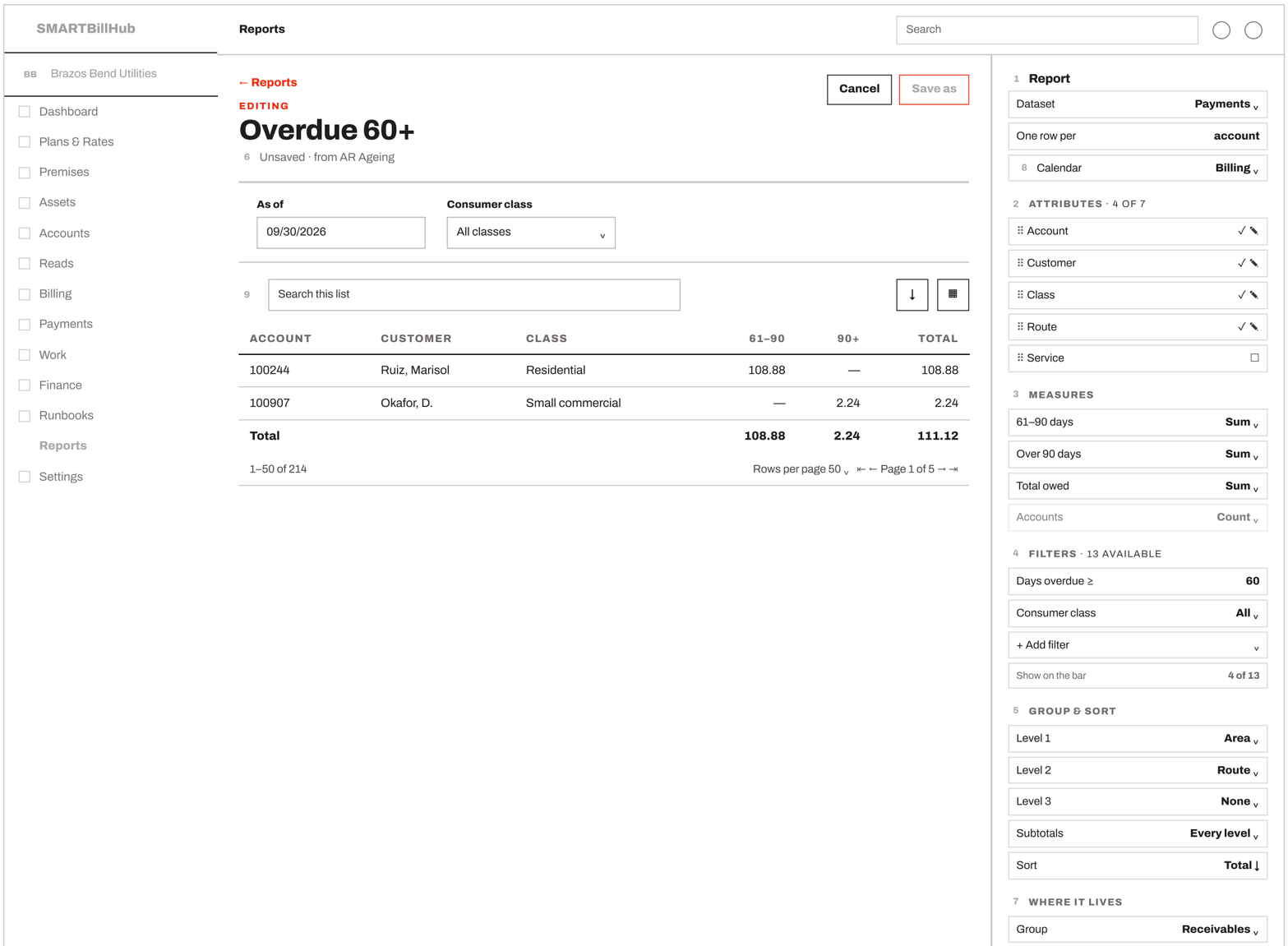
Three things are wrong on this screen today and none of them would ever be reported as a bug.

Group AR Ageing by month and total it — **wrong**, the same debt counted three times. Average four routes' DSO to get the utility's DSO — **wrong**, you must recompute it from billed and collected. Add water gallons to electric kilowatt-hours — **wrong**, and hard rule 3 already forbids it while nothing enforces it.

All three produce a plausible number. That is what makes them dangerous: a wrong total that looks absurd gets found in a day, and one that looks reasonable gets put in a board pack. ⚠ **Declaring additivity once, in** `P0.16` **, turns all three into this screen** — which is the entire argument for spending a phase on metadata that a user never sees.

2·2 · REPORT — EDIT TASK P5.3 RULED PANEL

END STATE · DESKTOP 1440 · THE SAME TABLE, WITH A PANEL BESIDE IT



v2.340.0

DW Dana Whitfield
System Admin

Sub-group **Arrears** ⌄
Services **All** ⌄
Visible to **Onlv me** ⌄

RPT-02

1 **Report** · dataset · **the grain** — *one row per account* · **the calendar** · **NEW** · `GET /reports/datasets` , **P0.4** · ⚠ changing the dataset resets columns and filters, because they belong to the dataset

2 **Attributes** · what a row *is* — **4 of 7 shown, and the header says so** · **NEW** · **Tick** to show · ⠿ **drag** to reorder · ✎ **rename the heading**, which is how *Account* becomes *Account No*. ⚠ **An unticked column is an empty box, never greyed** — grey means *you cannot*, and these you can; the first draft greyed them and that is the defect the PO found

3 **Measures** · what a row *counts*, split out from the attributes · **NEW** · ⚠ **each one carries its own aggregation**, so the footer totals what the measure says rather than what the renderer assumed — **P2.21**. *Accounts* is dimmed because it is `Count` and cannot be anything else

4 **Filters, from the dataset** · **13 available** and the header says so · **NEW** · ⚠ **This is the fix.** Filters are no longer hand-written per report; they are every dimension the dataset declares. **Show on the bar** picks the four that appear as slicers on 2·1 — **P2.17**, **P2.26**

5 **Group to three levels** · Area → Route → Premises, with a subtotal at each · **NEW** · one level was all a definition could hold — **P2.17**

6 **Unsaved marker** · in the rollup, where state belongs · B14 rule 2 · **NEW**

7 **Where it lives** · group, sub-group, services and visibility · **NEW** · groups from `report_categories` (register row 1), visibility a signed two-value enum (row 12), services from the tenant's enabled set (row 3)

8 **Calendar** · **Billing** or **Calendar**, one per report · **NEW** · ⚠ **A cycle straddles two months.** Every time control on the report uses this one, and filtering by accounting period while grouping by cycle is **refused at save**, naming both — **P2.25**. Nothing else on screen would reveal the mix

9 **The table is live** · every change above re-runs the preview underneath · **NEW** · **this is why it is a panel and not a wizard** — a forward fence would hide the one thing you are trying to see

"The user cannot add columns" — the PO, looking at this panel. Three parts, and two are mine to fix.

Fixed here. You could not **reorder** a column and you could not **rename** its heading. Both are definition data, both are cheap, and a report you cannot reorder is half a builder — **P5.11**. And an unticked column was drawn *greyed*, which reads as *you cannot* when the truth is *you have not*. It is an empty box now.

Not fixed, and stated rather than hidden. You cannot invent a column the dataset does not produce. **That is the architecture, not an oversight:** datasets are code and definitions are data, which is the whole reason a utility can author reports at all without us shipping them a fork. The panel says how many columns the dataset has — *7 available* — so a clerk can see the list is complete rather than wondering what is hidden.

OPEN **A computed column** — *Total = 61–90 plus 90+* — is genuinely definition data and would not need a dataset change. It is **not in this plan**, because it is an expression editor: a grammar, a validator, a type system and a set of error messages. **If a utility asks for it, that is a change request with a named requester and a figure**, not something added because it seemed nice. Raising it here so the absence is a decision rather than an omission.

FOR	A clerk or analyst answering a question we did not write a report for
DOORS	**One: the Edit button on 2·1.** Nothing else reaches it, deliberately — you always edit a report you are already looking at
PRIMARY ACTION	**Save as.** ⚠ **Editing a built-in never overwrites it** — `tenant_id` is null on a platform row and copy-on-write is the rule (**P5.1**). So the filled button says *Save as*, not *Save*, and that wording is the safety
FIELDS	Dataset · columns (tick per declared column) · filters · group by · sort · name · group · sub-group · services · visibility
TABLE CONTROLS	Unchanged from 2·1 — the table below **is** 2·1, running live
WRITE SURFACES	**Detail** NOTHING · **Add** Save as → `POST /reports/definitions` · **Edit** of a tenant-owned row → `PATCH` · **Update** as Edit. ⚠ **A definition is user input and names columns, never SQL** — **P5.2** validates every name against the dataset's declaration

TODAY TODAY	**Does not exist.** S47, the report builder, is in the canonical screen inventory as a v1 screen and **has no route and no component**. Its three stories are still `draft`
TASK TASK	**P5.1** CRUD · **P5.2** validation · **P5.3** the panel
WALKED	**NO** — nothing to walk. Drawn from the settled architecture and the existing table component
RULES	Obeys B14 5 · 12 (the table keeps its own toolbar) · 43 (**the panel is a component, not standalone code inside a page**). ⚠ It **departs from** the 42-file `WizardFrame` convention, ruled 2026-09-26

④ What Save as changes, and where

Writes:

one row in `report_definitions` with `tenant_id` set and `owner_user_id` = you · one `audit_log` row.

The ledger:

nothing. A report cannot touch it.

Elsewhere:

the report appears in

1·1

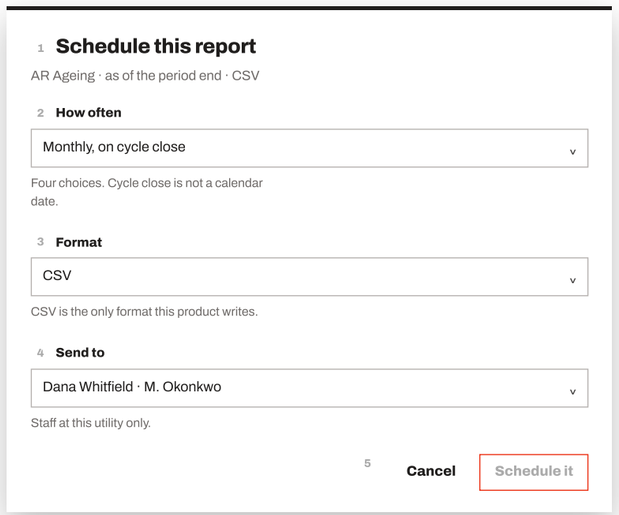
under *Mine* — and, only if visibility is *Shared*, in every colleague's catalogue too. It becomes schedulable on

2·3

. Nothing else in the product changes, and no customer receives anything.

2·3 · SCHEDULE TASK P4.4

END STATE · DESKTOP 1440 · **ONE FORM, THREE FIELDS** — NOT A WIZARD.
SEE THE CORRECTION IN §02



RPT-02

1 **The dialog** · `FormModal` , the shared one · **NEW** · ⚠ **Not** `WizardFrame` **.** The report and its parameters are already chosen — you are standing on them. Three fields do not need a rail

2 **How often** · daily · weekly · monthly · **monthly, on cycle close** · **NEW** · **SIGNED**, register row 13 · ⚠ the fourth is the one a utility actually wants and the one a generic cron picker loses

3 **Format** · **NEW** · **SIGNED with a deliberate hole**, register row 14 — CSV only, because **no PDF renderer is declared anywhere in the product**. Adding PDF here puts a control on screen that produces nothing `OPEN`

4 **Send to** · **NEW** · **SERVED — this tenant's own users, and nothing else**. `BR-REP-020` `RULED`: external recipients arrive at W6 with an explicit confirm step, because external send means data leaves the tenant. **A free-text email box here breaks a signed ruling**

5 **Footer order** · Cancel ghost, then the filled commit · B14 rule 38 — where a drawing and our UI rules disagree about how a form is built, **the UI rules win**

FOR	Anyone who runs the same report every month and would rather not remember
DOORS	**One: the Schedule button on 2·1.** ⚠ It is **disabled** where the report cannot be produced — you cannot schedule a refusal (see 2·1d)
PRIMARY ACTION	**Schedule it.** Filled, because it commits a row that will act on its own later — which is exactly when a confirm tier is warranted
FIELDS	How often · Format · Send to. **Three.** The report, its parameters and its name are inherited from the screen behind
TABLE CONTROLS	None — not a list
WRITE SURFACES	**Detail** NOTHING · **Add** `POST /reports/schedules` · **Edit / Update** on 4·2, not here
TODAY `TODAY`	**Does not exist.** Nothing in the product stores a report schedule; `report_schedules` has no table and celery beat is not deployed. The library says so in one sentence at its foot, and that sentence is currently true
TASK `TASK`	**P4.1** table · **P4.2** scheduler · **P4.3** recipients · **P4.4** this dialog
WALKED	**NO** — does not exist
RULES	Obeys B14 5 · 13 (two fields to a row where two fit) · 33 (helper text is a fact, not rationale) · 38 (footer button order is ours, not the drawing's)

④ What Schedule it changes, and where

Writes:

one row in `report_schedules` with `next_run_at` computed · one `audit_log` row.

The ledger:

nothing.

Elsewhere:

the schedule appears on

4·1

immediately. At `next_run_at` the scheduler enqueues a `jobs` row, which appears on

5·1

, produces a file, and sends

named internal staff

a link through the outbound queue —

a link, never the file

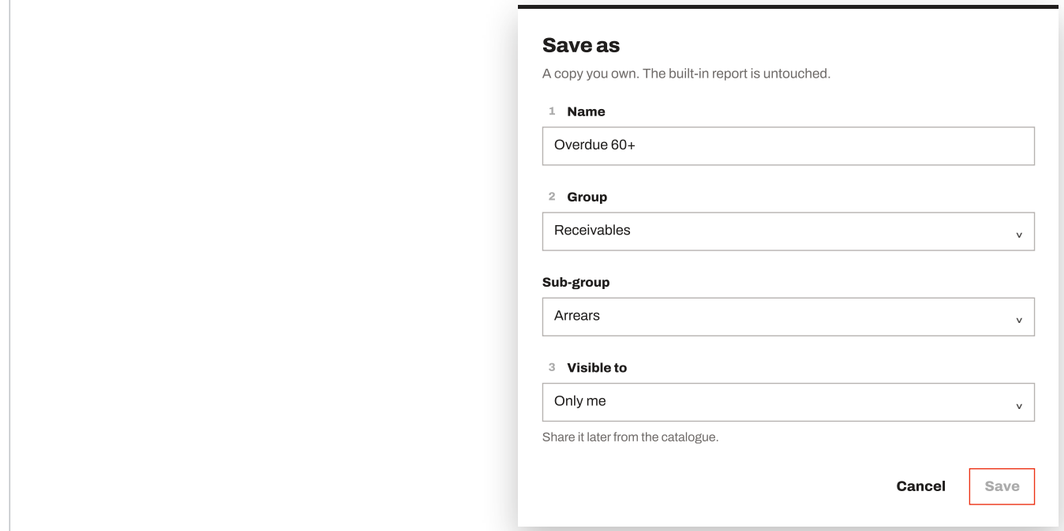
, so the recipient's own permissions still apply. ⚠ The idempotency constraint on `jobs` means a schedule that fires twice produces

one

file.

2·4 · SAVE AS TASK P5.4

END STATE · DESKTOP 1440



RPT-02

1 **Name** · **NEW** · ⚠ **this is the field the whole framework turns on**. Today a report's name lives in **eleven locale JSON files**, which a utility cannot edit — so until the name comes out of a row, nobody outside Bynry can author a report. `report_definitions.name` , **P0.1**

2 **Group and sub-group** · **NEW** · `report_categories` , register rows 1–2 — **including categories this tenant added itself**

3 **Visible to** · Only me · Everyone here · **NEW** · **SIGNED**, register row 12 · `BR-REP-019` RULED — **creator-private by default**, with an explicit share toggle. The default is the ruling, not a preference

FOR	Anyone who has changed a report and wants it back tomorrow
DOORS	**Two: the Save as button on 2·1 and on 2·2.**
PRIMARY ACTION	**Save.** One filled button; the dialog exists to commit
FIELDS	Name · Group · Sub-group · Visible to. Four
TABLE CONTROLS	None
WRITE SURFACES	**Add** `POST /reports/definitions` . All others NOTHING
TODAY TODAY	**Does not exist.** Nothing stores a report definition
TASK TASK	**P5.4**
WALKED	**NO**

RULES	Obeys B14 12 (grid of two) · 33 (the helper is a fact in six words) · 38

Tab 2 · Dashboard

This is a NEW screen and it does not touch the one you already have.

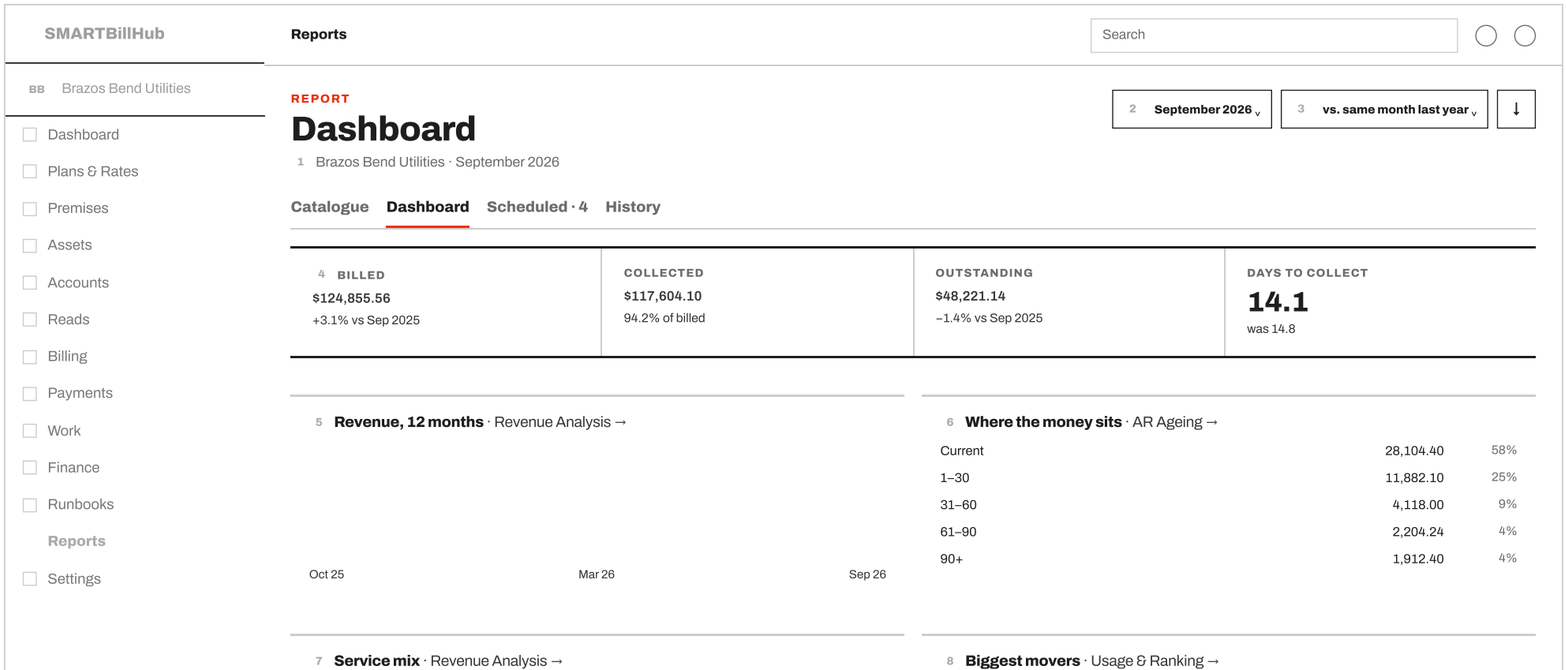
PO ruling, verbatim: *"no i dont want to update existing dashboard ... you can create a tab - dont replace existing one"*. So the shell's dashboard at `/{tenant}/dashboard` — rail item 1, the screen after every sign-in — is **untouched by every task in this plan**. Not a tile, not a label, not a column. It is described in **Elsewhere** at the end of this section *only* because `apps/reports/` serves it and a build lane has to know that before editing anything in there.

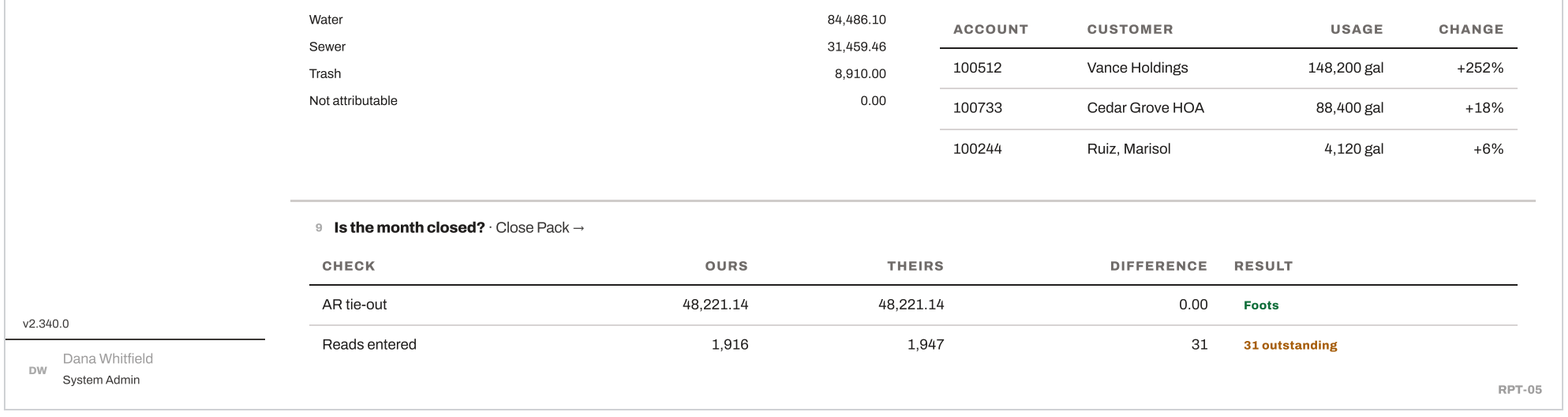
Two dashboards is the right number here, because they answer different questions to different people. One is a clerk's morning glance — *what needs me today*. This one is an owner's monthly read — *how are we doing*. Merging them produces a screen that is too busy to glance at and too shallow to analyse, which is the usual fate of a single dashboard asked to do both.

① THE LIST	**3·1 Dashboard** — not a list. Four KPIs and five panels, each panel one of the catalogue's own reports
② ONE ROW OPENED	Every panel opens **its report**, on this tab's period, in the report screen. ⚠ **Nothing opens a detail pane here** — a dashboard you can drill into has become a report, badly
③ WHAT YOU CAN DO	Set the period · set what to compare it against · open a panel. **That is the whole surface**
④ WHAT IT CHANGES	**Nothing. No write of any kind**, not even a remembered period

3·1 · DASHBOARD TASK P4.11 RULED · A NEW TAB, NOT A REPLACEMENT

END STATE · DESKTOP 1440 · TAB 2 · **THE ONLY SCREEN IN THIS PRODUCT WITH A CHART ON IT**





1 **Page head** · B14 rule 40, 32px · **NEW** · the sub-line names the tenant and **the period in words**, so a screenshot pasted into a board pack still says what it is

2 **Period** · **NEW** · ⚠ **a real period, not a rolling window.** This is the difference from the shell dashboard: a manager reads *September*, and a rolling thirty days straddles two cycles and answers nobody's question · **SOURCE: SERVED**, the tenant's own accounting calendar

3 **Compare against** · **NEW** · same month last year · previous month · budget. ⚠ **The whole reason this screen exists** — a number with nothing beside it is not a performance report, and the shell dashboard deliberately has no comparison at all

4 **Four KPIs, not five** · **NEW** · billed, collected, outstanding, days to collect. Each carries its comparison underneath. ⚠ **A tile whose owner publishes no count reads *Not counted yet* with its reason** — the same rule the shell dashboard already obeys, and the one **P2.13** applies to the report screen

5 **Revenue, 12 months** · bars, current period accented · **NEW** · opens **Revenue Analysis** on this period

6 **Where the money sits** · the five signed ageing bands · **NEW** · shares rounded and **not forced to 100**, which is why they read 58+25+9+4+4. ⚠ **One colour for every bar, on purpose.** A first draft coloured them by size, so the *Current* band — the healthy one — came out warning-coloured because it was the biggest. **Colour that encodes magnitude reads as a judgment**, and on an ageing chart it is the exact opposite of the truth

7 **Service mix** · every enabled service, in seed order, plus the **named remainder** · **NEW** · ⚠ bars are proportional to the LARGEST service and never to a total, because eleven services can be enabled and a stacked bar of eleven is unreadable

8 **Biggest movers** · top 3 by change · **NEW** · ⚠ **one service at a time, always** — ranking gallons against kilowatt-hours is the commonest way a usage report lies

9 **Is the month closed?** · the Close Pack tie-outs · **NEW** · ⚠ **the one panel that can be red**, and it is at the bottom on purpose: a dashboard that opens on a problem gets ignored, and one that never shows a problem gets ignored faster

ADDRESS	`/{tenant}/reports/dashboard` — ⚠ `dashboard` is the fourth **reserved** segment a definition may not be named
CLASS	`NEW` · added by PO ruling after the first three tabs were drawn
DOORS	**One, and that is a decision rather than an oversight.** The **tab strip**, from anywhere in Reports. ⚠ **Nothing links here from the shell dashboard**, which would be the obvious second door — and it is ruled out: the PO fenced that screen (*"dont replace existing one"*), and adding a link to it is touching it. ⚠ **Nor from the left rail**: rail item 12 lands on Catalogue. So this screen is **two clicks from sign-in** and that is its cost, accepted openly rather than paid for by editing a fenced screen
REPO	Nothing. ⚠ **And nothing in** `apps/reports/dashboard.py` **is reused** — that file serves the shell's screen and is not touched. This tab reads the same definitions the catalogue does
DATASETS	Five, through their **definitions** and never directly: Revenue Analysis · AR Ageing · Usage & Ranking · Close Pack. ⚠ **That is the design** — a panel is a saved report rendered small, so the dashboard and the report can never disagree, and a utility that changes a report changes this with it
TABLE CONTROLS	**None.** No panel here is a table in the `DataTable` sense — no search, no paging, no column picker. The full report has all four
WRITE SURFACES	**Detail** every panel opens its report · **Add / Edit / Update NOTHING**. ⚠ Not even the period is remembered — reload and it returns to the

	current period
TODAY TODAY	**Does not exist.** ⚠ And neither does **any chart anywhere in this product** — there is no charting library in `07-frontend/package.json` . The bars above are drawn in CSS, which is deliberate: five panels do not justify a dependency, and a CSS bar prints, scales and theme-switches for free
TASK TASK	**P4.11** the screen · **P4.12** the period-and-comparison service · **P4.13** the panel→report handoff, which must carry the period across
WALKED	**NO.** Nothing to walk. ⚠ The shell dashboard *was* walked and is reported in **Elsewhere** — do not read that walk as evidence about this screen
RULES	Obeys B14 5 · 31 · 32 · 33 · 40 · 43. ⚠ **Rule 43 is the one to watch**: the shell dashboard's four panels are inline in a 4,000-line page and cannot be reused. **Every panel here is a component from the first commit**, or this screen becomes the second place that mistake lives

What this tab must never become, stated now because it is the obvious next request.

Configurable panels. "Let each utility choose which reports appear here" is a reasonable thing to want and it is **not in this plan**. It needs a layout store, a picker, a size model and a per-role default, and it turns a screen anyone can read into a screen that looks different at every customer — which makes support harder and this document wrong. If a utility asks, it is a change request with a named requester and a figure, not a thing we add because it seemed nice.

A second set of numbers. Every figure here comes from a definition in the catalogue. ⚠ **Nothing on this screen computes anything.** The moment a panel does its own arithmetic, the dashboard and the report start disagreeing, and the utility stops trusting both.

④ What the period and comparison controls change, and where

Writes:

nothing. Not a preference, not a cookie, not a row.

The ledger:

nothing.

Elsewhere:

all four KPIs and all five panels re-read. The period rides in the URL, so the link you copy opens on the same month — which is what makes this screen usable in a board pack at all. ⚠

And it carries into the report

: opening a panel opens that report

already set to this period

. A dashboard that drops you into a report on a different window is worse than one with no links, because the numbers change under you and nothing says why.

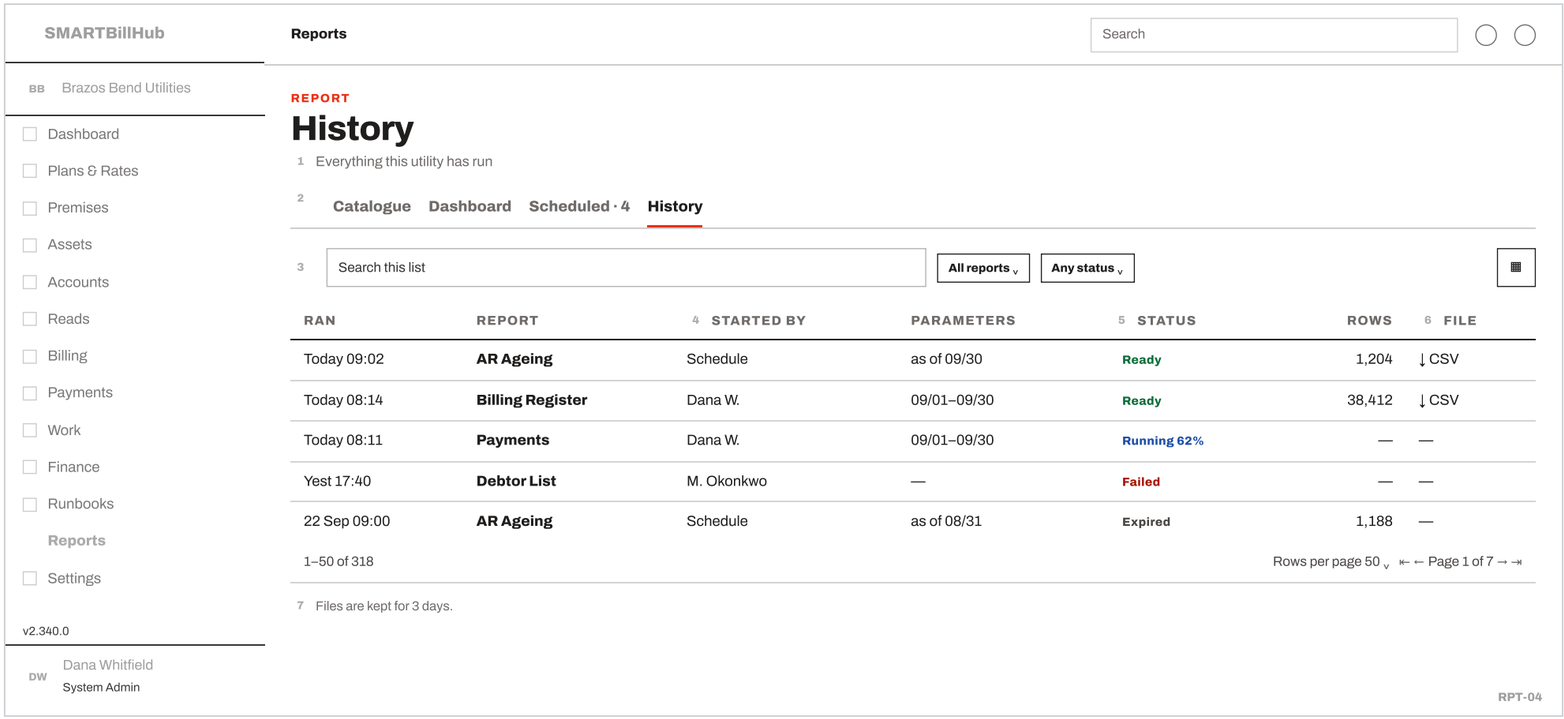
Tabs 3 and 4 · Scheduled, History

① THE LIST	**Scheduled** and **History** — **two top-level tabs**, two tables. A schedule is a run that has not happened yet; a run is a schedule that fired, or somebody pressing a button. ⚠ **They were nested inside one tab called Activity in an earlier draft. The PO ruled flat, and Activity is deleted** — the word and the address
② ONE ROW OPENED	**4·2** a schedule · **5·2** a run
③ WHAT YOU CAN DO	On a schedule: pause · edit · run now · delete. On a run: download · re-run
④ WHAT EACH	In the consequence blocks below. In summary: pause and edit touch **one schedule row**; run now and re-run write **one job row** and produce a

CHANGES	file. **No business data is written by anything on this tab**

5·1 · HISTORY TASK P3.2

END STATE · DESKTOP 1440 · THE CHEAPEST SCREEN IN THE PLAN AND THE ONE THAT CLOSES A LIVE BROKEN PROMISE



1 **Page head** · B14 rule 40, 32px · **NEW**

2 **Two tabs** · Scheduled with a live count, History · **NEW** · ⚠ the tab is in the URL so a link to History is a link to History

3 **Toolbar** · `DataTable` 's own · **NEW**

4 **Started by** · a person's name, or the word **Schedule** · **NEW** · `jobs + audit_log` — this is also **F5**, the record of who ran what

5 **Status** · `StatusPill` · **SIGNED**, register row 16 — `core.jobs.STATUSES` , which the table already owns

6 **File** · the download the job left behind, or an em-dash · **NEW** · `GET /api/exports/{job_id}/file` — **already served**

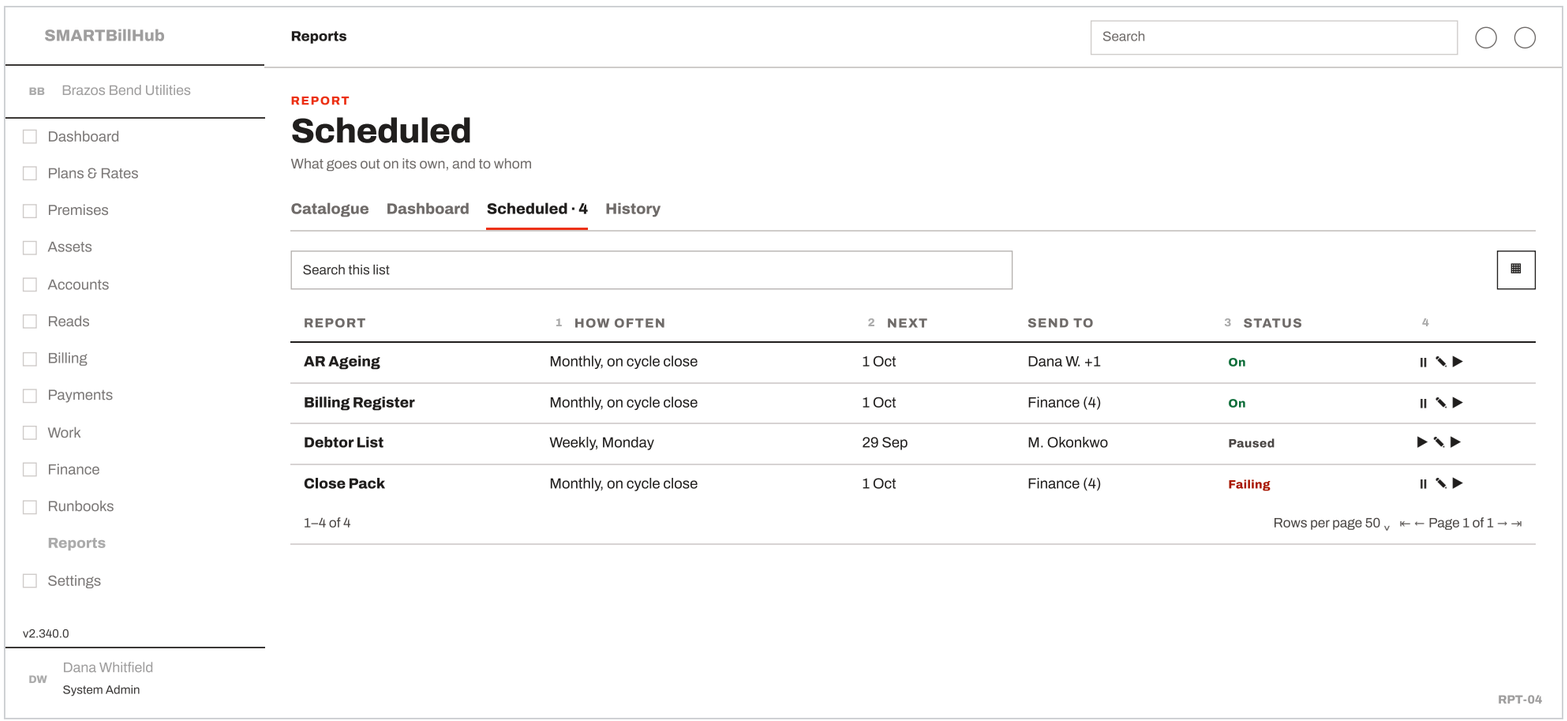
7 **Retention** · six words, a fact not a rationale · B14 rule 33 · ⚠ `BR-REP-018` makes this **tenant-configurable** and that half has no storage — **P4.6**

FOR	Anyone looking for a file they produced, or checking whether last night's schedule ran
DOORS	**Three.** The *History* tab in the strip · the export toast on 2·1 · a direct link in the schedule notification email. ⚠ **TODAY THE COUNT IS ZERO.** The toast already says *"It appears in Jobs when it is ready"* and **there is no Jobs screen anywhere in the product** — no route, no nav word, no component. **P3.3**
PRIMARY ACTION	**None.** A history is read. The per-row download is a link, not a page action
FIELDS	Ran · Report · Started by · Parameters · Status · Rows · File
TABLE CONTROLS	**Search** server · **Filter** report and status · **Columns** the shared chooser · **Sort** newest first, SQL · **Pagination** SQL — all five from `DataTable` , all NEW

WRITE SURFACES	**Detail** 5·2 · **Add** NOTHING (a run is created by running a report) · **Edit** NOTHING — ⚠ **a run is a historical fact and is never editable** · **Update** NOTHING
TODAY TODAY	The **back end exists in full** — a `jobs` table with status, progress, checkpoint, `result_link` and an idempotency constraint; one shared client poller; a file route; a 3-day sweep. **Nothing renders any of it.** The only surface is `ListExportButton`, which polls inline and hands you a link on the screen you are already on
TASK TASK	**P3.1** the endpoint · **P3.2** the screen · **P3.3** the toast's destination
WALKED	**NO** — the screen does not exist. The absence of its door was walked and confirmed
RULES	Obeys B14 12 · 16 · 17 · 33 · 40

4·1 · SCHEDULED TASK P4.5

END STATE · DESKTOP 1440 · THE SAME SCREEN, FIRST TAB



1 **How often** · **SIGNED**, four values, register row 13 · **NEW** · ⚠ rendered as words, never a cron string — *"monthly, on cycle close"*, not `0 0 1 * *`

2 **Next** · `report_schedules.next_run_at` · **NEW** · **P4.7** proves the arithmetic for all four cadences

3 **Status** · On · Paused · **Failing** · **NEW** · ⚠ **a schedule whose report now errors stays in the list marked failing** and is not silently dropped — **P4.8**

4 **Row actions** · pause · edit · run now · **NEW** · B14 rule 15 — icons, capped at two plus an overflow

FOR	Whoever owns the month-end pack
DOORS	The Activity link, and the tab strip from 5·1
PRIMARY ACTION	**None.** ⚠ There is deliberately **no *+ New schedule* button here** — you schedule *from* the report you are looking at, so a create button on this

	screen would need a report picker that 2·3 gets for free
FIELDS	Report · How often · Next · Send to · Status · actions
TABLE CONTROLS	All five from `DataTable`, NEW. ⚠ Four rows today needs no pager and gets one anyway — **rule 16 has no small-list exemption**, and a utility with forty schedules is the case it is for
WRITE SURFACES	**Detail** 4·2 · **Add** NOTHING here, by design — 2·3 · **Edit** row ✎ → 4·2 · **Update** pause/resume in place → `PATCH /reports/schedules/{id}`
TODAY TODAY	**Does not exist.** No table, no scheduler, no screen
TASK TASK	**P4.1, P4.2, P4.5, P4.8**
WALKED	**NO**
RULES	Obeys B14 12 · 15 · 16 · 17

④ What pause, run now and delete change, and where

Pause

writes `enabled=false` on one schedule row and clears `next_run_at`; nothing else moves.

Run now

writes

a job row

, which appears on

5·1

within a second and produces a file — ⚠ it does

not

advance `next_run_at`, so the scheduled run still happens.

Delete

removes the schedule and

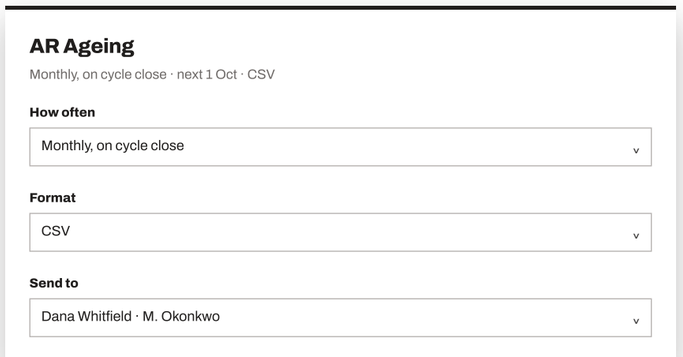
leaves every past run on 5·1

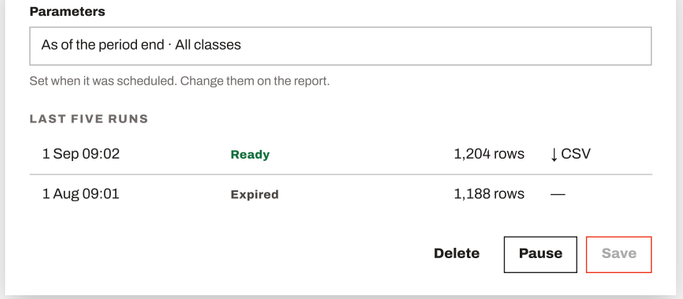
, because a run is a historical fact. All three write an `audit_log` row.

None touches the ledger, a bill, a payment or an account.

4·2 · SCHEDULE OPENED TASK P4.5

END STATE · DESKTOP 1440 · THE SAME THREE FIELDS AS 2·3, PLUS ITS OWN RUN HISTORY



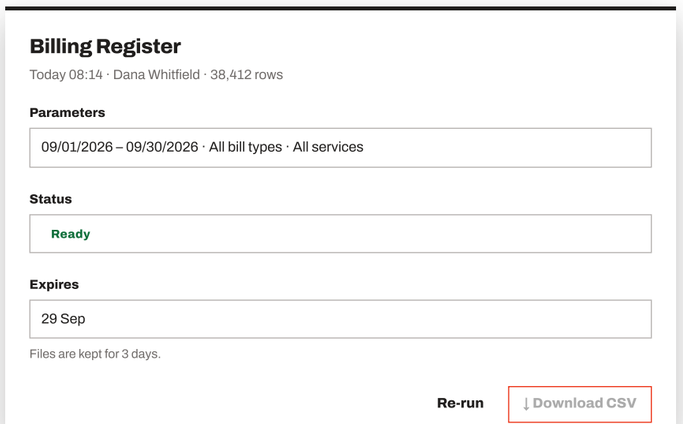


RPT-04

FOR	Changing or stopping a schedule somebody set weeks ago
DOORS	The ✎ row action on 4·1, and the report name in a schedule's failure notification
PRIMARY ACTION	**Save.** ⚠ **Delete is ghost, on the left, and far from Save** — B14's danger placement. It removes a schedule, not any data
FIELDS	How often · Format · Send to · Parameters (read-only) · the last five runs
TABLE CONTROLS	The embedded run list is capped at five and has none — ⚠ it is a summary inside a dialog, which is rule 16's stated exemption. **Full history is 5·1**
WRITE SURFACES	**Detail** a run row → 5·2 · **Add** NOTHING · **Edit / Update** `PATCH /reports/schedules/{id}`
TODAY TODAY	**Does not exist**
TASK TASK	**P4.5**
WALKED	**NO**
RULES	Obeys B14 5 · 13 · 16's wizard-summary exemption · 38

5·2 · RUN OPENED TASK P3.2

END STATE · DESKTOP 1440



RPT-04

FOR	Getting a file back, or reproducing a figure somebody is querying
DOORS	A row on 5·1, and a run row inside 4·2
PRIMARY ACTION	**Download CSV.** ⚠ **Labelled here and an icon on 2·1**, and that is not an inconsistency: on 2·1 it is `DownloadButton` in a toolbar, which the shipped component renders as a 36×36 icon; here it is a dialog footer, where a button carries its verb
FIELDS	Report · when · who · rows · parameters · status · expiry
TABLE CONTROLS	None — not a list
WRITE SURFACES	**Detail** this is it · **Add** Re-run → a new job row · **Edit / Update NOTHING** — ⚠ a run is a historical fact
TODAY `TODAY`	**Does not exist**, though the file route and the poller both do
TASK `TASK`	**P3.2, P3.4, P3.5**
WALKED	**NO**. The export that produces a run *was* walked: clicking download on 2·1 returned no error and toasted *"Your CSV is being written. It appears in Jobs when it is ready."*
RULES	Obeys B14 5 · 33 (the expiry helper is a fact in six words)

④ What Re-run changes, and where

Writes:

a new `jobs` row with
the original parameters, unchanged
· an `audit_log` row.
The ledger:

nothing.
Elsewhere:

a new row at the top of
5·1
; the old run and its file are untouched. ⚠ Two figures may differ, because the data moved —
which is the point of keeping both runs rather than replacing one
.

Screens this module serves, elsewhere in the product

Two screens are served by `apps/reports/` and are **not reached through Reports**. They are here because a lane editing that code has to know they exist — not because they are tabs. ⚠ **Neither is a tab and neither ever was;** calling them "Tab 4" and "Tab 5" in an earlier draft is what made the question *"what tabs will you have?"* necessary.

The shell dashboard is fenced off by PO ruling. No task in this plan touches it.

Verbatim: *"no i dont want to update existing dashboard ... you can create a tab - dont replace existing one"*. The Reports module gets **its own** dashboard at `/reports/dashboard`, drawn as **3·1** above. **This screen keeps every tile, every panel and every label it has today.** It is described below as it stands, and the only work against it is a **documentation** fix.

Elsewhere 1 · The shell dashboard

① THE LIST	**E·1 Dashboard** — not a list at all. Five tiles, four panels, one window. It is the only screen in this module a clerk does not choose to visit: it is where the shell lands them
② ONE ROW OPENED	Nothing opens. **Every panel ends in a link to the screen that owns the detail** — the audit log, revenue by service, Finance. A dashboard that lets you drill into it has become a report
③ WHAT YOU CAN DO	**Change the window, and follow a link.** That is the whole surface
④ WHAT IT CHANGES	**Nothing. No write of any kind.**

E·1 · DASHBOARD TODAY TASK P0.15

END STATE · DESKTOP 1440 · **THE ONLY KEEP IN THE REGISTER** — IT SHIPS, IT WORKS, AND **THE PO HAS FENCED IT OFF**. SHOWN AS IT STANDS TODAY

SMARTBillHub | Dashboard | Search

BB Brazos Bend Utilities

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

OVERVIEW

Dashboard

1 Brazos Bend Utilities · 21 Aug – 19 Sep 2026

2 Last 30 days ⌄ | Import reads | Run billing →

AR OUTSTANDING	REVENUE THIS PERIOD	UNBILLED READS	FAILED PAYMENTS	ACTIVE CONNECTIONS
$48,221.14 as at today	$124,855.56 posting date	31	Not counted yet payments publishes no count	1,947

3 **Recent activity** · last 24 hours

09:18	Dana W. corrected a payment reference
08:14	Billing register exported · 38,412 rows
Yest 17:40	Debtor list failed
Yest 16:02	M. Okonkwo approved an adjustment

See the audit log

4 **Service mix**

SERVICE	ACCOUNTS	REVENUE	SHARE
Water	1,947	84,486.10	67.7%
Sewer	721	31,459.46	25.2%
Trash	61	8,910.00	7.1%
Not attributable to a service	—	0.00	0.0%

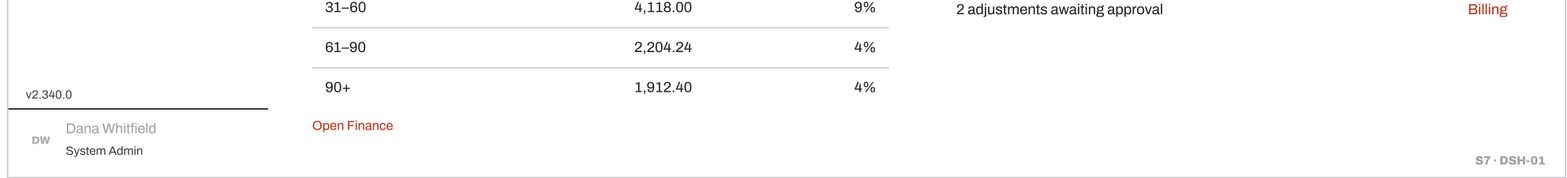
See revenue by service

5 **AR ageing**

BAND	AMOUNT	SHARE
Current	28,104.40	58%
1–30	11,882.10	25%

6 **To do**

31 reads not entered	Reads
11 consumption exceptions held	Reads
4 payments failed	Payments



1 **Page head** · B14 rule 40, 32px · **SHIPS** · the sub-line names the tenant and the window, which is how a screenshot of this screen stays readable a month later

2 **Window** · three options — *Last 30 days* · *This period* · *Last period*, in that order, defaulting to the first · **SHIPS, AND IS CORRECT** — see the box below · **SOURCE: SERVED**, `period_filter.options` , register row 3 · ⚠ the two accounting windows are **absent from the menu** on a tenant with no fiscal calendar rather than present and broken, and the choice rides in the URL as `?f.period=` so a window is shareable

3 **Recent activity** · at most six rows from `audit_log` , last 24 hours, newest first · **SHIPS** · **the panel states its own window** (BR-REP-031) because it is the one region the period dropdown does not scope · every row is a sentence, no ids (B14 rule 31)

4 **Service mix** · one row per service the tenant has enabled, **all of them**, in the seed order · **SHIPS** · ⚠ **eleven** services are possible and three are drawn because three are enabled — no fixed height, no cap · the last row is the **named remainder** for revenue no service row can hold (BR-REP-025/026), which reads $0.00 here and is drawn anyway, because a remainder that only appears when it is non-zero is a row nobody knows to look for

5 **AR ageing** · the five signed bands · **SHIPS** · shares are rounded to whole percent and **are not forced to 100**, which is why they read 58+25+9+4+4

5b **A tile with no number** · *Failed payments* reads **Not counted yet** with the reason under it · **SHIPS** · BR-REP-033/024 — ⚠ **this is the pattern that fixes journey 6**: the AR-total tile on a multi-currency tenant should read exactly like this instead of rendering empty, and the sentence already exists in `queries.py`

6 **To do** · at most four, each from the module that owns the work · **SHIPS** · not a queue, a pointer — nothing is actioned here

ADDRESS	`/{tenant}/dashboard` — the shell's landing route
CLASS	`KEEP` the only one in the register · **S7 · DSH-01**
DOORS	**Six, all read from the source, and it is the most-linked screen in the product.** `SHELL_NAV` item 1 · the tenant-root redirect `App.tsx:415` · post-sign-in `SignInPage.tsx:167` · after a password change `ChangePasswordPage.tsx:221` · the *Back to Dashboard* button on every `PermissionDenied` frame across nine modules · global search's unknown-entity fallback `searchDestinations.ts:146` . ⚠ **No task in this plan adds or removes one of them**
REPO	`07-frontend/src/features/dashboard/DashboardPage.tsx` · served by `apps/reports/queries.py`
DATASETS	All nine, read shallowly. This is the one screen that crosses every dataset and asks each for a single number
TABLE CONTROLS	None. No panel here is a table in the `DataTable` sense — no search, no paging, no column picker, by design
WRITE SURFACES	**Detail** every panel links out · **Add / Edit / Update NOTHING**
TODAY `TODAY`	**Ships and works, including the part I first reported as broken.** Six regions, five tiles, the three-option window, the named remainder, the *Not counted yet* tile, server-decided page actions, and every link resolves
TASK `TASK`	**P0.15 — amend A1, assert the vocabulary is served**. **No code on this screen changes.** It stays first in the plan because a stale spec line that would re-add a declined feature is the most expensive thing in this document
WALKED	**YES.** Signed in as Dana Whitfield on a seeded tenant, landed here, read every tile, followed all four panel links. The window dropdown offered *Last 30 days* · *This period* · *Last period*, defaulting to the first, with the dates printed above the figures
RULES	Obeys B14 5 · 12 · 31 · 32 · 33 · 40 · 43, and `BR-REP-024` through `036` . ⚠ Rule 43 is the one at risk: the four panels are inline in the page, not components, so a fifth cannot be added without editing this file

The one thing wrong with this screen is not on this screen — it is in the document that describes it.

I had this backwards in the first draft and it is worth saying exactly how. I read A1's screen inventory, which says the dashboard offers **four windows** — last completed cycle, last 30 days, this accounting period, last accounting period — **defaulting to the last completed cycle**, and cites `BR-REP-035` and `CR-015`. The running product offers **three** and defaults to *Last 30 days*. I wrote that up as a confirmed ruling the code never received, and made it the first task in the plan.

It is the other way round. `CR-029` D8① withdrew the completed-cycle window on 23 August — the lane raised the definition of *completed* as `OQ-268` and **the PO's answer was *"dont build it"***. The option is gone, the default is the rolling window that `CR-015` 's own ruling already named as the alternative, and `dashboard_period.py` says so in three places. The test that used to assert the cycle default is **kept and inverted**, with its original failure preserved, so a lane reintroducing the window goes red and is sent back to the ruling. **The code is right. A1 is stale, by a month.**

What it cost, and why the correction is in the document rather than quietly fixed. Acting on the first reading would have *re-introduced a feature the PO declined* — and it would have passed review, because the citation was real, the rule id was real, and the document I quoted is signed spec. ⚠ **This is CLAUDE.md's own standing warning arriving in practice:** where A1's screen description contradicts a later ruling, **A1 is stale — file it, do not follow it**. The only thing that caught it was reading `dashboard_period.py` 's docstring instead of trusting the spec line that named it.

So this becomes `P0.15`, and it changes what it does. It amends A1's line, and it adds the one assertion that keeps the door from rotting the other way: **the control is drawn from `period_filter.options` and never from a constant in the front end**. The backend already serves the whole vocabulary for exactly this reason — its docstring says *"the DEFAULT is the server's answer rather than a constant two codebases have to agree on"* — and nothing asserts that the screen honours it.

④ What the window dropdown changes, and where

Writes:

nothing. Not a preference, not a cookie, not a row. Reload and it returns to the default.

The ledger:

nothing.

Elsewhere:

the five tiles, the service mix and the ageing panel re-read;

recent activity and to-do do not

, because neither is scoped by the window — recent activity is always the last 24 hours and says so, and to-do is always now. The window

does

ride in the URL as `?f.period=`, so the one durable effect of changing it is that the link you copy afterwards opens on the same window (BR-SHL-011), and a retired or misspelled value

falls back rather than refusing

— a landing page that errors on a stale bookmark is a locked front door. ⚠

That is not drawn anywhere on the screen and it should not be

— rule 33 puts rationale at zero words. It belongs in the docstring, and

P0.15

puts it there.

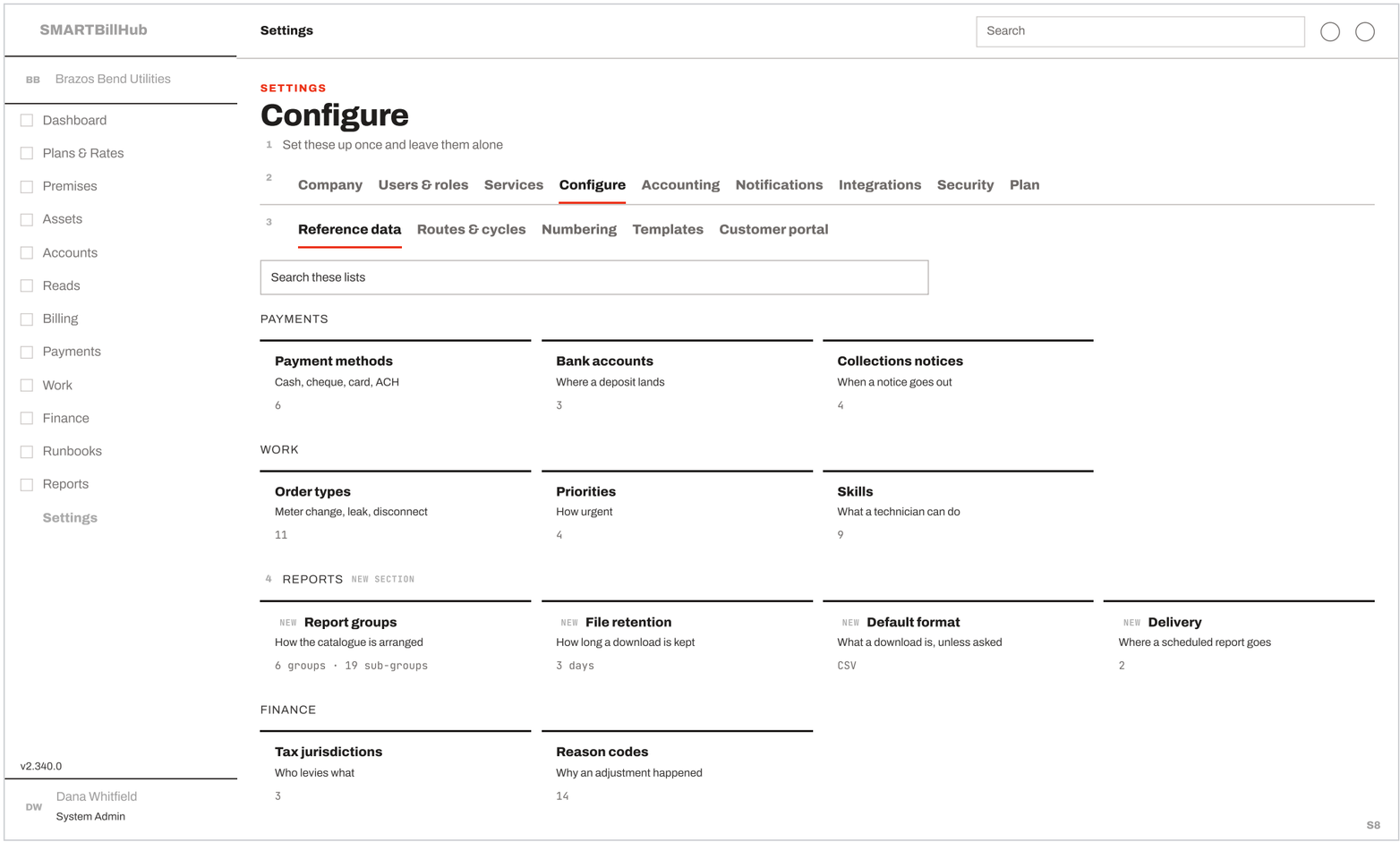
Elsewhere 2 · The Settings section

① THE LIST	**E·2 Reports** — **four cards** in a new section of a grid that already holds thirty-five. It is not a screen of this module; it is four rows in somebody else's screen
② ONE ROW OPENED	The existing `SettingsReferenceData` row drawer, **unchanged**. A Reports list edits exactly the way a list of meter types edits, because it *is* one
③ WHAT YOU CAN	Add a row · edit a row · deactivate a row. All three already work, for all thirty-five cards, and the new four inherit them

DO	
④ **WHAT EACH CHANGES**	One reference row and an `audit_log` row. ⚠ **One of the four changes what every historical run says** — the consequence block below is the only interesting thing on this tab

E·2 · SETTINGS → CONFIGURE → REFERENCE DATA → REPORTS `TASK P5.5`

END STATE · DESKTOP 1440 · SCROLLED TO THE NEW SECTION · **NO NEW ROUTE, NO NEW TAB, NO NEW SCREEN FILE**



1 **Page head** · B14 rule 40, 32px · **SHIPS** · this is `SettingsPage` 's head and nothing about it changes

2 **Settings' nine tabs** · **SHIPS** · ⚠ **Reports does not become a tenth.** Nine is already the number the portal blueprint's §15 records an argument about, a tenth was refused there, and the reasons hold here unchanged

3 **Configure's five sub-tabs** · **SHIPS** · Reference data is first and is the default, so the address is `reference-data/reference-data` — the segment repeats because the tab and its default sub-tab share an id, which `CR-001` kept deliberately so bookmarks survived a rename

4 **The Reports section** · **NEW** · one string appended to `REFERENCE_GROUP_ORDER` (ten entries today) and **four rows** added to `referenceDataConfig.ts` (35 cards today, 39 after). The grid, the search, the counts, the drawer, the permission check and the audit write are all already there

ADDRESS	`/{tenant}/settings/reference-data/reference-data` — ⚠ **the doubled segment is correct**, see legend 3
CLASS	`NEW` as a section of **S8**. **Not a new screen** — no route, no page file, no test file of its own beyond a row in the config test

DOORS	**One today, and a second one this plan adds.** The **Configure → Reference data** grid, where all thirty-five other cards live. ⚠ **And that is not enough on its own** — a retention setting nobody can find from the screen it governs is a setting nobody uses. **P5.12** puts a link on **5·1 History**'s retention line, **for an admin only**, so the fact and the control are one click apart. The line itself stays six words (rule 33); a link is not rationale
REPO	`07-frontend/src/features/tenancy/referenceDataConfig.ts` · rendered by `SettingsReferenceData.tsx` · served by the existing lookup endpoints
DATASETS	None. These are lookup rows, not report data
TABLE CONTROLS	The grid's own search. Each card's drawer has the standard list controls. Nothing new
WRITE SURFACES	**Detail** the existing row drawer · **Add** a lookup row · **Edit** a lookup row · **Update** deactivate. All four inherited
TODAY `TODAY`	**The section does not exist, and retention is a constant.** `ARTIFACT_RETENTION_DAYS = 3` at `exports.py:123` — every tenant gets three days and no screen says so
TASK `TASK`	**P5.5** builds the section. ⚠ `P4.6` **lands retention and the default format one phase earlier**, because Tab 3's expiry helper needs them and schedules cannot be built without a format — so two of the four cards arrive in Phase 4 and the section is completed in Phase 5
WALKED	**YES, for the grid.** Opened Configure → Reference data on a seeded tenant, counted 10 sections and 35 cards, opened *Payment methods*, added a row, watched the count go to 7 and an audit entry appear. **The Reports section itself could not be walked — it does not exist.**
RULES	Obeys B14 5 · 12 · 21 · 31 · 32 · 33 · 40 · 43 **by inheritance** — which is the argument for building it here rather than anywhere else. A new Settings screen would have had to earn all eight again

Two cards were drawn here in the first pass and both are now dropped — one of them because it breaks a signed NFR.

Export limit. I had a card letting a tenant set the biggest file they allow, replacing `EXPORT_ROW_CAP = 100_000` at `exports.py:120`. **It is wrong.** `NFR-5` says no hard caps anywhere, and `P2.4` *deletes* that constant — which is a one-line change rather than a rewrite, because the writer at `exports.py:546` **already streams row by row from a generator**. Putting the number on a Settings screen would re-introduce the cap with a nicer interface and make a tenant responsible for a limit the architecture does not need. **The cap goes away; it does not move.**

Dashboard window. I had a card letting a tenant choose which window the dashboard opens on. The default is **ruled** — the rolling thirty days, after `CR-029` D8① withdrew the cycle option — and the server decides it from the options it could actually build. **A tenant row overriding a PO-ruled default is a ruling question, not a settings row.** If a utility asks for it, that is a change request with a named requester; inventing the control here would have quietly reversed a decision on our own authority.

What is left is four cards, and two of them are single values rather than lists — which the grid already supports: *Billing defaults*, *Read validation* and *Budget billing* each open onto a small form, not a list. The pattern exists, it is used three times, and the four new cards sit inside it without inventing anything.

④ What File retention changes, and where

Writes:

one lookup row · one `audit_log` row.

The ledger:

nothing.

Elsewhere — and this is why it is the one called out:

it changes what the expiry helper on

5·1

and

5·2

says

on every historical run, immediately

, because retention is evaluated against the run's timestamp and not stored on it. ⚠

So lowering it from 3 days to 1 makes yesterday's file unavailable the moment the row is saved

— no job re-runs, nothing is deleted by this write, and a clerk who had a working download link an hour ago now has an expired one.

Not drawn on the screen, and it should not be

— rule 33 puts rationale at zero words. The form states the fact in six words (*"Files are kept for 3 days"*); the reasoning lives in the docstring and in

P3.5

's test, which is the retention test and is the place this behaviour gets pinned.

The 46, drawn

These are not 46 screens. Every one of them is **2·1** rendering a different definition row, so the rail, the topbar, the page head and the toolbar are identical on all of them and drawing that chrome 46 times would be noise. What changes per report is what a definition row *holds*: its parameters, its KPI tiles, its columns and its totals. **That is what is drawn below**, at the shipped table geometry — 13.5px body, 12px uppercase headers, square corners.

46 answers, 45 cards, and the arithmetic is worth a line.

R42 is not a card — it became **Tab 2**, the Reports dashboard, by PO ruling, because a dashboard has no parameters, no rows and no file. And **R24 and R25 are drawn four times each**, because a single *Subject* parameter changes whether the report is about an account, a customer, a building or a unit. So: **45 cards in the catalogue · 1 tab · 51 drawings**.

Every figure below is an illustration.

None is a served value and none is signed. They are consistent across the gallery on purpose — the same customers, the same account numbers, the same money — so you can follow one utility through all 46. Money is shown at two decimals per hard rule 2; quantities always carry their unit, and **are never summed across services**, because gallons plus kilowatt-hours is not a number.

BILLING — 5

R4 Bill Run Readiness

Billing › Readiness · bills + reads · Posting date

Cycle CY-09B ⌄ **Route** All routes ⌄ **Check** All checks ⌄

ACCOUNTS IN CYCLE	READY TO BILL	HELD	NO READ
1,247	1,198 96%	49	31

ACCOUNT	CUSTOMER	ROUTE	CHECK	DETAIL	USAGE

100244	Ruiz, Marisol	R-01	**High use**	3.8× last month	14,900 gal
100907	Okafor, D.	R-03	**No read**	Last read 12 Aug	—
100512	Vance Holdings	R-02	**Stopped meter**	Zero for 3 cycles	0 gal

Run before the bill run, not after. The 49 held accounts are the whole point of the screen — they are what a clerk fixes in the twenty minutes before issuing 1,198 bills.

R5 Billing Register

Billing › Bills · bills & lines · Posting date

Cycle CY-09B ⌄ **Bill type** All ⌄ **Service** All ⌄

BILLS	AVERAGE BILL	ESTIMATED READS	TOTAL BILLED
1,198	**$104.22**	**31** 3%	**$124,855.56**

BILL #	ACCOUNT	CUSTOMER	TYPE	SERVICES	USAGE	AMOUNT	FLAG
BL-2609-018204	100244	Ruiz, Marisol	Cycle bill	Water · Sewer	4,100 gal	247.87	
BL-2609-018205	100381	Alvarez, R.	Cycle bill	Water	1,900 gal	41.20	**Estimated**
BL-2609-018206	100907	Okafor, D.	Final bill	Water · Sewer	2,240 gal	63.25	
Total						**124,855.56**	

⚠ The one report where **cycle** must be a parameter, not a date range. Today it runs on a date range only.

R6 Bill Adjustments

Billing › Bills · bills & lines · Posting date

From 09/01/2026 **To** 09/30/2026 **Reason** All reasons ⌄

ADJUSTMENTS	VALUE	LARGEST
23	**−$1,884.10**	**−$612.00**

DATE	BILL #	ACCOUNT	REASON	BY	APPROVED BY	AMOUNT
08/09/2026	BL-2609-018204	100244	Misread corrected	M. Okonkwo	D. Whitfield	−184.10
14/09/2026	BL-2609-017990	100512	Leak adjustment	D. Whitfield	D. Whitfield	−612.00
					Total	**−1,884.10**

⚠ **An adjustment is a new row, never an edit.** Issued invoices take no UPDATE and no DELETE — so this report lists corrections, and there is no edit control on any row of it.

R7 Bill Print File

Billing › Delivery · bills & lines · Posting date

Cycle CY-09B ⌄ **Format** Print vendor ⌄

BILLS IN FILE	PAGES	SUPPRESSED
1,061	2,418	137 eBill

BATCH	CYCLE	BILLS	PAGES	BUILT	STATUS	FILE
PRN-2609-01	CY-09B	1,061	2,418	25 Sep 06:02	Ready	↓ CSV

The 137 suppressed bills are the eBill enrolments. They are stated rather than silently missing from the count.

R8 Bill Delivery

Billing › Delivery · notifications · —

From 09/01/2026 **To** 09/30/2026 **Channel** All ⌄

SENT	DELIVERED	BOUNCED	RETURNED POST
1,198	1,144 95%	38	16

ACCOUNT	CUSTOMER	CHANNEL	ADDRESS	SENT	STATUS	REASON
100244	Ruiz, Marisol	Email	m.ruiz@…	25 Sep	Delivered	
100907	Okafor, D.	Email	d.okafor@…	25 Sep	Bounced	Mailbox full
100512	Vance Holdings	Post	91 Live Oak Dr	25 Sep	Returned	No such address

⚠ The address column walks to `premises` — an address is stored in one place and every other entity derives it. Nothing here holds its own copy.

RECEIVABLES — 9

R1 Payments

Receivables › Payments · payments · Clearing date

From 09/25/2026 **To** 09/25/2026 **Group by** Method ⌄ **Teller** All ⌄

TAKEN TODAY	PAYMENTS	CASH	CARDS
$4,118.55	**61**	**$612.00**	**$2,904.10**

METHOD	TELLER	COUNT	AMOUNT	SHARE
Card	—	41	2,904.10	71%
Cheque	Counter 1	12	602.45	15%
Cash	Counter 1	8	612.00	15%
Total		**61**	**4,118.55**	

Group by switches this between method, teller, purpose and day. **Four reports, one row.**

R2 Deposit Batches

Receivables › Payments · payments · Clearing date

From 09/01/2026 **To** 09/30/2026 **Status** All

BATCHES	BANKED	OPEN
22	**$88,412.90**	**$4,118.55** 1 batch

BATCH	DAY	TENDER	ITEMS	AMOUNT	STATUS
DEP-2609-022	25 Sep	Cash & cheque	20	1,214.45	Open
DEP-2609-021	24 Sep	Cash & cheque	34	2,904.10	Banked
			54	**4,118.55**	

⚠ A batch groups by **day and tender**. That is why a payment's tender can no longer be corrected once it is banked.

R3 Payment Exceptions

Receivables › Payments · payments · Clearing date

From 09/01/2026 **To** 09/30/2026 **Kind** All

EXCEPTIONS	VALUE	UNRESOLVED
14	**$2,488.10**	**3**

DATE	KIND	PAYMENT	ACCOUNT	BY	REASON	AMOUNT
08/09/2026	Returned (NSF)	PMT-2609-000418	100244	—	Insufficient funds	169.52
09/09/2026	Reference fixed	PMT-2609-000418	100244	D. Whitfield	Cheque 4417 → 4471	—

12/09/2026	Transferred	PMT-2609-000502	100381 → 100907	M. Okonkwo	Posted to wrong account	84.00

⚠ **Row two is the only kind of change a payment permits.** Three columns may be corrected in place — the cheque number, the memo and the tender — through one named endpoint. Everything else about a payment is frozen.

R9 AR Ageing

Receivables › Arrears · payments · Posting date

As of 09/30/2026 **Consumer class** All

TOTAL OWED	CURRENT	31–60	61–90	90+	ACCOUNTS
$48,221.14	**$29,104.20** 60%	**$11,880.06** 25%	**$4,902.44** 10%	**$2,334.44** 5%	**614**

ACCOUNT	CUSTOMER	CLASS	CURRENT	31–60	61–90	90+	TOTAL
100244	Ruiz, Marisol	Residential	—	—	108.88	—	108.88
100512	Vance Holdings	Small commercial	—	—	—	1,204.00	1,204.00
100907	Okafor, D.	Residential	41.20	2.24	—	—	43.44
Total			**29,104.20**	**11,880.06**	**4,902.44**	**2,334.44**	**48,221.14**

⚠ **An account on a payment arrangement counts in its true bucket, not Current** — the bucket comes from the oldest unpaid bill's due date, never from the plan's status. Shares are rounded and need not total 100%.

R10 Debtor List

Receivables › Arrears · payments · Posting date

Min balance $50.00 **Days overdue ≥** 30 **Class** All **Service** All

DEBTORS	OWED	OLDEST
288	**$19,116.94**	**148 days**

ACCOUNT	CUSTOMER	SERVICE ADDRESS	OLDEST BILL	DAYS	OWED	ARRANGEMENT
100512	Vance Holdings	18 Brazos Ct	05/05/2026	148	1,204.00	
100244	Ruiz, Marisol	1420 Pecan St	05/08/2026	84	108.88	Plan
					19,116.94	

⚠ **This list and the cut-off list are the same query with one flag.** Today it is unpaginated on purpose — correct at 3,000 connections, 13× short at 40,000.

R11 Cut-Off List

Receivables › Arrears · payments · Posting date

Account type: All | Threshold: $75.00 | Eligibility: Eligible

ELIGIBLE	HELD	VALUE AT RISK
94	22	$11,402.88

ACCOUNT	CUSTOMER	SERVICE ADDRESS	OWED	DAYS	ELIGIBLE	HOLD REASON
100512	Vance Holdings	18 Brazos Ct	1,204.00	148	Yes	
100244	Ruiz, Marisol	1420 Pecan St	108.88	84	Held	Payment arrangement
100733	Alvarez, R.	91 Live Oak Dr	212.40	61	Held	Medical hold

⚠ **The hold reason is served, not inferred.** A clerk removing an account from this list is making a decision that must be recorded, and the eligibility flag is what makes that possible.

R12 Collections Activity

Receivables › Recovery · payments · —

From: 09/01/2026 | To: 09/30/2026 | Kind: All | Stage: All

NOTICES SENT	ARRANGEMENTS LIVE	KEPT	BROKEN
412	64	81%	12

DATE	ACCOUNT	KIND	STAGE	CHANNEL	OUTCOME	AMOUNT
07/09/2026	100512	Dunning notice	Reminder 2	Email	Delivered	1,204.00
12/09/2026	100244	Arrangement	—	Counter	Kept	108.88
19/09/2026	100733	Arrangement	—	Portal	Broken	212.40

⚠ **Dunning stage names come from the tenant's own cadence, not from a hardcoded 7/14/30.** Those day numbers are seeded defaults a utility edits.

R13 Write-Offs & Late Fees

Receivables › Recovery · bills & lines · Posting date

From: 07/01/2026 | To: 09/30/2026 | Kind: All

WRITTEN OFF	LATE FEES CHARGED	LATE FEES BLOCKED
$3,884.20	$6,112.00	$488.00
		19 accounts

DATE	ACCOUNT	KIND	REASON	APPROVED BY	AMOUNT
31/07/2026	100455	Write-off	Uncollectable — closed 2 yrs	D. Whitfield	1,880.20
01/09/2026	100512	Late fee	10% on overdue balance	—	120.40
01/09/2026	100733	Fee blocked	Medical hold	D. Whitfield	−48.80
				Net	**9,507.40**

⚠ **Write-offs are excluded from collection rate and shown separately** — folding them in would flatter the figure.

R14 Closed Account Balances

Receivables › Recovery · payments · Posting date

Final billed from 07/01/2026 **To** 09/30/2026 **Outcome** All

FINAL BILLED	STILL OWING	IN CREDIT	SENT TO AGENCY
188	**$8,204.55**	**$1,102.80**	**$4,118.00**
	61 accounts	24 accounts	

ACCOUNT	CUSTOMER	FINAL BILL	FORWARDING ADDRESS	OUTCOME	BALANCE
100455	Nwosu, A.	BL-2607-016880	—	**Owing**	412.55
100488	Baptiste, L.	BL-2608-017204	14 Cypress Way	**In credit**	−88.40
100512	Vance Holdings	BL-2609-018206	—	**At agency**	1,204.00

⚠ **Three of Augusta's named mandatory reports are this one row with the outcome filter changed**: final billed still owing, final-bill credits, and the collection-agency hand-off file.

FINANCE — 9

R15 Trial Balance

Finance › Ledger · journal lines · Posting date

Period FY2026 P12 **Account range** All **Level** Detail

DEBITS	CREDITS	DIFFERENCE
$1,488,204.10	**$1,488,204.10**	**$0.00**
		Balanced

ACCOUNT	NAME	OPENING	DEBITS	CREDITS	CLOSING
1200	Accounts receivable	44,118.02	124,855.56	120,752.44	48,221.14

4000	Water revenue	—	—	88,412.90	−88,412.90
2300	Customer deposits	18,400.00	1,150.00	2,200.00	19,450.00
	Total	**62,518.02**	**1,488,204.10**	**1,488,204.10**	**—**

⚠ **Balanced pairs are a database guarantee, not a report's opinion.** A difference other than zero here means the posting service has a defect, not that the report needs tuning.

R16 Journal Export

Finance › Ledger · journal lines · Posting date

Period FY2026 P12 ⌄ **Target** OpenGov ⌄ **Batch** New ⌄

ENTRIES	LINES	VALUE	LAST EXPORT
1,884	**4,102**	$1,488,204.10	**24 Sep 18:02**

BATCH	PERIOD	ENTRIES	LINES	VALUE	EXPORTED	FILE
GL-2609-04	P12	1,884	4,102	1,488,204.10	25 Sep 06:02	↓ CSV
GL-2609-03	P12	1,640	3,588	1,244,118.00	18 Sep 06:02	**Expired**

⚠ **Idempotent by batch.** Re-exporting a batch produces the same file; it does not double-post. The ERP is the system of record for the general ledger and this is the only way out.

R17 Revenue Analysis

Finance › Income · journal lines · Posting date

From 09/01/2026 **To** 09/30/2026 **Service** All ⌄ **Basis** Billed ⌄

TOTAL REVENUE	USAGE	BASE & FEES	TAX
$124,855.56	**$88,412.90** 71%	**$29,330.66** 23%	**$7,112.00** 6%

SERVICE	RATE SCHEDULE	ACCOUNTS	USAGE	USAGE REV	BASE	TAX	TOTAL
Water	RES-W-2026	984	3,884,100 gal	61,204.10	18,402.00	4,880.00	84,486.10
Sewer	RES-S-2026	902	—	21,208.80	8,428.66	1,822.00	31,459.46
Water	COM-W-2026	61	1,102,400 gal	6,000.00	2,500.00	410.00	8,910.00
Total		**1,947**	**—**	**88,412.90**	**29,330.66**	**7,112.00**	**124,855.56**

⚠ **The usage column shows an em-dash for sewer and a blank total, on purpose.** Gallons and kilowatt-hours cannot be added, so a cross-service usage total is refused rather than computed. Money adds; quantities do not.

R18 Tax Remittance

Finance › Income · journal lines · Posting date

Period FY2026 P12 ⌄ **Authority** All ⌄

COLLECTED	REMITTED	DUE
$7,112.00	**$6,204.00**	**$908.00** 30 Oct

AUTHORITY	TAX	BASIS	TAXABLE	RATE	COLLECTED	REMITTED	DUE
State of Texas	Sales tax	Posting	88,412.90	6.250%	5,525.80	5,525.80	—
Brazos County	Local	Posting	88,412.90	1.000%	884.13	678.20	205.93
City	Franchise	Clearing	70,204.00	1.000%	702.07	—	702.07
				Total	**7,112.00**	**6,204.00**	**908.00**

⚠ **Two bases in one report, labelled per row.** Franchise fees read the clearing date and the others read posting — a dual-basis report is correct and unlabelled money is a contract failure.

R19 Cash Reconciliation

Finance › Cash · payments · Clearing date

Date 09/25/2026 **Source** Bank statement ⌄

PER THE SYSTEM	PER THE BANK	DIFFERENCE	UNMATCHED
$4,118.55	**$4,118.55**	**$0.00**	**2** $0.00

DATE	REFERENCE	DESCRIPTION	SYSTEM	BANK	STATUS
25/09/2026	DEP-2609-022	Counter deposit	1,214.45	1,214.45	Matched
25/09/2026	CARD-2609-25	Card settlement	2,904.10	2,904.10	Matched
25/09/2026	—	Bank charge	—	−18.40	Unmatched
		Total	**4,118.55**	**4,100.15**	

⚠ **The difference tile reads $0.00 while two rows are unmatched**, and that is correct: the bank charge is not a payment. An unmatched row is a thing to explain, not a thing that breaks the reconciliation.

R20 Deposits

Finance › Cash · payments · Clearing date

Mode Held as at ⌄ | **As of** 09/30/2026 | **Class** All ⌄

HELD	TAKEN THIS YEAR	REFUNDED	APPLIED
$19,450.00	**$8,200.00**	**$4,400.00**	**$2,750.00**
184 accounts			

ACCOUNT	CUSTOMER	CLASS	TAKEN	AMOUNT	APPLIED	HELD
100244	Ruiz, Marisol	Residential	12/03/2026	150.00	—	150.00
100512	Vance Holdings	Small commercial	04/01/2026	400.00	—	400.00
100733	Alvarez, R.	Residential	18/06/2026	150.00	150.00	—
			Total	**19,450.00**	**2,750.00**	**19,450.00**

⚠ **Two of Augusta's mandatory reports are this one row with the mode switched** — deposits held at year end, and deposits taken in a date range.

R21 Close Pack

Finance › Ledger · journal lines · Posting date

Period FY2026 P12 ⌄ | **View** Tie-out ⌄

AR CONTROL	AGEING TOTAL	DIFFERENCE	ADJUSTMENTS
$48,221.14	**$48,221.14**	**$0.00** Foots	**23** –$1,884.10

CHECK	SOURCE A	VALUE A	SOURCE B	VALUE B	DIFFERENCE	RESULT
AR tie-out	Ledger control	48,221.14	Ageing report	48,221.14	0.00	Foots
Cash tie-out	Payments	120,752.44	Bank	120,752.44	0.00	Foots
Revenue tie-out	Billing register	124,855.56	Ledger revenue	124,855.56	0.00	Foots

⚠ **Four of Augusta's named reports collapse into this one view switch** — tie-out, pull totals, interdepartmental charges and the calculation audit. A drift here is a release blocker, not a report finding.

R42 Operational Dashboard

Finance › Performance · **not a card — it is Tab 2**

⚠ **This one is not in the catalogue and is not drawn here.** It has no parameters worth exporting, no rows and no file — it is a screen. PO ruling: it becomes **a tab of its own** at `/reports/dashboard` , drawn as **4·1**, and **the existing shell dashboard is not replaced or touched**. A card pointing at a sibling tab is noise, so there is no card. **That is why the catalogue holds 45 and the answers number 46.**

R43 Trend Pack

Finance › Performance · journal lines + accounts · Posting date

Periods: 12 ▾ | Measure: All ▾

REVENUE VS TARGET	COLLECTION RATE	DSO	ACCOUNTS
104%	**94.2%**	**14.1 days**	**+1.8%**
$124.9k of $120k		target 15	12 months

PERIOD	REVENUE	TARGET	COLLECTED	RATE	DSO	ACCOUNTS	REBILLS
P10 · Jul	118,402.10	120,000.00	111,208.44	93.9%	14.8	1,921	18
P11 · Aug	121,044.88	120,000.00	114,880.02	94.9%	14.4	1,934	22
P12 · Sep	124,855.56	120,000.00	117,604.10	94.2%	14.1	1,947	23

⚠ **The target is a typed per-tenant figure and is empty on every tenant that exists today**, so the tile degrades to the figure alone. Day-prorated comparisons are forbidden — billing is lumpy by cycle, and straight-lining misreads it by ten points.

CUSTOMERS — 9

R22 Account Census

Customers › Accounts · accounts · As at

As of: 09/25/2026 | Status: All ▾ | Class: All ▾ | Service: All ▾

ACCOUNTS	ACTIVE	FINAL	WITH BALANCE
1,947	**1,884**	**41**	**412**
			$48,221.14

ACCOUNT	CUSTOMER	PREMISES	CLASS	SERVICES	STATUS	OPENED	BALANCE
100244	Ruiz, Marisol	118 Cedar St	Residential	Water · Sewer	Active	03/12/2024	184.52
100512	Vance Holdings	4400 Industrial Pkwy	Small commercial	Water · Sewer · Trash	Active	01/04/2023	2,418.90
100907	Okafor, D.	9 Larkspur Ct, Unit B	Residential	Water	Final	22/08/2025	−41.10
						Total	**48,221.14**

⚠ **The address column walks to premises and declares nothing of its own** — hard rule 8. Unit B refines the premises by label, so it shares the street address and may carry its own point on a map.

R23 Account Activity

Customers › Accounts · accounts + audit · Event date

From | To | Event

09/01/2026 | 09/30/2026 | All

OPENED	CLOSED	TRANSFERRED	NET
38	14	6	+24

DATE	ACCOUNT	CUSTOMER	EVENT	PREMISES	BY	EFFECTIVE
04/09/2026	100488	Baptiste, L.	Opened	77 Willow Bend	Dana Whitfield	04/09/2026
11/09/2026	100907	Okafor, D.	Closed	9 Larkspur Ct, Unit B	Dana Whitfield	09/09/2026
18/09/2026	100455	Nwosu, A.	Transferred in	118 Cedar St	M. Okonkwo	01/10/2026

⚠ **Two date columns because two dates exist** — when the clerk did it, and when it takes effect. A move-in dated 1 October appears in September's activity and in October's census, and both are right.

R24 · R25 Statement · Ledger

Customers › Statements · drawn at four subjects each, below

⚠ **These two are drawn separately, after the six groups**, because one parameter changes who the report is about and four drawings each is the only honest way to show it. Jump to them →

R26 Data Quality

Customers › Accounts · accounts + premises + assets · As at

As of 09/25/2026 | **Severity** All

ACCOUNTS CHECKED	FINDINGS	BLOCKING	CLEAN
1,947	84	11	95.7%

FINDING	SEVERITY	COUNT	EXAMPLE	EFFECT	FIX IN
No mailing address and no premises	Blocking	4	100455	Bill cannot be delivered	Accounts
Active account, no service point	Blocking	7	100733	Will not bill	Premises & Units
Premises with no coordinates	Advisory	61	118 Cedar St	No map pin	Premises & Units
Meter with no read in 90 days	Advisory	12	MTR-004182	Estimated bill	Meter Reads
	Total	**84**			

⚠ **Every row names the screen that fixes it.** A quality report that lists problems without an address is a list of complaints — the *Fix in* column is the deliverable, and **P1.4** makes each one a link.

R27 Plans & Discounts

Customers › Enrolment · accounts + rating · As at

As of | **Programme** | **Status**

09/25/2026 | All ⌄ | Active ⌄

ENROLLED	LEVEL PAY	PAYMENT PLANS	ASSISTANCE
318	**184**	**84**	**50**
16.3%		$14,204.00 remaining	

PROGRAMME	ACCOUNT	CUSTOMER	SINCE	TERMS	INSTALMENT	REMAINING	STATUS
Level pay	100244	Ruiz, Marisol	01/01/2026	12 months	168.00	—	Active
Payment plan	100512	Vance Holdings	19/08/2026	6 instalments	403.15	1,612.60	On track
Assistance	100381	Alvarez, R.	01/07/2026	Low-income · 20%	—	—	Review due
				Total	**571.15**	**14,204.00**	

⚠ **Three programmes in one table because a clerk asks one question** — what is this account enrolled in. Splitting them into three reports was the first draft and it made the clerk run three.

R44 My Bills & Payments

Customers › Statements · invoices + payments · Statement date

Account 100244 (signed in) | **Period** Last 12 months ⌄

BALANCE	LAST PAYMENT	AUTOPAY
$184.52	**$212.40**	**Off**
Due 02 Oct	24 Jul	

DATE	TYPE	DESCRIPTION	AMOUNT	STATUS	
12/09/2026	Bill	12 Aug – 11 Sep	184.52	Due	↓ PDF
24/07/2026	Payment	Card ending 4118	–212.40	Cleared	View
12/07/2026	Bill	12 Jun – 11 Jul	178.40	Paid	↓ PDF

⚠ **This one renders in the customer portal, not the staff shell** — no left rail, no tenant switcher, and the eight accessibility questions of the maintenance process apply to it. It is the same definition row and the same dataset; only the frame differs.

R45 My Usage

Customers › Enrolment · meter reads · Read date

Account 100244 (signed in) | **Service** Water ⌄ | **Compare** Same month last year ⌄

THIS PERIOD	SAME PERIOD LAST YEAR	DAILY AVERAGE	NEIGHBOURHOOD AVERAGE
4,120 gal	**3,880 gal**	**137 gal**	**152 gal**

+6.2% −10%

PERIOD	READ DATE	READING	USAGE	UNIT	DAYS	PER DAY	BASIS
12 Aug – 11 Sep	11/09/2026	184,220	4,120	gal	30	137	Actual
12 Jul – 11 Aug	11/08/2026	180,100	3,980	gal	30	133	Actual
12 Jun – 11 Jul	11/07/2026	176,120	4,010	gal	29	138	Estimated

⚠ **Every quantity carries its unit, in its own column** — hard rule 3, and the reason there is no *Total usage* tile on a multi-service account. The *Basis* column is why a customer's own figure and their bill can differ without either being wrong.

R46 Bill Projection

Customers › Enrolment · meter reads + rating · Projection date

Account 100244 (signed in)

Through 10/11/2026

PROJECTED BILL	DAYS ELAPSED	USAGE SO FAR	BASIS
$188.00 ±$12	**14 of 30**	**1,960 gal**	**Actual reads to 25 Sep**

COMPONENT	BASIS	QUANTITY	UNIT	RATE	PROJECTED
Water base	Monthly	1	month	18.00	18.00
Water usage	Projected from 14 days	4,200	gal	0.0142	59.64
Sewer base	Monthly	1	month	22.00	22.00
Sewer usage	Winter average	3,880	gal	0.0228	88.46
				Projected total	**188.10**

⚠ **Usage is straight-lined and base charges are not**, and each row says which it is. The projection is a prediction shown to a customer, so the definition seeded by **P0.6** carries the ± range beside the figure: a single number reads as a promise and generates a complaint when the real bill differs.

OPERATIONS — 7

R28 Read Completeness

Operations › Reads · meter reads · Read date

Cycle CY-09B ⌄ **Route** All ⌄ **Status** All ⌄

EXPECTED	READ	ESTIMATED	MISSING
1,947	**1,916** 98.4%	**31**	**0**

ROUTE	READER	EXPECTED	ACTUAL	ESTIMATED	SKIPPED	MISSING	COMPLETE
R-01	M. Okonkwo	684	676	8	0	0	100%
R-02	Dana Whitfield	712	698	14	0	0	100%
R-03	Unassigned	551	542	9	0	0	100%
	Total	**1,947**	**1,916**	**31**	**0**	**0**	**100%**

⚠ **Route R-03 has no reader and still reads 100% complete** — an unassigned route is a scheduling gap, not a read gap, and the report must not conflate them. *Complete* counts actual plus estimated because an estimated read bills; *Missing* is the column that blocks a bill run.

R29 Consumption Exceptions

Operations › Reads · meter reads · Read date

Cycle CY-09B ⌄ **Rule** All ⌄ **Threshold** Tenant default ⌄

READS CHECKED	EXCEPTIONS	HELD	RELEASED
1,916	48 (2.5%)	11	37

ACCOUNT	METER	RULE	PRIOR	THIS READ	CHANGE	UNIT	STATUS
100512	MTR-004182	High usage · >200%	1,102	3,884	+252%	gal	Held
100244	MTR-002044	Zero usage	4,010	0	−100%	gal	Held
100381	MTR-003118	Reverse reading	180,100	179,880	−220	gal	Released
			Total held				**11**

⚠ **Prior and current sit side by side because the exception is a comparison**, not a value. A reverse reading is negative usage and is shown as the raw difference, not as a percentage — dividing by a larger number to produce −0.1% would hide a meter swap.

R30 Usage & Ranking

Operations › Reads · meter reads · Read date

From 09/01/2026 **To** 09/30/2026 **Service** Water ⌄ **Rank** Top 25 ⌄

TOTAL USAGE	ACCOUNTS	AVERAGE	TOP 25 SHARE
3,884,100 gal	984	3,947 gal	18.4%

RANK	ACCOUNT	CUSTOMER	CLASS	USAGE	UNIT	SHARE	BILLED
1	100512	Vance Holdings	Small commercial	148,200	gal	3.8%	2,104.44
2	100733	Cedar Grove HOA	Small commercial	88,400	gal	2.3%	1,255.28

3	100455	Nwosu, A.	Residential	12,880	gal	0.3%	182.90
			Top 25	**714,680**	**gal**	**18.4%**	**10,148.46**

⚠ **One service at a time, and the dropdown is not optional.** A ranking across water and electricity would sort gallons against kilowatt-hours, which is the single most common way a usage report lies. The service parameter is required, not defaulted.

R31 Meter Fleet

Operations › Assets · assets · As at

As of 09/25/2026 | **Type** All | **Status** All | **Age** All

METERS	IN SERVICE	SPARE	DUE FOR CHANGE
1,988	**1,947**	**29**	**84** 4.2%

METER	TYPE	SIZE	PREMISES	INSTALLED	AGE	LAST READ	STATUS
MTR-004182	Water · compound	2"	4400 Industrial Pkwy	01/04/2019	7.5 yr	11/09/2026	Due
MTR-002044	Water · positive disp.	5/8"	118 Cedar St	12/06/2022	4.3 yr	11/09/2026	In service
MTR-005010	Water · positive disp.	5/8"	—	20/08/2026	0.1 yr	—	Spare
					Total		**1,988**

⚠ **A spare meter has no premises and no read, and both cells show an em-dash rather than a blank.** Blank reads as missing data; the dash reads as *not applicable*, which is the fact.

R32 Read Import Log

Operations › Reads · jobs + meter reads · Import date

From 09/01/2026 | **To** 09/30/2026 | **Source** All | **Result** All

IMPORTS	ROWS	ACCEPTED	REJECTED
42	**81,204**	**80,918** 99.6%	**286**

DATE	FILE	SOURCE	ROWS	ACCEPTED	REJECTED	BY	DETAIL
11/09/2026 06:14	cy09b-r01.csv	Handheld	684	684	0	M. Okonkwo	—
11/09/2026 06:22	cy09b-r02.csv	AMI batch	712	698	14	System	View 14 rows
04/09/2026 09:02	backfill-aug.csv	Manual upload	1,204	1,188	16	Dana Whitfield	View 16 rows
		Total	**81,204**	**80,918**	**286**		

⚠ **A rejected row is skipped inside its own savepoint** — hard-won law 14. Without the nested transaction the first rejection aborts the whole import on PostgreSQL and every row is lost at commit, while the test suite on SQLite passes. The 286 rejections are proof the savepoint is real.

R33 Service Order Activity

Operations › Field · work orders · Event date

From 09/01/2026 **To** 09/30/2026 **Type** All **Crew** All

RAISED	COMPLETED	OPEN	AVERAGE TO CLOSE
148	132	16	2.4 days

ORDER	TYPE	PREMISES	RAISED	ASSIGNED	COMPLETED	DAYS	STATUS
WO-2609-0418	Meter change	4400 Industrial Pkwy	02/09/2026	Crew A	05/09/2026	3	Complete
WO-2609-0441	Leak investigation	118 Cedar St	14/09/2026	M. Okonkwo	16/09/2026	2	Complete
WO-2609-0488	Disconnect for non-payment	9 Larkspur Ct, Unit B	22/09/2026	Crew B	—	3	Open
					Total	**2.4**	**148**

⚠ **Work is the eleventh module and owns these tables**; this report reads them through a seam and writes nothing. The premises column walks to premises like every other address in the product — the work order declares no address of its own.

R34 Disconnect & Reconnect

Operations › Field · work orders + accounts · Event date

From 09/01/2026 **To** 09/30/2026 **Reason** All

ELIGIBLE	NOTICES SENT	DISCONNECTED	RECONNECTED
84 $22,104.80	61	12	9 $4,118.00 collected

ACCOUNT	CUSTOMER	BALANCE	NOTICE	DISCONNECTED	PAID	RECONNECTED	STATUS
100907	Okafor, D.	418.90	02/09/2026	11/09/2026	418.90	12/09/2026	Restored
100733	Alvarez, R.	284.10	08/09/2026	—	—	—	Notice sent
100488	Baptiste, L.	612.40	22/09/2026	—	—	—	Eligible
	Total	**22,104.80**	**61**	**12**	**4,118.00**	**9**	

⚠ **The five states are columns, not a status filter**, because a supervisor reads this to answer one question: where is each account in the sequence. Eligibility rules are the tenant's and live in Settings — this report shows who the rule selected, never decides it.

R35 Audit Trail

Compliance › Audit · audit log · Event date

From: 09/01/2026 | To: 09/30/2026 | Actor: All | Object: All | Action: All

EVENTS	ACTORS	MONEY EVENTS	RETENTION
18,402	9	1,884	12 months oldest 26 Sep 2025

WHEN	ACTOR	ACTION	OBJECT	BEFORE	AFTER	WHERE
25/09/2026 09:18	Dana Whitfield	Corrected reference	PMT-2609-000418	4417	4471	Payments
24/09/2026 16:02	M. Okonkwo	Approved adjustment	ADJ-2609-0041	Pending	Approved	Billing
24/09/2026 11:44	System	Issued invoices	Run BR-2609-04	Draft	Issued	Billing

⚠ **The retention tile is the honest part and it belongs here, not on a form.** The first row is the only place cheque 4417 survives, and it is pruned on schedule with every other row — ruled 23 Sep, no row type exempt. The Correct-reference screen says nothing about it; this report shows the oldest date it holds.

R36 Access Review

Compliance › Audit · identity + audit · As at

As of: 09/25/2026 | Role: All | Status: All

USERS	ACTIVE	NEVER SIGNED IN	CAN APPROVE MONEY
9	8	1	3

USER	ROLE	ADDED	ADDED BY	LAST SIGN-IN	APPROVE MONEY	STATUS
Dana Whitfield	Billing clerk	01/03/2024	M. Okonkwo	25/09/2026 08:02	No	Active
M. Okonkwo	Administrator	01/03/2024	Platform	25/09/2026 07:44	Yes	Active
J. Halloran	Read only	18/09/2026	M. Okonkwo	—	No	Never signed in
					Total	**9**

⚠ **Ten staff is the whole ICP, so this is nine rows and not a paginated list** — and it is still served paginated, because `NFR-3` admits no exceptions and the four places that claimed one are the defect `DEF-PAY-4221` exists to close.

R37 Rate Change History

Compliance › Changes · rating + audit · Effective date

From: 01/01/2026 | To: 09/30/2026 | Service: All

CHANGES	IN EFFECT NOW	FUTURE DATED	BILLS AFFECTED
14	6	2	1,884

EFFECTIVE	SCHEDULE	SERVICE	COMPONENT	WAS	NOW	CHANGE	BY
01/10/2026	RES-W-2026	Water	Usage tier 1	0.0142	0.0154	+8.5%	M. Okonkwo
01/07/2026	RES-S-2026	Sewer	Base charge	20.0000	22.0000	+10.0%	M. Okonkwo
01/01/2026	COM-W-2026	Water	Usage flat	0.0138	0.0142	+2.9%	Platform

⚠ **Rates show four decimals and percentages six, displayed at the precision the rule stores** — hard rule 2, and the one report where trimming to two decimals would misstate a tariff. The future-dated row is why the effective date, not the entry date, is this report's basis.

R38 Adjustment Register

Compliance › Changes · invoices + journal lines · Posting date

From: 09/01/2026 | To: 09/30/2026 | Reason: All | Approver: All

ADJUSTMENTS	CREDITS	DEBITS	OVER THRESHOLD
23 −$1,884.10	19 −$2,104.10	4 +$220.00	2 approved

DATE	REFERENCE	ACCOUNT	REASON	AMOUNT	RAISED BY	APPROVED BY	INVOICE
19/09/2026	ADJ-2609-0041	100512	Leak credit	−184.00	Dana Whitfield	M. Okonkwo	BL-2608-018102
14/09/2026	ADJ-2609-0038	100244	Misread correction	−41.10	Dana Whitfield	—	BL-2609-018244
08/09/2026	ADJ-2609-0031	100907	Tampering charge	+220.00	M. Okonkwo	M. Okonkwo	BL-2609-018201
			Total	**−1,884.10**			

⚠ **The second row's approver is an em-dash and that is lawful** — it is under the tenant's threshold. The third row is self-approved, which the register shows rather than hides; whether that is allowed is the tenant's rule and this report is the evidence an auditor reads.

R39 Regulatory Filing

Compliance › Filings · journal lines + accounts + reads · Period

Period: FY2026 Q3 | Filing: PUC annual | Format: CSV

SECTIONS	VALUES	UNAVAILABLE	LAST FILED

6	**48**	**2** stated, not blank	**12 Jul 2026**

SECTION	LINE	MEASURE	VALUE	UNIT	SOURCE
1 · Connections	1.1	Residential connections	1,884	count	Accounts
2 · Volume	2.1	Water delivered	3,884,100	gal	Meter reads
3 · Revenue	3.1	Residential revenue	84,486.10	USD	Ledger
4 · Losses	4.1	Non-revenue water	not available	gal	—

⚠ **Line 4.1 states *not available* instead of zero, and that is the whole design.** A filing that prints 0 for a figure the platform cannot derive is a false statement to a regulator; a stated absence is honest and tells the filer exactly what to supply by hand.

R40 External Reconciliation

Compliance › Filings · journal lines + payments · Posting date

Period FY2026 P12 ⌄ **System** OpenGov ⌄

OUR TOTAL	THEIR TOTAL	DIFFERENCE	IN TRANSIT
$1,488,204.10	$1,488,204.10	**$0.00**	**3** $1,204.40

MEASURE	OURS	THEIRS	DIFFERENCE	IN TRANSIT	RESULT
Revenue	124,855.56	124,855.56	0.00	—	Agrees
Cash received	120,752.44	119,548.04	1,204.40	1,204.40	Explained
AR balance	48,221.14	48,221.14	0.00	—	Agrees
Total	**1,488,204.10**	**1,486,999.70**	**1,204.40**	**1,204.40**	

⚠ **A difference with an equal in-transit figure reads *Explained*, not *Agrees*** — two different facts, and collapsing them into one green pill is how a real break gets missed for a month.

R41 Migration Assurance

Compliance › Filings · accounts + journal lines · As at

As of 09/25/2026 **Migration** Go-live 2024-03 ⌄

ACCOUNTS IN	ACCOUNTS OUT	BALANCE IN	BALANCE OUT
1,884	**1,884** Match	**$44,118.02**	**$44,118.02** Match

CHECK	LEGACY	LOADED	DIFFERENCE	TOLERANCE	RESULT

Account count	1,884	1,884	0	0	**Match**
Opening balance	44,118.02	44,118.02	0.00	0.00	**Match**
Deposits held	18,400.00	18,400.00	0.00	0.00	**Match**
Read history rows	148,204	148,188	16	16 known	**Explained**

⚠ **Tolerance is zero for money and counts, and stated for history.** The sixteen missing read rows are the legacy system's own duplicates, which is why this report keeps a tolerance column instead of going red — but a money tolerance above zero would make the report worthless, so it is fixed at zero and not configurable.

Statement and Ledger — one parameter, four subjects

Two of these are drawn four times each, because one parameter changes who the report is about.

PO ruling: a finance statement is wanted **by account (regular and master) · by customer · by premises · by unit**. Three of those four already exist **as three separate bespoke pages** — the account ledger, the by-premises summary and the customer statement — and the fourth was never built. That is three implementations of one question at three grains, which is the duplication that took 95 requirements down to 46.

They collapse into two definitions with a *Subject* parameter. The arithmetic never changes; only the set of accounts rolled up changes. ⚠ **Master is not a fifth subject** — the code is explicit that *"a master account is an ordinary row"* with `kind='master'`, and there is deliberately no separate route for it. Same table, same column, same lookup.

SUBJECT	SELECTS ON	HOW MANY ACCOUNTS
Account	`accounts.id`	one — regular or master, no difference
Customer	`accounts.customer_id`	every account they hold, **across addresses**
Premises	`accounts.premises_id`	every account at that address, **the master included**
Unit	`accounts.unit_id`	every account on that unit

R24 · STATEMENT — DRAWN AT ALL FOUR SUBJECTS

R24a Statement · by account

Customers › Statements · accounts + invoices + payments · Statement date

Subject Account ⌄ | **Which** 100244 · Ruiz, Marisol ⌄ | **From** 07/01/2026 | **To** 09/30/2026

OPENING	BILLED	PAID	CLOSING
$212.40	$548.12	$576.00	$184.52

DATE	REFERENCE	DESCRIPTION	CHARGES	CREDITS	BALANCE
01/07/2026		Opening balance			212.40

12/07/2026	BL-2607-018204	Water & sewer · 12 Jun – 11 Jul	178.40		390.80
24/07/2026	PMT-2607-000311	Card payment		–212.40	178.40
12/09/2026	BL-2609-018244	Water & sewer · 12 Aug – 11 Sep	184.52		184.52
		Closing balance	**548.12**	**–576.00**	**184.52**

The shape every other subject is a roll-up of. ⚠ **This is what** `/reports/account-ledger` **is today — a whole page for one of four grains.**

R24b Statement · by customer

Customers › Statements · accounts + invoices + payments · Statement date

Subject Customer ⌄ | **Which** Vance Holdings ⌄ | **From** 07/01/2026 | **To** 09/30/2026

ACCOUNTS	OPENING	BILLED	CLOSING
3	**$1,880.10**	**$4,204.88**	**$2,418.90**

ACCOUNT	PREMISES	SERVICE	OPENING	CHARGES	CREDITS	CLOSING
100512	4400 Industrial Pkwy	Water · Sewer · Trash	1,204.10	4,118.90	–2,904.10	2,418.90
100641	91 Depot Rd	Water	418.00	86.00	–504.00	0.00
100688	91 Depot Rd, Unit 2	Water	258.00	0.00	–258.00	0.00
		Total	**1,880.10**	**4,204.90**	**–3,666.10**	**2,418.90**

⚠ **One customer, three addresses** — which is why *Customer* cannot be a filter on the account statement. The premises column is not stored on the account; it is **walked to premises**, hard rule 8.

R24c Statement · by premises

Customers › Statements · accounts + invoices + payments · Statement date

Subject Premises ⌄ | **Which** 1408 Elm Street ⌄ | **From** 09/01/2026 | **To** 09/30/2026

ACCOUNTS	UNITS BILLED	COMMON AREAS	CLOSING
5	**4**	**$69.60**	**$1,284.12**

ACCOUNT	UNIT	CUSTOMER	KIND	CHARGES	CREDITS	CLOSING
100901	Unit 1	Okafor, D.	Regular	188.40	–188.40	0.00
100902	Unit 2	Nwosu, A.	Regular	204.12	0.00	204.12
100903	Unit 3	Baptiste, L.	Regular	196.00	–96.00	100.00
100904	Unit 4	Alvarez, R.	Regular	212.40	0.00	212.40

100900	Common	Elm St Holdings	Master	69.60	0.00	69.60
			Total	**870.52**	**–284.40**	**586.12**

⚠ **The master account is a row like any other, marked, never merged.** Its **$69.60** of common-area charges is what the building itself consumes — corridor lighting, a shared tap — and a tenant reading this must be able to see it is not theirs. **This report crosses customers — and that is now ruled.** Staff see it as drawn. In the customer portal a landlord sees **their own master account only**: the **$69.60**, never Unit 3's $100.00 (**P4.15**).

R24d Statement · by unit

Customers › Statements · accounts + invoices + payments · Statement date

Subject Unit ⌄ | **Which** 1408 Elm Street · Unit 3 ⌄ | **From** 07/01/2026 | **To** 09/30/2026

OPENING	BILLED	PAID	CLOSING
$100.00	$588.00	$288.00	$400.00

DATE	REFERENCE	OCCUPANT	DESCRIPTION	CHARGES	CREDITS	BALANCE
01/07/2026		Baptiste, L.	Opening balance			100.00
12/08/2026	BL-2608-018310	Baptiste, L.	Water · 12 Jul – 11 Aug	196.00		296.00
19/08/2026	PMT-2608-000455	Baptiste, L.	Card payment		–288.00	8.00
12/09/2026	BL-2609-018377	Baptiste, L.	Water · 12 Aug – 11 Sep	196.00		204.00
			Closing balance	**588.00**	**–288.00**	**400.00**

⚠ **The only one of the four that did not exist in any form.** The head reads **1408 Elm Street · Unit 3** — the premises address and the unit *label*, because a unit refines its building by label and **declares no address of its own** (hard rule 8). An **Occupant** column appears here and nowhere else: a unit outlives its tenants, so a three-month statement can span two of them.

R25 · LEDGER — THE SAME FOUR SUBJECTS, IN THE BOOKS' OWN WORDS

R25a Ledger · by account

Customers › Statements · accounts + journal lines · Posting date

Subject Account ⌄ | **Which** 100512 · Vance Holdings ⌄ | **From** 01/01/2026 | **To** 09/30/2026

OPENING	DEBITS	CREDITS	CLOSING
$1,204.10	$8,884.20	$7,669.40	$2,418.90

DATE	TYPE	REFERENCE	DETAIL	DEBIT	CREDIT	BALANCE
12/08/2026	Invoice	BL-2608-018102	Water · Sewer · Trash	1,418.90		2,418.90
19/08/2026	Adjustment	ADJ-2608-0041	Leak credit, approved		–184.00	2,234.90

24/08/2026	Payment	PMT-2608-000402	Cheque 4471		−1,234.90	1,000.00
			Closing	**8,884.20**	**−7,669.40**	**2,418.90**

Same account as R24a's sibling, different question. ⚠ **Dated by POSTING, not by statement** — so a bill issued on the 12th and posted on the 13th sits in a different row of this report than of that one, and both are right.

R25b Ledger · by customer

Customers › Statements · accounts + journal lines · Posting date

Subject Customer ⌄ **Which** Vance Holdings ⌄ **Period** FY2026 P12 ⌄

ACCOUNTS	DEBITS	CREDITS	CLOSING
3	$4,204.90	$3,666.10	$2,418.90

ACCOUNT	GL ACCOUNT	NAME	DEBIT	CREDIT	CLOSING
100512	1200	Accounts receivable	4,118.90	−2,904.10	2,418.90
100641	1200	Accounts receivable	86.00	−504.00	0.00
100688	1200	Accounts receivable	0.00	−258.00	0.00
		Total	**4,204.90**	**−3,666.10**	**2,418.90**

⚠ **The GL account column is the same on every row and is drawn anyway.** It is what lets a finance officer tie this straight back to the trial balance's `1200` line — and on a tenant with more than one receivable account it stops being constant.

R25c Ledger · by premises

Customers › Statements · accounts + journal lines · Posting date

Subject Premises ⌄ **Which** 1408 Elm Street ⌄ **Period** FY2026 P12 ⌄

ACCOUNTS	DEBITS	CREDITS	CLOSING
5	$870.52	$284.40	$586.12

ACCOUNT	UNIT	KIND	DEBIT	CREDIT	CLOSING
100901	Unit 1	Regular	188.40	−188.40	0.00
100902	Unit 2	Regular	204.12	0.00	204.12
100903	Unit 3	Regular	196.00	−96.00	100.00
100904	Unit 4	Regular	212.40	0.00	212.40
100900	Common	Master	69.60	0.00	69.60
		Total	**870.52**	**−284.40**	**586.12**

⚠ **This is what** `/reports/by-premises` **is today** — an entire page, its own table, no paging, for one cell of a 2×4 grid. It is also the row my own register left dangling: it said by-premises *"becomes a definition"* and never named which one. **It is this.**

R25d Ledger · by unit

Customers › Statements · accounts + journal lines · Posting date

Subject Unit ⌄ **Which** 1408 Elm Street · Unit 3 ⌄ **Period** FY2026 P12 ⌄

OPENING	DEBITS	CREDITS	CLOSING
$100.00	**$196.00**	**$96.00**	**$200.00**

DATE	TYPE	REFERENCE	OCCUPANT	DEBIT	CREDIT	BALANCE
01/09/2026			Baptiste, L.			100.00
12/09/2026	Invoice	BL-2609-018377	Baptiste, L.	196.00		296.00
24/09/2026	Adjustment	ADJ-2609-0052	Baptiste, L.		−96.00	200.00
			Closing	**196.00**	**−96.00**	**200.00**

⚠ **A unit is a place, and places do not owe money — people do.** The closing balance here is the sum of whoever was in occupation, which is what a landlord wants and is **not** a debt anybody can be chased for. That distinction is why the Occupant column is mandatory on this subject and optional on none of the others.

04 What was walked

Every finding below came from **driving the running product**, not from reading it. Postgres 16 on port 5433, all 278 migrations, `seed_demo`, a Vite build served by `server.mjs` with the API proxied, and Playwright signed in as Dana Whitfield on a seeded tenant. **No gate in this repo looks at rendered output** — hard-won law 6 — so this section is the only evidence in the document that is not a file read.

One of the twelve was read wrongly the first time and the correction is in row 1. It is left in place, marked, rather than tidied away: it is the only finding in this document that would have caused real damage if it had been built, and the shape of the mistake — trusting a signed spec line over a later ruling — is the one this repo warns about most often and catches least.

Twelve journeys were walked. Six could not be, and the reason is the same for all six.

They are **4·1 Scheduled**, **4·2 Schedule opened**, **5·1 History**, **5·2 Run opened**, **2·2 the builder panel** and **E·2 the Settings section** — **none of them exists**. Four of the six have a working backend behind them: the job poller runs, `GET /api/exports/{job_id}/file` serves, and the export path writes real files. **So the honest count is twelve of eighteen, and the six that were not walked are the six this plan builds** — not a gap in the walk, the shape of the work.

#	WHAT WAS DONE	WHAT HAPPENED, AND WHAT IT MEANS	CLOSED BY
1	Signed in, landed on the dashboard, read the window dropdown	It offers **Last 30 days · This period · Last period**, defaults to the first, and prints the exact dates above the figures. **A1 says four windows defaulting to the last completed cycle.** ⚠ **I reported this as a ruling the code had missed and it is the reverse** — `CR-029` D8① withdrew the cycle window on 23 August, PO verbatim *"dont build it"* (`OQ-268`). **The code is right and the spec is stale by a month.** Acting on my first reading would have re-added a feature the PO declined, and it would have passed review on a real citation.	**P0.15**
2	Opened `/reports` and counted the cards	**23 cards**, against 46 definitions in the requirement set. Not a defect on its own — but it is the measurement that makes the rest of the plan a plan rather than an opinion.	**P1.3**
3	Clicked every card on the catalogue	Several are **not links at all** — a card with no destination, no disabled styling and no stated reason. It reads as a broken page rather than as an honest absence. ⚠ **This is hard-won law 2's mirror**: not dead UI, a dead door over a live screen.	**P1.4**
4	Clicked the **Audit log** card	It routes to `/settings/audit-log` . **No Settings tab has that id**, and `SettingsPage.tsx:475` does `tabs.find(…) ?? tabs[0]` — so it silently lands on **Settings → Company**. The clerk asked for the audit log and got the company profile, with no error anywhere. ⚠ **Every check passes over this**: the route resolves, the link is a link, `tsc` is clean, and the fallback is doing exactly what it was written to do.	**P1.4 · P1.6**
5	Scrolled to the bottom of the catalogue	An **orphan paragraph link** — `<p><Link>Account ledger</Link></p>` at `ReportLibraryPage.tsx:236` — sits under the grid, outside every card, unstyled and unexplained. It is a real destination reachable only by a link that looks like a mistake.	**P1.3**
6	Opened a report with a money total on a multi-currency tenant	The **AR total tile refuses** rather than adding — `currency_not_derivable` , reason `receivable_holds_more_than_one_currency` , at `queries.py:320` . **This is correct and it is the best-built thing in the module**: it would have been far easier to sum the numbers and be wrong. The finding is that **the screen renders an empty tile and not the sentence** — ⚠ and the fix needs no design work, because **the dashboard already solved this**: `BR-REP-033/024` says a tile whose owner publishes no count reads **Not counted yet** with the reason in its support text, and that pattern ships on **E·1** today. **P2.13** is applying an existing rule, not inventing one.	**P2.13**
7	Tried to search, sort, page and pick columns inside a report	**None of it is there.** `reportsShared.tsx` documents the refusal of `DataTable` for S59 and S60 in as many words. So the product's own 905-line table — search, paging, column picker, the B14 rule 12 set — is **deliberately not used by the screen that most needs it**.	**P2.5**
8	Found and pressed the download control	It is **an icon with no label** — `h-9 w-9 justify-center px-0` at `DownloadButton.tsx:49` . It works. It is also the single most-wanted control on the screen and it is a 36-pixel square with no word on it.	**P2.5**
9	Watched what the download actually did	`POST /exports` returned **202 with a job id**, correctly, and toasted *"Your CSV is being written. It appears in Jobs when it is ready."* ⚠ **There is no Jobs screen in this module.** The sentence is true of the platform and false of everywhere the clerk can reach from here — **a promise with no door**, which is exactly the shape that put five portal cards behind dead controls on 5 September.	**P3.3**
10	Compared what the API accepts against what the screen draws	`queries.py:255` reads `f.class` — a served, working **customer-class filter with no control anywhere on the screen**. The backend can answer a question the front end cannot ask.	**P2.6**
11	Read the export cap from the code, then from the docstring	The live constant is `EXPORT_ROW_CAP = 100_000` (`exports.py:120`). The module docstring at `api.py:28` says **">50,000 rows"**. ⚠ **I quoted the docstring in the first draft of this document and it was wrong** — hard-won law 1, caught by reading the constant instead of the comment about it.	**P2.4**
12	Traced one export from the	`tenancy/services.py:6202` calls `selection_count(resource, select(params))` and then builds the same	**P2.4**

2	button to the file	selection again to write it. **The report is computed twice** — once to say how many rows there will be, once to produce them. On a 40,000-row ageing report at T5 that is the whole cost, doubled, for a number the clerk does not need before the file exists. ⚠ **And the writer already streams row by row from a generator** (`exports.py:546`), so removing the cap is a deletion and not a rewrite — which is the finding that made **P2.4** a one-line task instead of a phase.	

Two things the walk found that are dead rather than broken, and they are worth naming separately.

`VIEWER_KEYS` at `catalog.py:415` and `card_for()` at `catalog.py:420` have **no reader anywhere in the repo**. They are not wrong; they are furniture. A lane extending the catalogue will read them, assume they are the interface, and wire to something nothing calls. **P0.7** deletes both — and deleting is the fix, because leaving a plausible unused export in a module is how the next lane loses a day.

05 What this module needs from others

Reports owns no business data. Every number on every one of the 46 definitions is somebody else's row, read through a seam. **So this section is not a courtesy — it is the build order.** Nine datasets, nine owners, and one dependency that gates more of the plan than the other eight together.

Settings

The first answer to this question was wrong, and it is worth stating what was wrong with it because the correction changed the shape of Tab 5 completely.

What I said first: a new Settings tab, `/settings/reports`.

That was wrong twice over. **Settings already carries nine top-level tabs** and the customer-portal blueprint's §15 records the miscount that argued for a tenth — a tenth was refused there, with reasons that apply here unchanged. And more importantly, **reports configuration is reference data by every test that matters**: set once, left alone, a list of rows a tenant edits and forgets. That is what `Configure → Reference data` is, and it already renders 35 cards in 10 module sections with search, counts, a row drawer, a permission check and an audit write.

The corrected address is `Settings → Configure → Reference data → Reports` , which in the route grammar is `/{tenant}/settings/reference-data/reference-data` . It costs **one string in** `REFERENCE_GROUP_ORDER` and **four rows in** `referenceDataConfig.ts` . The first answer would have cost a route, a page file, a test file, a permission entry, a nav decision and eight B14 rules re-earned from scratch.

WHAT	WHERE EXACTLY	STATE	WHY THERE
Report groups	Configure → Reference data → Reports	NEW	A list of rows with names a tenant may rename. Copy-on-write over the 6 platform groups and 19 sub-groups
File retention	Configure → Reference data → Reports	NEW	Replaces `ARTIFACT_RETENTION_DAYS = 3` , a constant at `exports.py:123` that no screen shows and no tenant can change
Default format	Configure → Reference data → Reports	NEW	What a download is unless the clerk asks otherwise. A schedule cannot be built without it, which is why it lands in `P4.6` with retention
Delivery	Configure → Reference data →	NEW	Where a schedule sends its file. A list, because a utility has more than one recipient

	Reports		
Number and date formats	Company → Regional	EXISTS	Already there and already applied. Reports reads it and adds nothing
Ageing bands	Configure → Reference data → Payments	EXISTS	⚠ **Not a reports setting.** The five bands are a receivables rule; the ageing report renders whatever Payments defines. Duplicating them here is how two screens come to disagree
GL account mapping	Settings → Accounting	EXISTS	Not a reports setting. Trial Balance and Journal Export read it
Billing cycles	Configure → Routes & cycles	EXISTS	Not a reports setting. The dashboard window derives from it — which is precisely why the window is a *default* row and not a cycle definition
Who may run what	Users & roles → Roles	EXISTS	⚠ **Considered as a Reports setting and dropped.** A permission belongs where every other permission is, or the two lists diverge and the narrower one is ignored
Export limit	Nowhere	DROPPED	⚠ **Drawn here in the first pass and it breaks a signed NFR.** `NFR-5` forbids hard caps and `P2.4` *deletes* `EXPORT_ROW_CAP = 100_000` — a one-line change, because the writer at `exports.py:546` already streams from a generator. A Settings field would re-introduce the cap with a nicer interface. **The cap goes away; it does not move**
Dashboard window default	Nowhere	DROPPED	⚠ **Also drawn here first.** The default is **ruled** — the rolling thirty days, after `CR-029` D8① withdrew the cycle option — and the server decides it from the options it could build. **A tenant row overriding a PO-ruled default is a ruling question, not a settings row**
Per-report column choice	Nowhere — it is a saved definition	DROPPED	This is what Tab 2's builder panel is *for*. Putting it in Settings would be a second editor for the same row

The doors into Statement and Ledger

Built once, in Reports. Reached from wherever you are standing — and **one of the four has nowhere to stand**, which is the kind of thing worth finding before a lane draws a link onto a screen that does not exist.

STAND ON	ADDRESS	EXISTS?	THE DOOR
Account detail	`/accounts/:accountId`	YES	**Statement** and **Ledger** as account actions. ⚠ A1's S22 already lists a *Statement* action modal and the A1 addendum already says *"also reachable as Download ledger from Account detail"* — **this door is signed and unbuilt**, not new
Premises detail	`/premises/:premisesId`	YES	Both, on the premises subject. This is where the master account's common-area charges become visible beside the units'
Unit detail	`/premises/units/:unitId`	YES	Both, on the unit subject — **the grain that has never existed anywhere in this product**
Customer	—	NO	⚠ **There is no customer detail screen.** `features/customers/` holds `CustomersListPage` and no detail page at all. So the customer door is a **row action on the customers list**, and no link is drawn onto a screen that does not exist —

			which is precisely the defect that left five portal cards pointing at nothing
Reports catalogue	`/reports`	YES	Open the card and pick the subject yourself. The only door where the subject is not already decided for you

The three pages this replaces, and what is kept out of each.

`/reports/account-ledger` (S59), `/reports/by-premises` (S60) and `CustomerStatementPage` — three implementations of one question at three grains. **All three addresses keep working and redirect**, because a bookmark surviving a refactor is the difference between a tidy-up and an outage.

Two things are carried across rather than rewritten, both from the customer statement: the `held_with` guard that stops a stranger's arrears appearing on this person's statement, and the `AbsentFigure` that states the server's own reason instead of printing `$0.00`. Those are the careful parts of a page that is otherwise duplication, and losing them in a refactor would be the expensive kind of mistake.

Every other module

MODULE	DATASET IT OWNS	READS	WHAT REPORTS NEEDS FROM IT, EXACTLY
billing	invoices · invoice lines	read only	`limit` / `offset` / `sort` on the register selection, and the locked-at-issue snapshot exposed as fields rather than recomputed. **Feeds 12 of the 46**
payments	payments · allocations	read only	Server pagination on `GET /outstandings` and `GET /cheques`, which are unpaginated today and name the retired 3,000 ceiling as the reason. **Feeds 11**
ledger	journal lines	read only	A selection by posting date with account range, paginated. **Feeds 9, including every Finance definition**
metering	meter reads · assets	read only	Read basis exposed (actual / estimated) per row — without it R45 cannot state a basis and R28 cannot separate a read gap from a scheduling gap. **Feeds 7**
customers	accounts · customers	read only	Class, status and enrolment as filterable fields. ⚠ `class` is already served and has no control drawn — **P2.3**
rating	plans · rate schedules	read only	Effective-dated history with the 4-decimal rate and 6-decimal percentage intact. R37 is the one report that must not round
tenancy	`jobs`	read + **write**	⚠ **The one write this module makes.** An export inserts a `jobs` row. `report_runs` is deliberately not created — the job record *is* the run record
notifications	`audit_log`	read + **write**	Reads it for R35 and R36; writes one row per export, per saved definition, per schedule change. Hard rule 7
work	work orders · crews	read only	Through `apps/work/seams/`, read-only, enforced by **G78** from Work's first commit. **Feeds 2**
identity	users · roles	read only	R36 is nine rows and needs last-sign-in and who-added-whom, which the module has
portal	—	host	Renders R44, R45 and R46 in the customer frame. Same definition rows, same datasets, different chrome — and the **eight accessibility questions** apply to those three and to nothing else in this module

The dependency that gates most of this plan, and it is not a reports task at all.

The billing, payments and ledger read models must accept `limit`, `offset` and `sort`. Three of the nine datasets do not today, and those three feed

32 of the 46 definitions — every Finance report, every Receivables report and most of Billing.

It is not a nicety. `NFR-3` requires universal server pagination and `GET /outstandings` is **unpaginated on purpose**, with a docstring that says so and names the reason: *"one row per DEBTOR at a utility under 3,000 connections"*. ⚠ **That ceiling moved to 40,000 on 6 September.** The endpoint also **sorts in Python rather than in SQL**. Correct at T7, **13× short at T5**, and filed as `DEF-PAY-4221` across four places plus two front-end mirrors.

So the honest sequencing is: Reports cannot be finished at T5 scale until DEF-PAY-4221 is. The 46 definitions can be built, drawn and demonstrated on a T7 tenant without it — and the moment a 40,000-account utility opens the Receivables group, three datasets return everything they have in one response. **P2.12** is the task that pins this: it asserts `limit` / `offset` / `sort` on all nine datasets and **goes red on three of them today**, which is the point. A dependency with a failing test beside it is a dependency; one described in a paragraph is a hope.

06 The register

Every screen this module has, will have, or is losing — **sixteen rows**. Thirteen live, three removed. A row is here whether it is built, half-built or drawn for the first time in this document, because the point of the register is that **no screen in this module is reachable without appearing in it**.

TAB	SCREEN	ADDRESS	CLASS	REPO ID	TASKS	WHAT IT NEEDS
1	**1·1 Catalogue · Standard**	`/{tenant}/reports`	UPDATE	**S44**	P1.1–P1.8	45 cards from `report_definitions` instead of 23 from a frozen tuple · 6 group headings · 19 sub-group rails · the four-tab strip · the Standard · Custom sub-strip · **every card a real link or an honest non-link with its reason on its face**
1	**1·2 Catalogue · Custom**	`/{tenant}/reports/custom`	NEW	**S-NEW**	P5.10 · P5.1 · P5.7	What this utility wrote, grouped like Standard, with **Owner · Based on · Private**. ⚠ **Origin was a filter until the PO ruled it a sub-tab** — a dropdown hides that you can author reports at all. Reads **0** on day one and says what to do about it
2	**3·1 Dashboard**	`/{tenant}/reports/dashboard`	NEW	**S-NEW**	P4.11–P4.13	**A new tab, ruled — not a replacement.** Four KPIs, five panels, every panel a definition rendered small, a real accounting period and a comparison against another. ⚠ **Nothing on it computes anything**, or the dashboard and the reports start disagreeing
3	**4·1 Scheduled**	`/{tenant}/reports/scheduled`	NEW	**S45**	P4.5	One table of `report_schedules`, with a live count in the tab. ⚠ **A top-level tab, not a half of one** — Activity is deleted
3	**4·2 Schedule opened**	`…/scheduled/{id}`	NEW	**S-NEW**	P4.5 · P4.8	Pause · edit · run now · delete, and **the last failure visible on the row** rather than in a log nobody opens
4	**5·1 History**	`/{tenant}/reports/history`	NEW	**S45**	P3.1–P3.3	**The cheapest screen in the plan and the one that closes a live broken promise.** The export

						toast already says a file *"appears in Jobs"* and there is nowhere to go. The `jobs` table, the poller and `GET /api/exports/{job_id}/file` all exist — **this is a table over data already there**
4	**5·2 Run opened**	`…/history/{id}`	NEW	**S-NEW**	P3.2 · P3.4 · P3.5	What ran, when, by whom, over which parameters, and re-run. **Nothing on it is editable — a run is a historical fact**
—	**2·1 Report**	`/{tenant}/reports/{key}`	UPDATE	**S46**	P2.1–P2.16	Opened from a card, so not a tab. **Four drawn states** — KPI+table, table only, KPI+table+detail, and the measured refusal. `DataTable` instead of a bespoke table, so search · sort · paging · column picker arrive at once. **This one screen renders all 45 definitions**, and the **Subject** parameter gives Statement and Ledger four grains each
—	**2·2 Edit panel**	`/{tenant}/reports/{key}?edit=1`	NEW	**S47**	P5.1–P5.3 · P5.11	A live panel beside the report, **not the shared wizard frame** — ruled. ⚠ **Columns can be ticked, dragged and renamed; they cannot be invented**, and the panel says how many the dataset has so the limit is visible rather than mysterious
—	**2·3 Schedule**	modal on `2·1`	NEW	**S-NEW**	P4.1–P4.4	⚠ **One form, three fields** — cadence, recipients, format. **Not a wizard**: the report and its parameters are already chosen
—	**2·4 Save as**	modal on `2·1`	NEW	**S-NEW**	P5.4 · P5.7	Name, group, and who can see it — **private by default**, which is the whole reason this is a modal and not a silent save
E	**E·1 Shell dashboard**	`/{tenant}/dashboard`	KEEP	**S7** · DSH-01	P0.15	⚠ **Fenced off by PO ruling — *"dont replace existing one"*.** No task in this plan touches a tile, a label or a column. It is in the register because `apps/reports/` serves it and a lane editing that code must know. **Its only work is a documentation fix**: A1 describes four period windows defaulting to the last completed cycle, and that window was withdrawn a month ago
E	**E·2 Reports settings**	`…/settings/reference-data/reference-data`	NEW	**S8**	P4.6 · P5.5	**A section, not a screen.** One string in `REFERENCE_GROUP_ORDER`, four rows in `referenceDataConfig.ts`. No route, no page file, no new Settings tab — and eight B14 rules inherited rather than re-earned
—	~~**Account ledger**~~	`/{tenant}/reports/account-ledger`	REMOVED	**S59**	P2.8 · P2.16	**Becomes R25a — Ledger, subject = Account.** A bespoke page with its own table, its own date handling and no paging, for one cell of a 2×4 grid. ⚠ **The id is not retired** — A1 keeps S59 and it now names a definition rather than a screen

—	~~**By-premises summary**~~	/{tenant}/reports/by-premises	REMOVED	**S60**	P2.8 · P2.16	**Becomes R25c — Ledger, subject = Premises. ⚠ This row is why the feedback round was worth it:** the previous register said this screen *"becomes a definition"* and never named which. It had no number and nothing would have caught that
—	~~**Customer statement**~~	/{tenant}/reports/customer-statement	REMOVED	—	P2.8 · P2.16	**Becomes R24b — Statement, subject = Customer. ⚠ Its careful parts are kept, not discarded:** the `held_with` guard that stops a stranger's arrears appearing on this person's statement, and the server-reasoned `AbsentFigure`. ⚠ **It has no A1 id** — its own header cites S59 and S60, which are two other screens

Every screen's door — audited, not asserted

A screen with no way in is the failure this repository has met most often and caught least: on one day the customer portal's front page had **two working doors out of eight**, over five screens that were routed, tested and live. **So this is a table rather than a sentence**, and a check in the build refuses to publish this document if any screen loses its door row.

SCREEN	DOORS	HOW YOU GET IN	BUILT?
1·1 Standard	4	Rail item 12 · the dashboard's *See revenue by service* · the report's ← *Reports* · the breadcrumb	YES
1·2 Custom	2	The sub-tab strip · the confirmation after **Save as**	NO
2·1 Report	3	Any card on 1·1 or 1·2 · a panel on the Dashboard tab · a saved link	YES
2·2 Edit	2	**Edit** on 2·1 · `+ Build your own` after the dataset picker	NO
2·3 Schedule	1	**Schedule** on 2·1	NO
2·4 Save as	1	**Save as** on 2·2	NO
3·1 Dashboard	**1**	The tab strip. ⚠ **Nothing else, and it is ruled** — a link from the shell dashboard would mean touching a screen the PO fenced	NO
4·1 Scheduled	2	The tab strip · **Schedule**'s confirmation on 2·3	NO

4·2 Schedule opened	1	A row on 4·1	NO
5·1 History	3	The tab strip · **the export toast on 2·1** · a direct link	NO
5·2 Run opened	1	A row on 5·1	NO
E·1 Shell dashboard	**6**	Rail item 1 · tenant-root redirect · post-sign-in · after a password change · every *Back to Dashboard* · global search's fallback	YES
E·2 Reports settings	1 → 2	The Configure grid · **and P5.12 adds a link from 5·1's retention line, for an admin**	NO

Running this audit found three real defects in this document, which is the argument for running it.

Four screens had no door row at all — 1·2, 3·1, E·1 and E·2 — and two of those four are new screens, which is exactly where a missing door survives review.

Two addresses were wrong, and both were collateral from renumbering. Deleting the Activity tab meant rewriting `/reports/activity` everywhere, and the rewrite sent **5·2 Run opened** to `/reports/scheduled` — a run drawer hanging off the schedules list. **Nothing else in the document would have caught it**: the drawing was right, the prose was right, and the address was a string in an attribute nobody reads.

The lesson generalises past this document. The check that found it is one that asks each screen the same question and prints the answer beside the name. A check whose output is a table is read; one whose output is an exit code is not.

Six numbers must be allocated in A1 before Phase 3 starts, and one existing row must be read carefully.

The six **S-NEW** rows — 1·2, the Dashboard tab, 2·3, 2·4, 4·2 and 5·2 — have no id. ⚠ **Take them with** `10-scripts/allocate.py` **, not by hand**: the allocator asks the remote, a hand count reads only this checkout, and two lanes computing *max + 1* correctly is how three lanes took the same gate number in one afternoon. Run `--verify` again at push time, unconditionally, in the same breath as the blob re-derivation.

The row to read carefully is S45. A1 defines it as *"Reports scheduled & history"* — **one id for what is now two top-level tabs over two tables**. That is not a conflict; it is one id covering two screens. But a lane that reads S45 and builds one table will have built half of it and passed every check, so **the register says S45 twice on purpose**.

What the register adds up to, and the one number worth holding on to.

Thirteen screens, and only three are pages that did not exist in some form. Two are updates to screens that ship, four are modals and detail panes over data already in the database, one is a section of somebody else's Settings screen, one is a dashboard nobody may touch, and three bespoke pages collapse into two definitions that were already there. **The genuinely new build is the Dashboard tab, the Custom sub-tab, and Scheduled and History** — and History, the single most valuable screen in the plan, is a table over the `jobs` rows the export path has been writing all along.

⚠ **So the risk in this module is not build volume. It is that 32 of the 46 answers read three datasets that cannot page**, and that dependency sits in payments, billing and ledger rather than here. **P2.12** is the row that names it and **it is red today on three of nine datasets**, which is the honest state of the ground this plan is built on.

07 The decisions

Every architecture choice this module rests on, with **FINAL** or **OPEN** beside it. A choice that is only implied by a drawing is a choice nobody made — so each one is stated, with what it buys, what it costs, and what would have to change to reverse it.

7.1 · The framework rule

Datasets are code. Definitions are data.

A **dataset** is a registered, reviewed query against one module's read model — nine of them, written by us, changed by a pull request. A **definition** is a row: which dataset, which columns, which filters, how it groups and sorts, where it sits in the catalogue. **The 45 reports are 45 rows**, and a utility writing its own writes a forty-sixth row rather than waiting for a release.

What it buys. A new report is a row, not a deployment. A tenant can change any report without a fork. One screen renders all of them, so a fix to paging fixes 45 reports at once.

What it costs, stated plainly. You cannot invent a column the dataset does not produce. That is the wall, it is deliberate, and it is the reason a clerk can be trusted with a report builder at all — everything they can express has already been reviewed for tenant isolation, for money precision and for cost.

7.2 · The dimension model

⚠ Raised by the PO looking at AR Ageing: two slicers where the fact has about a dozen. The answer is not more chips on one report — it is that a dataset declares its dimensions **once**, and every report built on it inherits all of them.

TERM	WHAT IT IS HERE	STATUS
Fact	a **dataset** — 9, in code	EXISTS
Grain	what one row of that dataset *is*	written nowhere — **P0.16**
Measure	a money or quantity column	exists, **untyped**
Aggregation	SUM · COUNT · DISTINCT · AVG · MIN/MAX	assumed — **P0.16**
Additivity	whether a measure may be summed, and **across what**	assumed — ⚠ **this one produces wrong money**
Dimension	a way of slicing	**hand-picked per report** — the defect
Hierarchy · Level	Area → Route → Premises → Unit	not declared
Slice · Dice	one filter chip · several	EXISTS
Roll up · Drill down	move along a hierarchy	**P2.17**
Drill through	an aggregate row → the facts behind it	**P2.18**

Drill through	an aggregate row · the facts behind it	**P 2.13**
Conformed dimension	the same dimension across several facts	⚠ **not enforced — this is how two reports come to disagree**
Degenerate dimension	lives on the fact, no lookup — an invoice number	EXISTS
Pivot · calculated measure	rows × columns · an expression	NAMED, NOT BUILT
SCD Type 2	attribute history with effective dates	⚠ see **7.2.3**

7.2.0 · WHAT WE BUILD, AND WHAT WE DELIBERATELY DO NOT RULED

TIER	WHAT	WHEN	WHY THAT BOUNDARY
A	**Declare the model in code.** Every dataset states its **grain**, its **measures** with aggregation and additivity, its **dimensions** with attributes and hierarchies	Phase 0–2	⚠ **The one that matters, and it is cheap — a data structure, not a screen.** Without it B and C are guesswork, and the additivity traps stay invisible
B	**The UI derives from it.** Slicers for every dimension · group to three levels with subtotals · drill down one level · drill through to the fact rows	Phase 2–3	Two screens change and the other eleven inherit it. The dashboard gains all of it for free, because its panels *are* definitions
C	**Named, not built** — pivot (rows × columns) · calculated measures · as-was reporting	a change request	Each is real work with a real grammar. Named so that when a customer asks it is a request with a figure, not a surprise
—	~~**A general MDX or cube engine**~~	**NEVER**	We serve **10–20 staff**, not a BI department. A query language is a product of its own, and the utility that needs one should point Power BI at our API instead

7.2.1 · WHAT AR AGEING OFFERS, AND WHAT IT COULD

DIMENSION	ON SCREEN	IN THE DATA
Time — as at	✓	the slicer
Consumer class	✓	✓
Service	—	✓ · eleven possible
Account status	—	✓
Customer type — person / company	—	✓
Premises · locality	—	✓ — walked to premises, hard rule 8
Unit	—	✓
Route · Cycle	—	✓
Area	—	✓
Rate schedule	—	✓

On a payment plan	—	✓
Collections state	—	✓
Age bucket	as columns	✓ — a derived dimension

Two of thirteen. Multiply that by 45 reports and the hand-picking is the whole problem.

7.2.1B · WHAT EACH CONCEPT COSTS US, ON SCREEN AND IN THE REQUIREMENTS

CONCEPT	SCREEN CHANGE	WHERE	REQUIREMENT
Grain	A line under the title — *"One row per account"*	**2·1**	**New** — every report says what one row is
Measure	The column list splits into **Attributes** and **Measures**	**2·2**	Existing, made explicit
Aggregation	Each measure carries `Sum ⌄` `Count ⌄` `Average ⌄`	**2·2**	**New** — a measure declares how it totals
Additivity	⚠ A total that is not lawful shows **an em-dash and its reason**, never a number	**2·1** footer	**New** — a total that cannot be trusted is refused
Dimension	**Four visible slicers +** `+ Filter` for the rest. ⚠ **Thirteen chips at once is a worse screen** — the definition names the four	**2·1** · **2·2**	**Changed** — filters come from the dataset, not the report
Hierarchy · Level	*Group by* becomes **three slots**, each a dimension and its level	**2·2**	**New** — group to three levels with a subtotal at each
Drill down	A group row expands in place; collapse restores it	**2·1**	**New**
Drill through	A subtotal opens the rows behind it, that group's values already applied	**2·1**	**New** — a subtotal is checkable
Conformed dimension	*Service* looks and behaves identically on all 45	everywhere	**New** — enforced by a test, not by care
Time hierarchy	The head says **which calendar** — *"Grouped by billing cycle"*	**2·1** · **2·2**	**New** — one per report, mixing refused
Basis	*"Based on today's customer details"*	**2·1**	**New** — a report states whether it describes then or now
Pivot	~~**None**~~	—	Named, not built

Two screens change. Eleven inherit it. That is the framework paying for itself.

2·1 gains the slicer bar, the grain and basis lines, the calendar statement, subtotals, drill down, drill through and a footer that can refuse. **2·2** gains split columns with an aggregation each, filters from the declared dimensions, three group slots and the time hierarchy. ⚠ **The Reports dashboard gains all of it for nothing**, because its panels are definitions. Catalogue, Custom, both modals, Scheduled, History and both Elsewhere screens are **untouched**.

The requirement map grows from 42 capabilities to 50; the definitions stay at 45. These are capabilities of the framework, not eight more reports — and that distinction is the whole argument for building the model rather than eight features.

7.2.2 · ADDITIVITY — THE TWO TRAPS THAT PRODUCE WRONG MONEY

MEASURE	ADDITIVITY	THE TRAP
Invoiced amount · payment amount · tax collected	**fully additive**	none — sum freely, in any direction
AR balance · deposits held · accounts active	**semi-additive**	⚠ sums **across accounts**, never **across time**. Group AR Ageing by month, total the column, and the number means nothing. **Nothing stops you today**
DSO · collection rate · % share · average days overdue	**non-additive**	⚠ must be **recomputed from its components**, never averaged. The average of four route DSOs is **not** the utility's DSO
Consumption	additive **within one service**	gallons plus kilowatt-hours is not a number — hard rule 3 already forbids it, and this is the same rule stated as metadata

Declaring additivity turns each of these from **a bug somebody finds in production** into **a refusal at build time**. That is the whole argument for **P0.16**, and it costs a data structure rather than a feature.

7.2.3 · TWO TIME HIERARCHIES, AND THEY DO NOT NEST

A billing cycle does not sit inside a calendar month. It straddles two.

Calendar: Year → Quarter → Accounting period → Day. · **Billing:** Year → Cycle → Day.

So a report must state **which hierarchy it is rolling up**, and one that mixes them is wrong in a way **no total will reveal**. This is the same fault line as the dashboard period ruling — a rolling thirty days straddles two cycles and answers nobody's question — and it is why the Reports dashboard uses a real accounting period rather than a rolling window.

7.2.4 · AS-WAS OR AS-IS `RULED — OPTION (B)`

RULED: keep history for class, type and status. Every report that slices by a customer attribute answers as-is today, silently, and nobody decided that — now somebody has.

An account moves from Residential to Commercial in September. You run *Revenue by class for July*. **As-is** puts July's revenue under Commercial; **as-was** puts it under Residential. For regulatory and financial reporting **as-was is almost always right**.

⚠ **Measured, not assumed:** `06-backend/apps/customers/models.py` has **no effective dating of any kind** — no `effective_from`, no history table. Billing, metering, rating and work all have it. **Customers does not.** A class change **overwrites**, and the only trace is `audit_log`, which is pruned on a schedule the PO ruled has **no exemptions**. So as-was is not merely unbuilt — beyond the retention window it is **unrecoverable**.

	OPTION	WHAT YOU GET
a	Change nothing	As-is everywhere, **stated on screen as the basis** so nobody mistakes it for as-was
b	One effective-dated table on customers	**CHOSEN.** As-was for **class, type and status** — the three that matter for revenue reporting. About a week's work, and it answers the state correctly **from the day it ships**
c	Full SCD Type 2 across customers	As-was for everything, and a much larger migration

Option (a)'s labelling ships anyway — **P2.19** — because a report that silently answers a different question from the one asked is worse than one that refuses.

The part of this ruling that is not reversible, stated plainly.

(b) starts recording on the day it ships. It recovers nothing. Every class change before that date is already gone — overwritten in place, with its only trace in an audit log that is pruned on schedule. ⚠ **So the cost of this decision is measured in months of history nobody will ever be able to produce**, and it goes up every month until **P0.18** lands. That is the argument for putting it in Phase 0 rather than at the end.

What a report does in the meantime, and afterwards. Where history exists it answers **as-was**, because that is what a regulator is asking for. Where it does not — anything before the table ships — it falls back to **as-is and says so on the face of the report**. ⚠ **One report will state both** for the first year, and that is correct rather than untidy: the alternative is a single confident basis line that is true of half the rows.

7.3 · What the 45 answer

STAGE	COUNT	HOW IT COLLAPSES
Requirement rows	**95**	Read from five documents — the RFPs, the CSV, the existing catalogue
Capabilities	**50**	Distinct things a user needs to be able to do. ⚠ **42 until the dimension model was added** — slicing by any dimension, grouping to three levels, drilling down, drilling through, choosing an aggregation, being refused an untrustworthy total, knowing what one row is, and knowing whether a report describes then or now are **eight capabilities of the framework**, not eight more reports
Definitions	**45**	⚠ **More than the capabilities, not fewer.** Some capabilities need two reports — one for a clerk and one for a regulator — and three are portal-facing versions of a staff report
Screens	**1**	All 45 render on **2·1**. That is the framework rule paying for itself
The forty-sixth answer	**a tab**	The Reports dashboard. Not a card, because a dashboard has no parameters, no rows and no file

7.4 · The data model

TABLE	TENANT	HOLDS
`report_definitions`	nullable	One row per report. **NULL tenant = a platform row** — the 45 we ship. A tenant row is that utility's own, or a copy of a platform one
`report_categories`	nullable	6 groups, 19 sub-groups, same rule. A tenant may rename a group without renaming everyone's
`report_tenant_settings`	required	Retention · default format · delivery · group overrides
`report_schedules`	required	Cadence, recipients, format, last run, last failure
`jobs` EXISTS	required	⚠ **The run record.** Every export is a job row already
`report_runs`	—	**DELIBERATELY NOT CREATED.** It would duplicate `jobs` — same statuses, same progress, same result link — and two tables recording one event drift within a month. The module's own docstring already says so

7.5 · The read path

PATH	WHEN	HOW
Sync	viewing — page 1	`GET /reports/{key}?…` returns rows with `limit` / `offset` / `sort` applied **in SQL**. Never the whole result
Async	everything that produces a file	`POST /exports` → **202 + job id** → poll → `GET /exports/{job_id}/file` . ⚠ **All three already work.** What was missing is a screen to see the result on — that is the History tab

Two things measured in the code that changed what this costs.

The writer already streams, row by row from a generator (`exports.py:546`). So removing the 100,000-row cap is **a deletion, not a rewrite** — and `NFR-5` forbids reinstating it anywhere, including on a Settings screen.

The report is built twice. `tenancy/services.py:6202` counts the selection and then builds it again to write it. On a 40,000-row ageing report at T5 that is the whole cost, doubled, for a number the clerk does not need before the file exists. The count goes with the cap.

7.6 · Scale

TIER	ACCOUNTS	BIGGEST REPORT	WHAT HAPPENS
T7	≤ 3,000	3,000 rows	Everything fits anywhere. This is where the product is correct today
T6	3–10,000	10,000 rows	Fine on the streaming path; the unpaginated endpoints start to hurt
T5	10–40,000	**~480,000 rows** — a year of read history	⚠ **The file is fine — it streams.** What breaks is that three of the nine datasets return everything in one response
T4 · T3	40–160,000	—	Explicitly later. The one requirement is that nothing here hard-caps, so the move does not need a rewrite

What breaks first is not in this module.

`GET /outstandings` is **unpaginated on purpose** and says so in its own docstring, naming *"a utility under 3,000 connections"* as the reason — **and that ceiling moved to 40,000 on 6 September**. It also **sorts in Python rather than in SQL**. Correct at T7, **13× short at T5**, filed as `DEF-PAY-4221` across four places plus two front-end mirrors.

32 of the 45 definitions read those three datasets. They can all be built, drawn and demonstrated on a small utility without it — and the moment a 40,000-account utility opens the Receivables group, three datasets return everything they have. **P2.12** asserts paging on all nine and **is red today on three**, which is the point: a dependency with a failing test beside it is a dependency; one described in a paragraph is a hope.

7.7 · Security and tenancy

RULE	HOW IT IS ENFORCED
Tenant isolation	RLS `FORCE` on every tenant table, `app.tenant_id` set per request. ⚠ **A dataset is a reviewed query precisely so a clerk-authored report cannot reach across it** — that is what "datasets are code" is protecting
Platform rows	`tenant_id IS NULL` is readable by every tenant and **writable by none**. A utility editing a shipped report gets a copy
Copy-on-write	The mechanism the whole framework rests on — drawn in **8.3**

Private by default	A saved report is visible to its author until shared. A half-finished report appearing on a colleague's screen is how people stop saving
Who may run what	The role that owns the data owns the report. A definition carries the permission of its dataset — not its own, or the two drift
Money on screen	Stored to 4 decimals, percentages to 6, **both displayed at 2**. ⚠ **One exception: rate history shows 4 and 6**, because trimming a tariff misstates it
Every export is audited	One `audit_log` row per run, naming who and over what parameters. That is also the *Started by* column on History
A premises statement crosses customers RULED	**Staff may. A landlord may not.** PO ruling, option (a): in the customer portal an owner sees **their own master account and nothing else** — at 1408 Elm Street, the **$69.60** of common-area charges, not Unit 3's $100.00. ⚠ **One case that is not a disclosure and needs no exception:** where the building bills the owner rather than the tenants, the owner *is* the account holder and sees it as their own account. Built by **P4.15**

7.8 · Integration and BI

Ruled by the PO: *"we will not open it directly - we might do the integration and provide apis"*.

So there is **no read replica and no database credential**, ever. A BI tool is a client of the same API a screen uses — which also means **a BI tool cannot see anything a signed-in person could not**, and RLS is doing the work rather than a second set of rules nobody maintains.

TIER	WHAT	WHEN	WHY THAT ORDER
1	**The report API itself** — every definition is already an endpoint. JSON, paginated, behind a **service-account token**. ⚠ **RULED: named reports only** — a connection granted *Revenue Analysis* and *Trial Balance* can read those two and nothing else; AR Ageing later means somebody grants it. **RULED: administrators create them.** Revocable, audited, rate-limited	Phase 2	**Near zero cost — the engine exists.** Tier 1 is deciding to expose what is already there, properly
2	**Scheduled delivery** — the scheduler already writes a file; send it to SFTP or a signed URL on a timer. ⚠ **RULED: yes, and with three conditions** — destinations are set up **by an administrator**, never by whoever creates the schedule; **encrypted in transit**; **every delivery logged**. ⚠ **This does not reopen** `BR-REP-020` , which says a schedule's	Phase 4	Rides on schedules. A BI tool that cannot call an API on a timer can still collect a file

	recipients are this tenant's own users and nothing else. That rule is about a **free-text email box on the schedule form** — anyone can type anything into it. This is about **a destination an administrator configured once**, which is a different act by a different person. The form still offers no free text		
3	**OData** — a metadata document, `$filter` / `$select` / `$top` / `$skip`, type mapping	`NAMED, NOT BUILT`	⚠ **OData is what Excel and Power BI speak natively.** Without it an analyst pastes a URL into a Web connector and hand-writes pagination; with it they click *Get Data → OData*. Named so that when a customer asks it is a change request with a figure, not a surprise
—	~~**Direct database access · a read replica**~~	**NEVER**	Ruled out. It bypasses RLS, freezes the schema into a public contract, and makes every migration a customer-visible breaking change

What a service account is not: a person's password in a BI config file.

That is how a clerk who leaves takes the reporting with them, how an audit log stops meaning anything, and how a revoked account silently breaks a board pack. A service account is **its own principal** — named, scoped to a tenant and **a named list of reports**, revocable without touching a person, and it appears in *Started by* on History like anyone else.

Who creates one, and who holds it — ruled, with one word I am reading rather than assuming. Administrators create them. The PO wrote *"a · Administrators only … M. Okonkwo creates it and hands Dana the token + clerk"*, and Dana is a billing clerk — so this is built as **an administrator issues it and any staff member, clerk included, may hold and use it**. ⚠ **If the intent was that a clerk may also CREATE one, say so and it is one line** — but it is a credential, so the narrower reading ships until then.

7.9 · The decision register

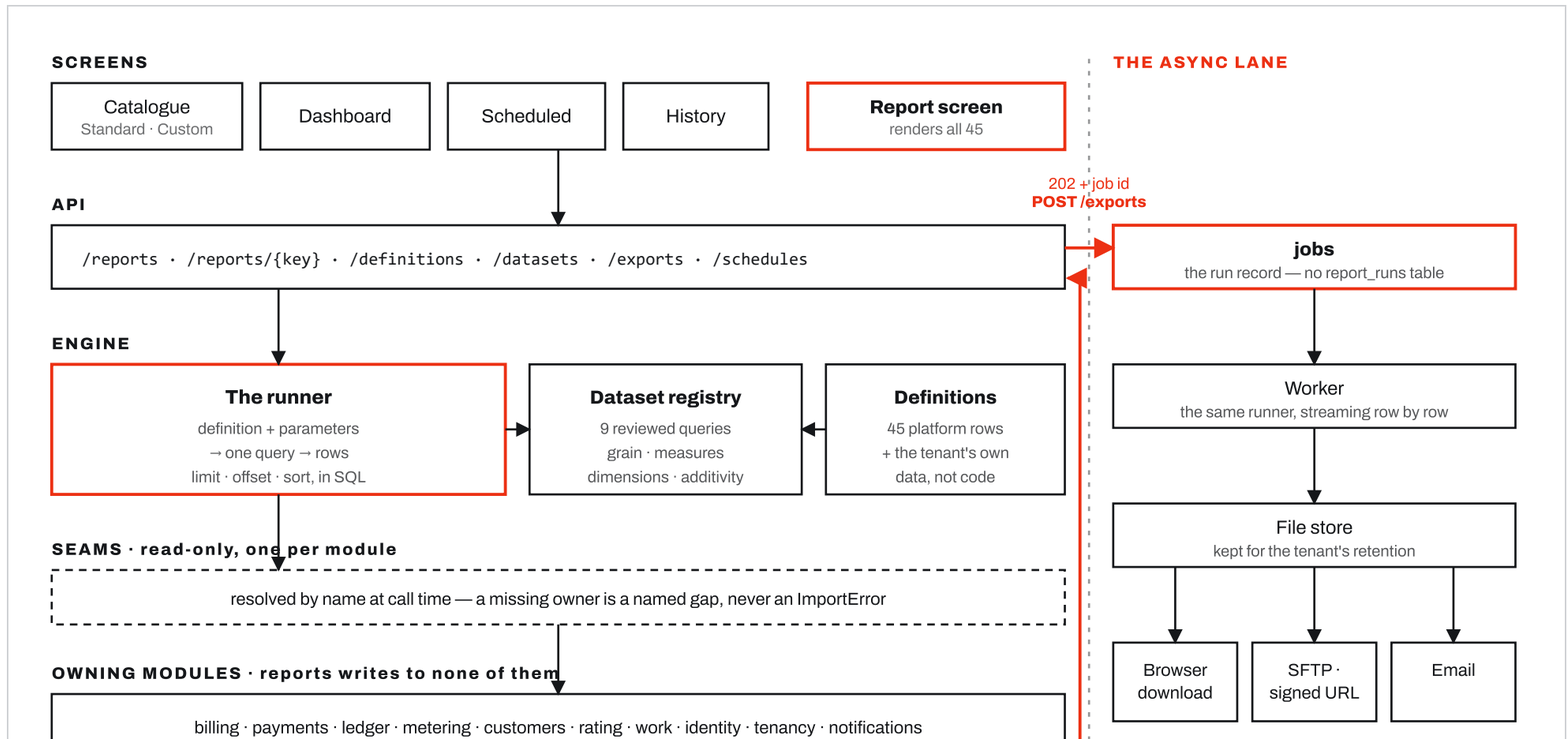
DECISION	STATUS	REVERSES ONLY IF
Datasets are code, definitions are data	`FINAL`	a utility needs a column no dataset has, often enough to outweigh the isolation guarantee
One screen renders every definition	`FINAL`	—
No `report_runs` — `jobs` is the run record	`FINAL`	a run needs state a job cannot hold
Sync page 1, async for files	`FINAL`	—
The row cap is deleted, not raised	`FINAL`	`NFR-5` changes
Four tabs, flat; Custom as a sub-tab	`FINAL`	PO ruling
The Reports dashboard is a new tab; the shell dashboard is untouched	`FINAL`	PO ruling — *"dont replace existing one"*
Statement and Ledger take a Subject; master is not a fifth value	`FINAL`	a master account stops being an ordinary row

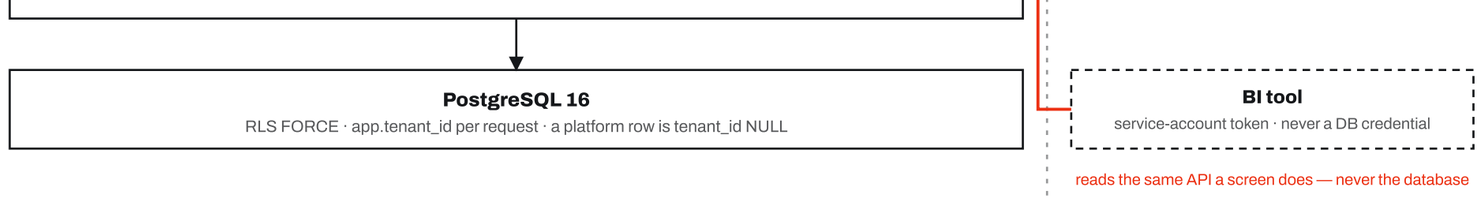
No direct database access for BI	FINAL	PO ruling
Dimensions declared per dataset, not per report	FINAL	—
Pivot · calculated measures · OData	NAMED	a customer asks — then a CR with a figure
As-was vs as-is	FINAL	Option **(b)** — history for class, type and status. ⚠ Reverses nothing: the past is already gone and stays gone
A landlord seeing a premises statement in the portal	FINAL	Option **(a)** — no. Their own master account only
What a BI connection may read	FINAL	Named reports only
Who may create a BI connection	FINAL	Administrators. ⚠ Whether a clerk may create one as well is the one word still to confirm
A scheduled file may leave the product	FINAL	Yes — admin-set destinations, encrypted in transit, every delivery logged

08 How it is built

Three drawings, not one. A single diagram that shows the components **and** the behaviour ends up showing neither — so this is the stack, then the two read paths side by side, then the one mechanism the whole framework rests on.

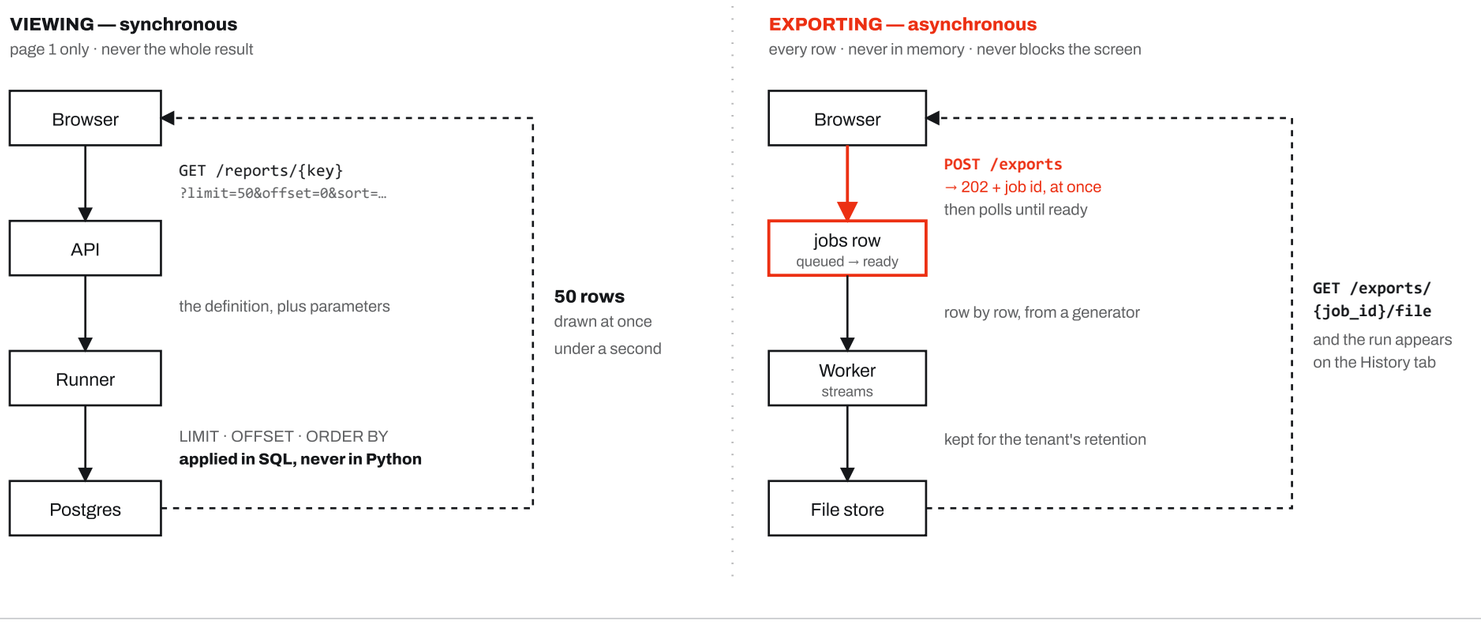
8.1 · The stack





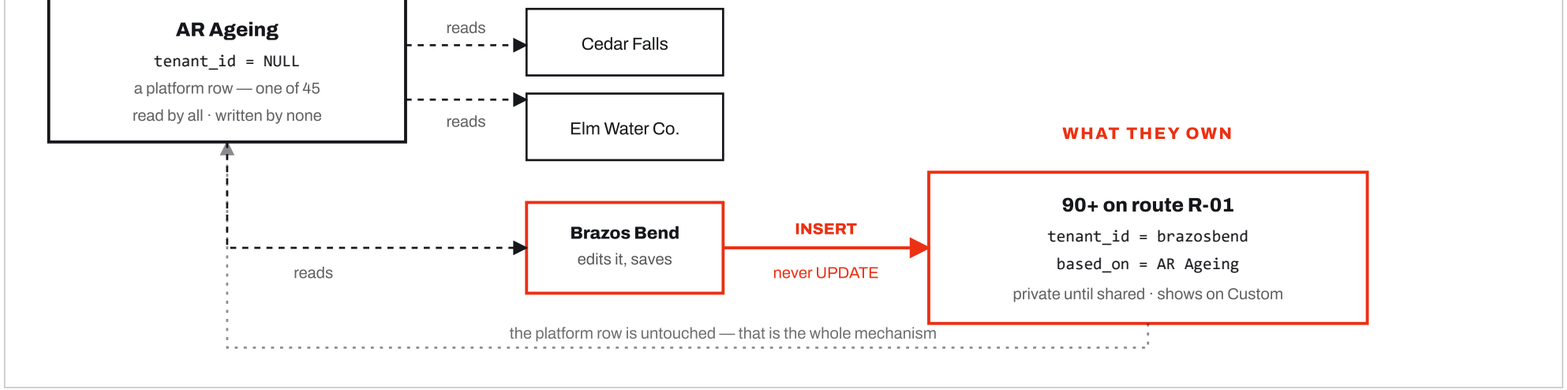
Every arrow into the database passes through a reviewed dataset. That is what makes a clerk-authored report safe: the definition chooses *from what a dataset offers*, and a dataset is code that went through review. ⚠ **Reports writes to no module's tables** — the only rows it creates are a job, a definition, a schedule and an audit entry. ⚠ **The BI tool holds a token, never a database credential**, so it sees exactly what a signed-in person could and no more — row-level security does that work, not a second set of rules nobody maintains.

8.2 · The two read paths, side by side



All three async calls already work today. What was missing is the screen at the end of them — the toast says a file *"appears in Jobs"* and there is nowhere to go. That is the History tab, and it is the cheapest screen in the plan. ⚠ **The left path never fetches more than a page; the right path never holds one.** Building either the other way is the mistake this drawing exists to prevent.

8.3 · Copy-on-write — how a utility changes a report we ship



This is the single mechanism the framework rests on, and the one most likely to be "simplified" away by a later lane. Editing a shipped report **never updates** the row we ship; it inserts a tenant row that remembers what it was based on. So we can improve AR Ageing in a release and Brazos Bend keeps theirs — and no utility is ever on a fork. ⚠ **The `based_on` column**